

BETTERCP/M 1.0
SYSTEM ARCHITECTURE GUIDE

BETA EDITION

HUMONGOUS CP/M ARCHIVES

2026

BETTERCP/M 1.0

SYSTEM ARCHITECTURE GUIDE

Beta Edition
September 2026

***HUMONGOUS* CP/M ARCHIVES**

TABLE OF CONTENTS

BETTERCP/M 1.0 SYSTEM ARCHITECTURE GUIDE

1. System Architecture

1.1 Architectural Principles.....	7
1.2 CP/M 2.2 Compatibility.....	8
1.3 Major System Components.....	9
1.4 Component Relationships.....	9
1.5 Architectural Boundaries.....	10
1.6 Control and Service Flow.....	11
1.7 Architectural Invariants.....	12

2. Memory Architecture

2.1 The BetterCP/M Memory Model.....	13
2.2 Address-Space Organization.....	14
2.3 The Transient Program Area.....	14
2.4 The Resident System.....	15
2.5 Compatibility-Visible Memory.....	15
2.6 Memory Ownership.....	16
2.7 ROM and RAM Ownership.....	16
2.8 Reclaimable and Persistent Memory.....	17
2.9 Memory Constraints and Invariants.....	17

3. Boot and System Initialization

3.1 Boot Model.....	19
3.2 Initial Machine State.....	19
3.3 System Loading.....	19
3.4 System Initialization.....	20
3.5 Cold Boot.....	21
3.6 Warm Boot.....	22
3.7 System Reconstruction.....	22
3.8 Boot and Recovery.....	23
3.9 Boot Interface Boundary.....	24

4. The Command Environment

4.1 Command Environment Model.....	25
4.2 The Console Command Processor.....	26
4.3 Command Input and Interpretation.....	26
4.4 Drive/User Navigation.....	27
4.5 Core Resident Commands.....	27
4.6 Command Processor Extensions.....	28
4.7 Transient Commands.....	29
4.8 Command Dispatch Order.....	29
4.9 Program Invocation.....	30
4.10 Command-Line Editing and History.....	30
4.11 SUBMIT and XSUB.....	31
4.12 Reentry and Restoration.....	31

5. Program Execution

5.1 The CP/M Execution Model.....	33
5.2 Transient Program Loading.....	34
5.3 Initial Program State.....	34
5.4 Page Zero.....	35
5.5 Command Tail and Default FCBs.....	35
5.6 Program Memory and the TPA.....	35
5.7 System-Service Access.....	36

5.8	Program Termination.....	37
5.9	Return to the Command Environment.....	37
5.10	Execution Compatibility.....	38
6.	BDOS and System Services	
6.1	System-Service Model.....	39
6.2	The BDOS Interface.....	40
6.3	Console and Character Services.....	40
6.4	File and Record Services.....	41
6.5	Drive and User Services.....	42
6.6	Program-Support Services.....	42
6.7	System State and Service State.....	43
6.8	Error and Return Behavior.....	43
6.9	BetterCP/M Extensions.....	44
6.10	Extended BDOS Namespace.....	44
6.11	Resident Services.....	45
6.12	Resident-Service Discovery.....	46
6.13	Service Address Lifetime.....	46
6.14	Compatibility and Service Extension.....	47
7.	Hardware Abstraction and the BIOS	
7.1	Hardware-Abstraction Model.....	48
7.2	Hardware-Independent and Hardware-Dependent Boundaries.....	49
7.3	Console and Character Devices.....	49
7.4	Storage Devices.....	50
7.5	Logical and Physical Storage.....	50
7.6	Hardware Initialization.....	51
7.7	Platform Capabilities and Configuration.....	52
7.8	The BIOS Interface.....	52
7.9	Platform Implementation.....	53
7.10	ROM and RAM Considerations.....	54
7.11	Hardware Compatibility.....	54
7.12	Portability Boundaries.....	55
8.	Storage and File-System Architecture	
8.1	Storage Model.....	56
8.2	Logical Drives.....	57
8.3	CP/M Disk Formats.....	57
8.4	File-System Model.....	58
8.5	Directory and Allocation.....	58
8.6	Records and Extents.....	59
8.7	Logical-to-Physical Translation.....	60
8.8	File Attributes.....	60
8.9	Media State and Media Changes.....	61
8.10	Storage State.....	61
8.11	Storage Compatibility.....	62
8.12	Storage Evolution.....	62
9.	Disk Formats and Disk State	
9.1	Physical Drives and Logical Bindings.....	64
9.2	Disk Geometry.....	65
9.3	Disk Format Definitions.....	65
9.4	Format Selection and Provenance.....	66
9.5	Format Validation.....	66
9.6	Normalized Disk State.....	67
9.7	Active and Saved Disk Configuration.....	67
9.8	Runtime Authority and Provenance.....	68

9.9	Disk Capabilities.....	69
9.10	Filesystem Working Memory.....	69
9.11	Media Observation.....	70
9.12	Configuration Changes.....	70
9.13	Unsupported Formats and Operations.....	71
9.14	Disk-State Invariants.....	71
10.	System State and Configuration	
10.1	System-State Model.....	72
10.2	State Ownership.....	72
10.3	Global System State.....	73
10.4	Command and Program State.....	73
10.5	Storage State.....	74
10.6	Configurable System State.....	74
10.7	Configuration States.....	75
10.8	State Lifetime.....	76
10.9	State Transitions.....	76
10.10	Active, Pending, and Saved Configuration.....	77
10.11	Device Configuration.....	78
10.12	Disk Configuration.....	78
10.13	Resident Startup Profiles.....	79
10.14	Startup Command.....	79
10.15	Configuration Ownership.....	80
10.16	Configuration Recovery.....	80
11.	Persistent System State	
11.1	Persistent-State Model.....	83
11.2	The Persistent Data Segment.....	83
11.3	BetterCP/M 1.0 PDS Layout.....	84
11.4	Command-History State.....	84
11.5	Cold-Boot Initialization.....	85
11.6	Warm-Boot Preservation.....	85
11.7	Validation and Recovery.....	86
11.8	PDS Discovery.....	87
11.9	PDS Ownership Exclusions.....	87
11.10	Persistent State and RSXs.....	88
11.11	Persistent State and ROM Systems.....	88
11.12	PDS and the TPA.....	89
11.13	Shared Workspaces Are Not Persistent State.....	89
11.14	BetterCP/M 1.0 Persistence Boundary.....	90
12.	Extension Architecture	
12.1	Extension Model.....	91
12.2	Extension Principles.....	92
12.3	CPXs and RSXs.....	92
12.4	Extension Profiles.....	93
12.5	Extension Loading and Validation.....	93
12.6	Extension Ordering.....	94
12.7	Resource Accounting.....	94
12.8	Transactional Configuration.....	95
12.9	Failure and Rollback.....	96
12.10	Extension Discovery and Identification.....	96
12.11	Extension State.....	97
12.12	Extension Compatibility.....	97
12.13	Extension Recovery.....	98
12.14	BetterCP/M 1.0 Extension Boundary.....	98

13. Command Processor Extensions	
13.1 CPX Architecture.....	100
13.2 BCPX Modules.....	101
13.3 Command Enumeration.....	102
13.4 Command Dispatch and Chaining.....	102
13.5 Initialization and Shutdown.....	103
13.6 CPX Profiles.....	103
13.7 Loading and Unloading CPXs.....	104
13.8 Warm-Boot Reconstruction.....	105
13.9 CPX State.....	106
13.10 CPX Memory Use.....	106
13.11 The Standard RCP.CPX.....	107
13.12 CPX Configuration and Startup.....	108
13.13 CPX Interface Conventions.....	108
13.14 CPX Compatibility and Limits.....	109
14. Resident System Extensions	
14.1 RSX Architecture.....	111
14.2 The RSX Chain.....	112
14.3 BRSX Modules.....	113
14.4 RSX Profiles.....	114
14.5 Reconstruction Classes.....	114
14.6 Stateful RSXs.....	115
14.7 Relocation and Pointer Repair.....	116
14.8 State Preservation.....	117
14.9 Loading and Unloading RSXs.....	117
14.10 Transactional Reconstruction.....	118
14.11 Failure and Rollback.....	119
15. Resident Services	
15.1 Resident-Service Model.....	121
15.2 Service Identification.....	122
15.3 Service Advertisements.....	122
15.4 The Resident-Service Registry.....	123
15.5 Service Publication.....	124
15.6 Service Discovery.....	124
15.7 Enumeration and Registry Description.....	125
15.8 Service Versioning.....	126
15.9 Service Calling Conventions.....	126
15.10 Service Address Lifetime.....	127
15.11 Reconstruction and Rebinding.....	128
15.12 Service Removal.....	128
15.13 Failure and Registry Integrity.....	129
15.14 BetterCP/M 1.0 Service Boundary.....	129
16. Date and Time Architecture	
16.1 Clock Architecture.....	131
16.2 The TIME Service.....	132
16.3 Canonical Clock Record.....	132
16.4 Reading the Clock.....	133
16.5 Setting the Clock.....	133
16.6 Hardware Clock Providers.....	134
16.7 Provider Discovery.....	135
16.8 Native Consumers.....	135
16.9 P2DOS Compatibility.....	136
16.10 P2DOS Result Behavior.....	136

16.11 Read-Only Providers.....	137
16.12 Other Historical Clock Interfaces.....	137
16.13 Clock Data and Calendar Conversion.....	138
16.14 Clock Services and File Timestamps.....	139
16.15 Date and Time Invariants.....	139
17. Compatibility Architecture	
17.1 Compatibility Model.....	141
17.2 Application Compatibility.....	142
17.3 BDOS Compatibility.....	143
17.4 Command-Environment Compatibility.....	143
17.5 Memory and Execution Compatibility.....	144
17.6 File-System and Disk Compatibility.....	144
17.7 BIOS and Hardware Compatibility.....	145
17.8 Documented and De Facto Behavior.....	146
17.9 Implementation Freedom.....	147
17.10 Compatibility Testing.....	147
17.11 BetterCP/M Extensions and Compatibility.....	148
17.12 Compatibility Limits.....	149
17.13 Distribution Utility Compatibility.....	149
18. Architectural Constraints and Evolution	
18.1 System Invariants.....	151
18.2 Memory and Resource Constraints.....	152
18.3 Interface Stability.....	152
18.4 Compatibility Constraints.....	153
18.5 Extension Constraints.....	154
18.6 Simplicity and Comprehensibility.....	154
18.7 BetterCP/M 1.0 Architectural Boundary.....	155
18.8 Deferred Facilities.....	156
18.9 Future Architectural Evolution.....	156
18.10 Architectural Change Process.....	157
18.11 Long-Term Compatibility.....	158
18.12 Architectural Invariants Summary.....	158
Appendixes	
Appendix A. BetterCP/M 1.0 Memory Map.....	160
Appendix B. System Component Summary.....	161
Appendix C. Boot and Reconstruction Sequence.....	162
Appendix D. Command Dispatch Sequence.....	164
Appendix E. BDOS and Extended-Service Summary.....	165
Appendix F. CPX and RSX Summary.....	166
Appendix G. System-State Ownership Summary.....	168
Appendix H. Architectural Invariants.....	170
Appendix I. Glossary.....	172

1. System Architecture

BetterCP/M is a small, single-user, single-tasking operating system for Z80-based computers. It provides an operating environment compatible with CP/M 2.2 while using a new implementation designed specifically for the Z80 and for continued development beyond the original CP/M system.

BetterCP/M preserves the parts of CP/M that are visible to programs and users without requiring its internal implementation to duplicate Digital Research CP/M. A CP/M program sees the familiar memory environment, system-service interface, file system, command conventions, and program-execution model. Behind those interfaces, BetterCP/M is free to organize the operating system differently where doing so improves clarity, portability, extensibility, or implementation.

The system remains deliberately small. BetterCP/M does not attempt to place a modern operating-system architecture behind a CP/M compatibility layer. Additional abstraction and generality are introduced only where they solve an identified problem or establish a useful architectural boundary.

Conceptually, BetterCP/M consists of a command environment, system services, hardware abstraction, program execution environment, storage and file-system facilities, and system state. These components describe responsibilities rather than necessarily corresponding to separate binaries or memory regions.

1.1 Architectural Principles

BetterCP/M begins with CP/M rather than with a new operating-system model.

Its initial specification is the environment presented by CP/M 2.2: the interfaces, conventions, data structures, memory locations, file-system behavior, command behavior, and other observable characteristics upon which CP/M software depends. The implementation of those characteristics is not itself part of the compatibility requirement unless software can observe or depend upon it.

This distinction permits BetterCP/M to improve the mechanisms behind the CP/M environment without unnecessarily changing the environment itself.

Several principles guide the architecture:

Compatibility. Existing CP/M 2.2 software should run without BetterCP/M-specific modification except where a program depends upon particular hardware or implementation details outside the ordinary CP/M interface.

Native Z80 implementation. BetterCP/M is designed for the Z80. Intel 8080 compatibility is not an implementation requirement.

Small system size. Resident memory is a limited resource because memory occupied by the operating system reduces the Transient Program Area available to applications. New resident facilities must therefore justify their cost.

Simplicity and comprehensibility. BetterCP/M favors explicit, readily understood mechanisms over unnecessary layers or generalized frameworks. Important control flow, state, ownership, and interfaces should be possible to trace through the system.

Defined architectural boundaries. Major responsibilities are separated where the separation clarifies ownership, permits independent implementation, or

protects an interface. Architectural separation does not imply that each component must occupy a separate binary, source file, or memory region.

Data-driven configuration. Machine characteristics and system policy that can reasonably vary without rebuilding the operating system are represented as configuration data where practical.

Implementation freedom. Compatibility with CP/M behavior does not require reproduction of Digital Research's private implementation.

Controlled extensibility. The architecture avoids unnecessarily preventing foreseeable improvements, but BetterCP/M does not carry speculative mechanisms merely because they might someday be useful.

These principles are intended to produce a practical operating system rather than a pedagogical model. BetterCP/M is nevertheless designed to remain sufficiently clear that its architecture and implementation can also be studied as a Z80 operating system.

1.2 CP/M 2.2 Compatibility

CP/M 2.2 defines the basic compatibility environment for BetterCP/M 1.0.

Compatibility is principally behavioral. BetterCP/M preserves characteristics visible to CP/M programs while distinguishing them from details that belonged only to Digital Research's implementation.

Compatibility-visible characteristics include such things as:

- system-call entry mechanisms and calling conventions;
- service semantics and return values;
- required memory locations, vectors, and data structures;
- transient-program loading and execution conventions;
- command tails and default FCBs;
- file-system behavior;
- command-environment behavior; and
- termination and error behavior upon which existing software depends.

The published Digital Research documentation establishes the documented CP/M interface. Observed behavior and the original implementation can provide additional evidence, particularly where existing software has come to depend upon behavior that was not fully documented.

This does not mean that every characteristic found in the original CP/M source becomes a BetterCP/M requirement. An implementation-private mechanism need not be reproduced merely because Digital Research used it.

BetterCP/M therefore distinguishes among documented CP/M behavior, observable implementation behavior, architectural consequences of that behavior, and implementation-private details.

Where BetterCP/M adds facilities, those facilities are designed not to silently redefine the established CP/M environment. An extension that cannot preserve CP/M-compatible behavior must define the resulting difference explicitly rather than quietly changing the compatibility contract.

Compatibility is discussed in detail in Chapter 17.

1.3 Major System Components

The BetterCP/M architecture can be viewed as seven major components:

Boot system. Establishes the initial machine state, loads or initializes the operating system, applies the required initial configuration, and transfers control to the running system.

Command environment. Provides the normal interactive interface. It displays the command prompt, accepts and interprets commands, performs resident command functions, dispatches commands supplied by Command Processor Extensions, and initiates transient programs.

System services. Provide operating-system functions to applications and to other BetterCP/M components. These include console and character services, file and record operations, drive and user-area operations, program-support services, and access to operating-system state.

Hardware abstraction. Contains machine-dependent operations and provides the boundary through which the hardware-independent system interacts with console devices, storage hardware, clocks, and other platform facilities.

Program execution environment. Establishes the CP/M-compatible conditions under which transient programs execute, including memory organization, program entry state, access to system services, command information, termination, and return to the command environment.

Storage and file-system subsystem. Implements the logical CP/M storage environment while separating that environment, where practical, from the physical characteristics of the underlying media and devices.

System-state subsystem. Comprises operating-system information that persists across individual service calls or program execution and represents the current operating environment.

These are architectural components rather than necessarily physical modules. Two components can share code, data, source files, or memory while retaining distinct responsibilities.

1.4 Component Relationships

BetterCP/M components interact through defined interfaces and ownership boundaries.

The Command Environment is responsible for interpreting commands, but it does not privately implement general operating-system functions merely because a command needs them. File operations, console operations, storage services, and other system functions are obtained through the appropriate system facilities.

Transient applications execute within the Program Execution Environment and request operating-system services through the CP/M-compatible system-service interface.

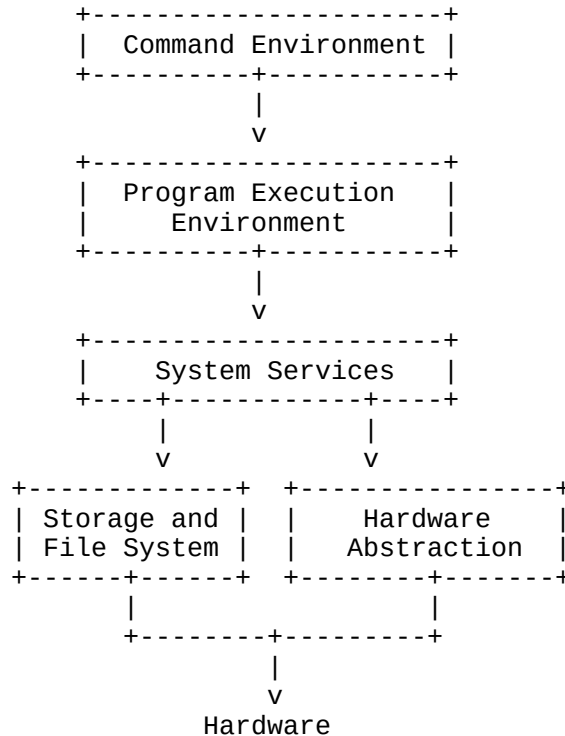
System Services provide hardware-independent operating-system behavior. When a service requires machine-specific operation, it crosses the Hardware-Abstraction boundary rather than embedding knowledge of a particular machine throughout the operating system.

The Storage and File-System subsystem translates CP/M-visible file and record operations into the logical and physical storage operations required to perform

them. Physical device access ultimately occurs through the appropriate hardware-dependent interface.

System state has defined ownership. Components may examine or modify shared state only through the interfaces appropriate to that state. BetterCP/M avoids maintaining multiple authoritative copies of the same system information where those copies could diverge.

The resulting organization can be summarized conceptually as:



System State participates across these relationships rather than forming a simple layer in the control path. The component that owns a particular item of state defines how that state is accessed and how long it remains valid.

Later chapters refine this high-level model.

1.5 Architectural Boundaries

BetterCP/M separates responsibilities where doing so establishes a useful boundary.

The **Command Environment** owns command acquisition, interpretation, resident-command dispatch, CPX command dispatch, transient-program initiation, and restoration of the interactive environment after a program terminates. It does not own general file, console, or storage services simply because commands use them.

System Services own hardware-independent operating-system operations exposed to applications and other components. They provide the CP/M-compatible service environment without requiring the internal organization of the Digital Research BDOS to be reproduced.

Hardware Abstraction owns operations whose implementation depends upon the target machine. Hardware-independent code should not need direct knowledge of I/O ports, controller protocols, device registers, physical disk geometry, or other machine-specific mechanisms unless an explicit requirement makes such knowledge unavoidable.

Applications remain outside the operating system. They own their program-specific behavior and data and use BetterCP/M through the CP/M-compatible execution and service interfaces.

An architectural interface need not imply an elaborate runtime abstraction. On a small Z80 system, a boundary can be represented by an assembly-language calling convention, entry point, table, or documented shared structure. BetterCP/M does not introduce indirection merely to make a conceptual boundary physically elaborate.

The important questions are instead:

- which component owns the responsibility;
- which components may request its services;
- what information crosses the boundary;
- which state can be examined or changed;
- what conditions hold before and after the operation; and
- whether any part of the interface is visible to CP/M software.

Where an interface is compatibility-visible, CP/M compatibility takes precedence over BetterCP/M's preferred internal organization.

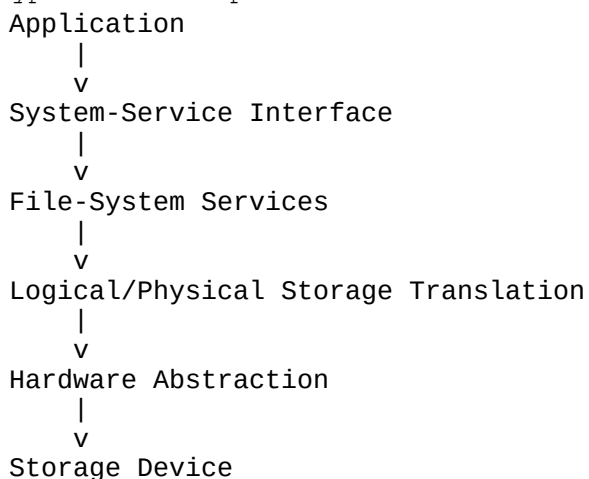
1.6 Control and Service Flow

At any given time, BetterCP/M 1.0 executes either operating-system code, the command environment, or one transient application. It does not provide multitasking between applications.

After initialization, control normally rests with the Command Environment. The user enters a command, which can be handled directly by the core command processor, by an installed CPX, or by loading a transient program.

When a transient program is selected, BetterCP/M establishes the CP/M-compatible execution environment and transfers control to the program. The application requests operating-system functions through the conventional system-service interface.

A typical file operation therefore follows a path conceptually similar to:



Console and other device operations follow analogous paths.

When the application terminates, BetterCP/M reestablishes the Command Environment and returns to the command-processing cycle. Parts of the command environment need not remain resident while the transient program executes; they can be reconstructed when control returns.

Boot, warm boot, CPX reconstruction, and RSX reconstruction add additional control paths, discussed in Chapters 3, 12, 13, and 14.

1.7 Architectural Invariants

Several properties define the fundamental boundaries within which BetterCP/M 1.0 operates.

The system remains **single-user and single-tasking**. At most one transient application executes at a time.

A CP/M-compatible application receives the required CP/M execution environment and does not need to know BetterCP/M's private internal organization.

Compatibility-visible interfaces retain their required CP/M semantics. BetterCP/M extensions do not silently redefine existing CP/M behavior.

Every significant memory region has a defined purpose and lifetime. Resident memory is treated as a constrained resource because it reduces the TPA available to applications.

Hardware-independent components do not unnecessarily depend upon machine-specific implementation details.

The logical CP/M storage environment remains distinct from the physical devices used to implement it. Logical disk geometry is not assumed to be identical to physical device geometry.

Every item of persistent system state has a defined owner, purpose, and lifetime. Shared state has one authoritative value rather than independent authoritative copies maintained by multiple components.

State remains valid only while the assumptions on which it depends remain valid. A media change, configuration change, extension reconstruction, or other state transition invalidates or reconstructs state whose assumptions no longer hold.

Architectural separation does not require expensive abstraction. BetterCP/M favors simple, explicit mechanisms where they preserve the same responsibility and compatibility boundaries.

Finally, BetterCP/M 1.0 does not contain speculative machinery merely to support possible future facilities. Future extensions are expected to respect the established compatibility and architectural boundaries or to define explicitly where a new environment differs from them.

These invariants recur throughout the remainder of this manual.

2. Memory Architecture

BetterCP/M executes within the ordinary 16-bit Z80 address space. As in CP/M 2.2, memory is shared between the operating system and the currently executing transient program. Memory occupied by resident system components is therefore memory unavailable to applications, making memory use one of the central constraints on the architecture.

BetterCP/M preserves the memory organization visible to CP/M software while allowing the internal arrangement of the operating system to differ from Digital Research CP/M. Page-zero vectors, the Transient Program Area, system-service entry points, and other compatibility-visible relationships retain their required meanings. BetterCP/M-private code and data can be organized according to the needs of the system.

The default BetterCP/M 1.0 configuration provides at least 53 KiB of usable TPA. Optional resident facilities can consume additional memory, but their cost is explicit and reflected in the TPA available to applications.

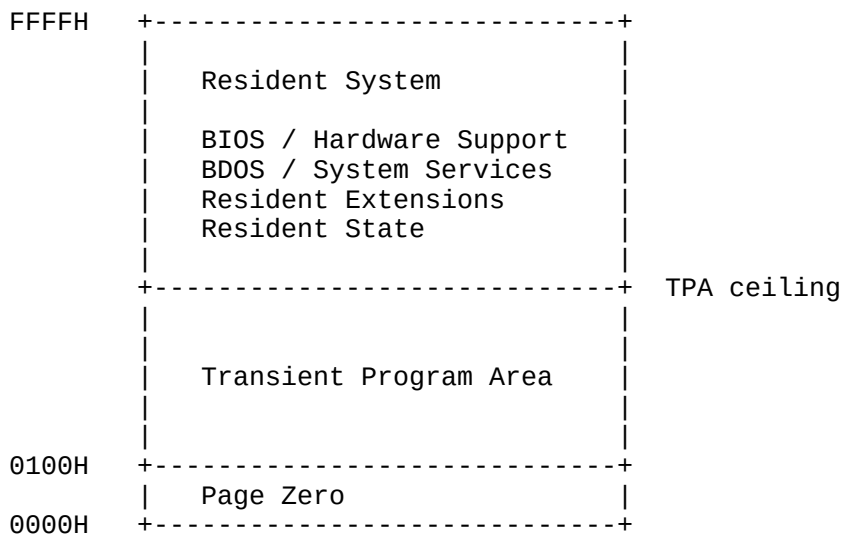
BetterCP/M also distinguishes between memory according to ownership and lifetime. Immutable system code, persistent system state, extension state, stacks, temporary workspaces, and transient-program memory have different requirements even when they occupy adjacent areas of the same address space.

2.1 The BetterCP/M Memory Model

BetterCP/M 1.0 uses a single 64 KiB Z80 address space.

The lower portion of memory contains page zero followed by the Transient Program Area. The upper portion contains the resident operating system and any resident extensions. The boundary between transient and resident memory determines the maximum memory available to an application.

Conceptually:



The exact internal boundaries within the resident system are BetterCP/M implementation details except where an address or relationship forms part of a documented interface.

The TPA ceiling can change when resident facilities are added or removed. BetterCP/M publishes the current usable boundary so that the execution environment reflects the configuration actually in use.

No bank switching, virtual memory, memory protection, or multiple simultaneous application address spaces are required by the BetterCP/M 1.0 memory model.

2.2 Address-Space Organization

The Z80 provides a 16-bit address space extending from 0000H through FFFFH.

BetterCP/M divides this address space according to responsibility rather than treating memory as a single undifferentiated pool.

The principal areas are:

Page zero. The first 256 bytes contain the CP/M-compatible vectors and program-interface information expected by transient applications.

Transient Program Area. Begins at 0100H and extends upward to the boundary published by the resident system. Transient .COM programs and their working memory occupy this area.

Resident system. Occupies upper memory and contains the operating-system code and state that must remain available while a transient program executes.

Resident extensions. RSXs consume protected resident memory. Their presence can lower the TPA ceiling. CPXs belong to the reclaimable command environment and are treated differently, as described later in this chapter.

The physical organization of individual operating-system components is not itself the architectural boundary. Code and data belonging to different conceptual subsystems can be adjacent or combined where this reduces size without obscuring ownership or interface requirements.

2.3 The Transient Program Area

The Transient Program Area, or TPA, is the memory available to ordinary CP/M programs.

It begins at address 0100H. Its upper boundary is determined by the current resident-system configuration and is advertised through the conventional CP/M page-zero interface.

The default BetterCP/M 1.0 configuration retains at least **53 KiB of usable TPA**.

BetterCP/M's default record-aligned transient-program ceiling is 54,272 bytes. A .COM image that fits within this boundary can be loaded without overlapping the resident system; an image that exceeds the available TPA must be rejected rather than allowed to overwrite resident memory.

The TPA is not persistent operating-system storage. While an application is executing, it owns the application portion of this memory. When that application terminates, BetterCP/M is free to reuse the area.

This principle permits part of the command environment to occupy memory while the system is at the command prompt and then be discarded or overwritten when a transient program executes. The command environment can subsequently be reconstructed.

The amount of available TPA is therefore determined by what must remain resident during application execution, not simply by everything that happens to be loaded while the command prompt is active.

2.4 The Resident System

The resident system consists of the code and state that must remain available while a transient program owns the TPA.

This includes the fixed operating-system core, the system-service environment, required hardware-dependent facilities, protected resident state, and installed RSXs.

Resident memory is expensive. Every additional byte that must remain above the TPA while an application executes can reduce the largest program BetterCP/M can run.

BetterCP/M therefore distinguishes carefully between facilities that must be resident and facilities that merely need to be available at the command prompt.

Command Processor Extensions illustrate this distinction. A CPX provides resident commands from the user's point of view, but it does not need to remain resident while an ordinary transient application executes. The CPX and the reloadable portion of the command environment can therefore occupy reclaimable memory and be reconstructed when the application terminates.

An RSX is different. It extends the resident system-service environment and must remain available while applications execute. Its memory consequently reduces the TPA.

This distinction allows BetterCP/M to provide a richer command environment without imposing the entire command environment as a permanent cost on transient programs.

2.5 Compatibility-Visible Memory

Some memory locations and relationships are visible to CP/M applications and therefore form part of the compatibility environment.

Page zero is the most obvious example. Compatible programs expect the conventional CP/M vectors and program state to appear there. The BDOS entry mechanism, command information, default FCBs, DMA conventions, and other documented or deliberately adopted CP/M interfaces must remain available in their expected forms.

The upper boundary of application memory is likewise compatibility-visible. Applications must be prevented from overwriting the resident operating system, and software that determines the available TPA through the conventional CP/M mechanism must receive a value corresponding to the actual current configuration.

BetterCP/M can reorganize memory that is private to the operating system, but such reorganization must not invalidate the memory environment promised to applications.

This distinction is fundamental throughout BetterCP/M: **addresses are fixed where compatibility or a defined ABI requires them to be fixed; otherwise, ownership and interface are more important than reproducing historical placement.**

2.6 Memory Ownership

Every BetterCP/M memory region has an owner, a purpose, and a lifetime.

Ownership determines which subsystem controls the contents and under what circumstances those contents remain valid. The mere continued presence of bytes in memory does not make them valid system state.

BetterCP/M distinguishes several broad classes of memory:

Immutable system code and data. Code and constant data that do not change during operation.

Persistent system state. Mutable information that is explicitly required to survive particular system transitions.

Fixed subsystem state. Mutable state owned by a resident subsystem and retained for as long as that subsystem remains active.

Stacks. Runtime storage whose size and ownership must be defined and whose capacity must be sufficient for its actual call paths.

Temporary workspaces. Scratch storage needed only during particular operations. Workspaces can share physical storage where their lifetimes are proven not to overlap.

Command-environment memory. Memory occupied by the CCP and CPXs while the command environment is active but reclaimable when a transient program executes.

Transient-program memory. Memory owned by the current application and invalid as operating-system storage once application ownership ends.

These distinctions prevent accidental dependencies on whatever data happens to remain at a particular address after an operation has finished.

2.7 ROM and RAM Ownership

BetterCP/M permits system configurations in which immutable operating-system code executes directly from ROM.

The BetterCP/M 1.0 ROM qualification profile is a genuine execute-in-place model: immutable system code resides in enforced write-protected memory and executes there rather than being copied into RAM and executed from the copy.

This requires a strict distinction between immutable and mutable objects.

Executable code and constant descriptors can reside in ROM. Mutable system state cannot. Configuration state, stacks, extension profiles, persistent state, transaction state, and working storage must reside in writable RAM under the ownership of the appropriate subsystem.

Where mutable state is initialized from constant values, BetterCP/M can retain an immutable template in ROM and construct the live instance in RAM.

Runtime operation must not depend upon patching executable code or immutable descriptors. This applies not only during normal service calls but also during boot, warm boot, extension reconstruction, and configuration changes.

The ROM/RAM distinction is therefore an ownership rule rather than merely a placement decision:

Immutable objects may live in ROM; live mutable objects belong in RAM.

The same architecture remains applicable when both happen to reside in RAM on a particular platform.

2.8 Reclaimable and Persistent Memory

Memory lifetime in BetterCP/M does not correspond simply to whether something is described as "resident."

The command environment demonstrates reclaimable memory. The CCP and configured CPXs must be available while the user is entering commands, but they need not occupy application memory while a transient program executes. BetterCP/M can reconstruct them after program termination or warm boot.

RSXs occupy protected resident memory because they provide services during application execution. Their current memory addresses, however, are not necessarily permanent. Changing the resident-extension configuration can reconstruct the resident chain and move extensions to new addresses.

Persistent information must therefore not depend casually upon current runtime addresses.

BetterCP/M 1.0 also contains a small Persistent Data Segment, or PDS. The PDS is fixed at 192 bytes and is used only for the built-in command-history facility in the 1.0 architecture. It survives the transitions for which its state is defined to persist, while a true cold boot initializes it to a valid empty state.

The PDS is not a general resident-memory allocator. Extension profiles, disk configuration, service-registry state, stacks, and workspaces remain under their natural owners rather than being placed into the PDS merely because they persist for some period.

Persistent state is discussed in detail in Chapter 11.

2.9 Memory Constraints and Invariants

Several rules govern memory use throughout BetterCP/M.

The CP/M-compatible application environment must remain valid regardless of BetterCP/M's private internal organization.

The TPA begins at **0100H**, and its upper boundary must accurately describe the memory available to the current transient program.

Resident components must not overlap the TPA, hardware-mapped regions, one another, or other memory whose ownership is incompatible with their use.

Persistent operating-system state must not be left in application-owned memory after application ownership ends.

Mutable live state must reside in writable memory. A ROM configuration cannot qualify as execute-in-place merely by copying the nominal ROM system into RAM and running it there.

Runtime addresses of reconstructible components are not persistent identities. CPXs and RSXs that can move during reconstruction must not expose private addresses as though those addresses remained permanently valid.

Temporary workspaces can share storage only where their lifetimes are known not to overlap. Memory saving by overlay is therefore based on demonstrated lifetime rather than convenient assumption.

Stacks and workspaces must be sized from actual requirements. A memory layout is not correct merely because its regions do not overlap statically; runtime stack growth and temporary use must also remain within their assigned bounds.

Finally, resident memory remains a deliberately constrained resource. BetterCP/M can add facilities beyond CP/M 2.2, but the cost of those facilities must remain visible. The default system preserves at least 53 KiB of usable TPA, and optional resident functionality must justify any reduction it imposes.

These rules allow BetterCP/M to extend CP/M while preserving one of the original system's most important practical characteristics: most of the machine remains available to the program the user actually wants to run.

3. Boot and System Initialization

Booting BetterCP/M establishes a complete, internally consistent operating environment from a machine state in which that environment cannot yet be assumed to exist.

The boot process is therefore more than loading the operating-system image and transferring control to it. BetterCP/M must establish the hardware-dependent environment, initialize system state, construct the configured command and extension environment, establish valid CP/M compatibility state, and only then enter normal command processing.

BetterCP/M distinguishes a **cold boot** from a **warm boot**. A cold boot establishes a new operating-system instance and initializes state according to the saved system configuration. A warm boot returns control to an already running system and preserves state whose defined lifetime extends across that transition.

The distinction is important because BetterCP/M contains reconstructible components. The command environment can be discarded while a transient program executes and rebuilt when control returns. CPXs and RSXs also have defined reconstruction behavior. Boot and reconstruction are therefore closely related, but they are not interchangeable operations.

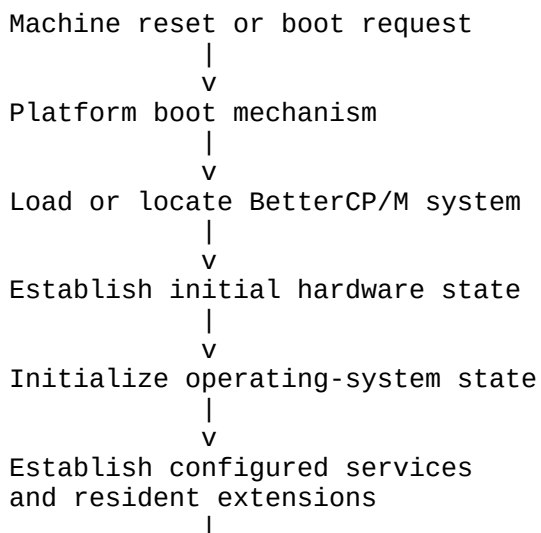
3.1 Boot Model

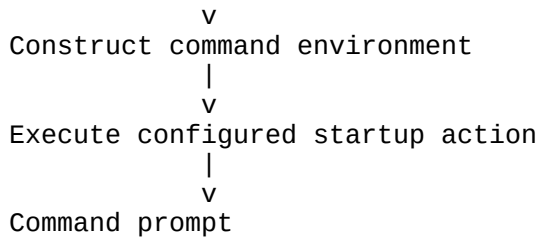
BetterCP/M separates the mechanics of obtaining the operating-system image from the initialization of the operating system itself.

The earliest boot operations are necessarily platform-dependent. Different machines can obtain BetterCP/M from different devices and can begin execution under different initial hardware conditions. The boot mechanism must perform whatever machine-specific work is necessary to make the system image available and establish the minimum environment required by the operating system.

Once that boundary has been crossed, BetterCP/M initialization becomes substantially hardware-independent.

Conceptually, a cold start proceeds through the following stages:





Not every platform must implement these stages with separate programs or entry points. They describe responsibilities rather than requiring a particular loader organization.

3.2 Initial Machine State

BetterCP/M cannot assume that a machine entering the boot process already satisfies the runtime conditions expected by the operating system.

The platform boot code is responsible for establishing the initial machine state needed to begin BetterCP/M initialization. This can include processor state, memory availability, hardware initialization, boot-device access, and other machine-specific requirements.

Hardware-dependent boot code should do only the work that actually depends upon the platform. It should not become an alternative implementation of hardware-independent operating-system initialization.

Once BetterCP/M assumes control, the system establishes its own runtime state rather than depending upon incidental values left by firmware, a previous operating system, or an earlier execution environment.

A cold boot consequently establishes a known starting condition. State that is defined to begin empty or initialized on a cold boot is initialized regardless of whether apparently usable values happen to remain in memory.

3.3 System Loading

The boot mechanism must make the BetterCP/M system image available in the memory locations required for initialization.

The exact method is platform-dependent. A system can be loaded from disk or supplied through another platform-specific mechanism. A ROM configuration can already contain immutable portions of the operating system in their execution locations.

BetterCP/M does not require every platform to use the same physical boot procedure. Instead, the boot boundary defines the state that must exist when hardware-independent system initialization begins.

The native system image is represented by **SYSTEM.SYS**. The system-building process constructs this image from the BetterCP/M system components; installation places it or its constituent system areas onto suitable boot media as required by the target platform.

The distinction between **building**, **installing**, and **booting** is intentional:

- building constructs a valid BetterCP/M system image;
- installation places that system on suitable boot media;

- booting uses the installed system to establish a running BetterCP/M environment.

The internal build and installation procedures are described in the appropriate development and system-generation documentation rather than in the boot architecture itself.

3.4 System Initialization

After the system has been loaded or otherwise made available, BetterCP/M initializes the operating environment.

Initialization establishes the fixed operating-system core, hardware-dependent facilities, system-service state, storage state, configured resident facilities, compatibility-visible state, and the initial command environment.

The resulting state must be internally consistent before it is published as the live system.

This requirement is particularly important for facilities whose configuration affects memory layout. BetterCP/M does not construct part of a new resident configuration, expose it to the running system, and then continue modifying it in place. Where configuration requires reconstruction, the prospective configuration is validated before becoming authoritative.

Initialization also establishes the CP/M-visible environment. Page-zero vectors, the system-service entry mechanism, current drive/user state, DMA state, and other required compatibility-visible information must be valid before ordinary applications can execute.

Platform initialization and operating-system initialization remain separate responsibilities. Hardware-specific code establishes and exposes machine capabilities; BetterCP/M establishes the operating-system policy and state that use those capabilities.

3.5 Cold Boot

A cold boot creates a new BetterCP/M runtime instance.

It initializes system state according to the saved cold-boot configuration rather than attempting to preserve the previous running environment. State whose lifetime begins with a cold boot is created afresh.

Among the important consequences of a cold boot are:

- the saved system configuration is read and applied;
- the configured disk environment is established;
- resident extension profiles are constructed;
- the command environment is initialized;
- compatibility-visible system state is established;
- the built-in command history begins empty; and
- the configured startup command, if any, can be executed.

Cold boot is therefore the transition at which saved configuration becomes the active initial configuration.

BetterCP/M distinguishes saved configuration from state that happened to be active before the boot. A warm boot does not silently reload cold-boot defaults, while a true cold boot deliberately establishes a new runtime environment from them.

The startup command, when configured, executes once after cold initialization and after the services required to execute it are available. It is not repeatedly invoked by warm boot or ordinary command-environment reconstruction.

3.6 Warm Boot

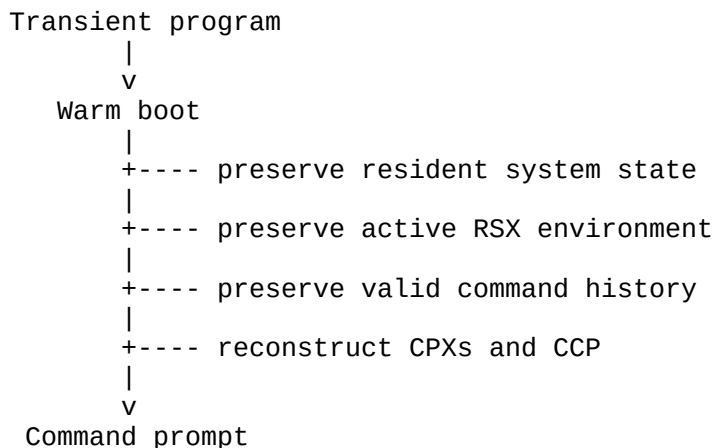
Warm boot returns control to BetterCP/M without creating a new operating-system instance.

The CP/M warm-boot mechanism is used by transient programs to terminate and return to the operating system. BetterCP/M also uses warm-boot processing as a controlled point at which parts of the command environment can be reconstructed.

A warm boot preserves state whose defined lifetime extends across it. This includes valid system state and resident-extension configuration that are not defined to reset on warm boot.

Other state can be reconstructed. In particular, the CCP and configured CPXs need not remain intact while a transient program executes. Warm-boot processing can restore the command environment from its active configuration and return to the prompt.

Conceptually:



Warm boot does not reapply the saved cold-boot configuration merely because control has returned to the operating system. Active session state remains authoritative until an operation explicitly changes it or a cold boot establishes a new instance.

3.7 System Reconstruction

Reconstruction allows BetterCP/M to rebuild components whose current memory image need not remain permanently resident.

The command environment is the ordinary example. The CCP and CPXs are needed while commands are being processed but can occupy reclaimable memory that

becomes available to a transient application. When the application terminates, BetterCP/M reconstructs the command environment.

Extension configuration introduces a more substantial form of reconstruction.

CPXs are maintained in an explicit configured order. Their runtime addresses are not persistent identities. Reconstruction can load and relocate the configured modules into the available command-environment region and then place the CCP beneath them.

RSXs belong to the protected resident system-service environment. Loading or unloading an RSX can change the protected resident layout and the TPA boundary. Because such a change can also alter the addresses available to the command environment, an RSX reconfiguration can require the CPXs and CCP to be reconstructed as well.

A reconstruction is treated as a controlled state transition. BetterCP/M validates the proposed configuration and its memory requirements before publishing it as the live environment.

This leads to an important architectural rule:

Configuration identifies which components should exist and in what order; current runtime addresses are consequences of reconstruction, not persistent configuration.

Code that can be reconstructed must therefore not assume that it will always return to the same address.

3.8 Boot and Recovery

A failed optional configuration must not leave BetterCP/M permanently unable to reach a usable command environment.

Boot and reconstruction therefore include defined failure and recovery behavior.

Prospective configurations are validated before publication wherever possible. An invalid or incompatible extension, damaged module, impossible memory layout, or other detectable failure is rejected rather than knowingly becoming the active system.

Where a failure occurs during a transition that cannot simply return to the former state, BetterCP/M enters a defined recovery path rather than exposing a partially constructed environment.

The system also retains the concept of a recovery configuration in which optional facilities can be suppressed sufficiently to regain control of the machine.

The configured startup command participates in this recovery model. Because an automatically executed command could itself prevent normal startup, BetterCP/M provides a documented means to suppress the startup action during recovery. The startup command is not retried repeatedly after failure.

The general principle is:

Optional configuration can enhance the running system, but it must not turn a recoverable configuration error into an unrecoverable system disk.

Detailed recovery procedures belong in the User's Guide. The architectural requirement is that the boot and configuration mechanisms provide a path by which those procedures can work.

3.9 Boot Interface Boundary

The boot interface separates platform-specific boot mechanics from the operating-system state expected by BetterCP/M.

Below the boundary, a platform can use whatever mechanism is appropriate to locate the system image, access boot media, initialize required hardware, or arrange immutable ROM contents.

Above the boundary, BetterCP/M expects a defined environment and assumes responsibility for establishing the operating-system state.

This separation prevents the boot implementation of one machine from becoming the architecture of BetterCP/M as a whole.

The boot boundary also allows different storage technologies and firmware environments to be supported without requiring the hardware-independent operating system to know how each platform originally obtained its system image.

A platform port must therefore answer two distinct questions:

- 1. How does this machine obtain and enter BetterCP/M?**
- 2. How does BetterCP/M interact with this machine after initialization?**

The first belongs to boot support. The second belongs principally to the hardware-abstraction and BIOS architecture described in Chapter 7.

Keeping those responsibilities distinct allows BetterCP/M to retain a common system architecture even where the machines that boot it differ substantially.

4. The Command Environment

The BetterCP/M Command Environment provides the normal interactive interface between the user and the operating system. It performs the architectural role of the CP/M Console Command Processor, or CCP: presenting the command prompt, accepting command input, executing a small set of core commands, dispatching commands supplied by Command Processor Extensions, and initiating transient programs.

BetterCP/M deliberately keeps the core command processor small. Facilities do not become part of the CCP merely because they can be invoked from a command line. General file, console, storage, and system operations remain services of the appropriate operating-system components.

BetterCP/M extends the traditional CCP model through Command Processor Extensions, or CPXs. A CPX adds commands or other command-processing behavior while remaining part of the reclaimable command environment. CPXs are distinct from Resident System Extensions, which extend the system-service environment and remain protected while transient applications execute.

The resulting command environment preserves conventional CP/M operation while allowing a richer interactive environment without permanently reducing the TPA by the full size of that environment.

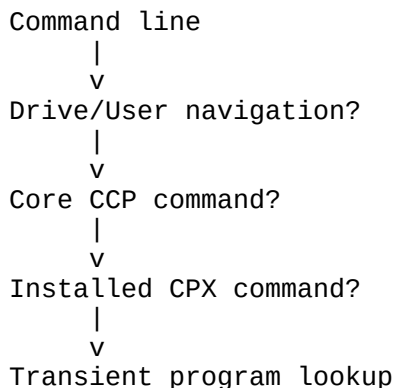
4.1 Command Environment Model

At the command prompt, BetterCP/M waits for a command line. Once the line has been entered, the Command Environment determines how it is to be interpreted.

A command can represent:

- drive/user navigation;
- a core command implemented directly by the CCP;
- a command supplied by an installed CPX; or
- a transient program to be located and executed.

The dispatch order is fixed:



The first component that owns the command handles it. An unrecognized CPX command is passed along the configured CPX chain, and a command declined by every CPX proceeds to transient-program lookup.

This ordering prevents a CPX from silently replacing core compatibility behavior simply because it happens to recognize the same command name.

The Command Environment is not a general shell framework. BetterCP/M 1.0 implements the command-processing facilities required by its CP/M-compatible environment and its defined extensions without introducing a generalized command language merely for future use.

4.2 The Console Command Processor

The CCP is the core of the BetterCP/M Command Environment.

Its responsibilities include:

- presenting and maintaining the command-processing cycle;
- accepting command input;
- recognizing drive/user navigation;
- executing the core resident commands;
- offering eligible commands to the CPX chain;
- initiating transient-program lookup;
- supplying the information needed to establish the transient execution environment; and
- regaining control after transient execution terminates.

The CCP does not remain wholly resident while a transient program executes. Its reclaimable portions can be overwritten by application use of the TPA and reconstructed when BetterCP/M returns to command processing.

The core CCP therefore contains only commands for which direct inclusion has a specific justification. BetterCP/M 1.0 defines the following core commands:

GET JUMP PEEK (P) POKE GO SAVE

GET, PEEK, POKE, GO, JUMP, and SAVE together provide a small resident monitor facility for loading, examining, modifying, executing, and saving memory images.

SAVE particularly illustrates why command placement is architectural rather than merely organizational. A transient **SAVE.COM** would overwrite part of the TPA image that the command is intended to save. SAVE therefore belongs in the core command environment.

Frequently used file-management commands need not be built into the CCP itself. They are supplied by the standard resident command CPX described in Section 4.6.

4.3 Command Input and Interpretation

BetterCP/M accepts command lines according to the compatibility-visible conventions of CP/M while extending command input where doing so does not disturb that environment.

The command processor normalizes and interprets the command line sufficiently to determine its destination. Internal parsing need not reproduce the implementation of the Digital Research CCP.

Command interpretation remains separate from the services requested by the resulting command. For example, recognizing **ERA** as a command is a command-

environment operation; actually erasing a file uses the normal BetterCP/M file services.

Similarly, recognizing the name of a transient program does not make the CCP responsible for implementing the complete program loader or file system. The Command Environment coordinates the operation through the appropriate system facilities.

CPXs receive commands through a defined dispatch interface. In BetterCP/M 1.0 the command offered to a CPX is normalized to uppercase and accompanied by its length. A CPX either handles the command or declines it so that dispatch can continue.

This creates a deliberately narrow boundary. A CPX is not automatically an observer of every command and does not receive first refusal on drive/user navigation or core CCP commands.

4.4 Drive/User Navigation

BetterCP/M extends the conventional CP/M drive and user-area model with direct **DU** navigation.

The command processor recognizes navigation before ordinary command dispatch. Forms such as:

B:
5:
C3:

select a drive, a user area, or both.

This makes the current drive/user combination a direct property of command navigation rather than requiring every selection to be expressed through a transient or CPX command.

The traditional **USER** command remains available for compatibility and convenience, but it is not required in order to select a user area.

Drive/user navigation is resolved before core commands, CPXs, and transient lookup. It therefore remains available even in a minimal command environment.

A qualified operation or directory request does not silently change the caller's current drive/user merely because it refers to another location. Selection of the current DU and qualification of an individual operation remain distinct.

Named directories, NDR management, and PATH semantics are not part of BetterCP/M 1.0. The canonical 1.0 navigation model is the explicit drive/user combination.

4.5 Core Resident Commands

The core CCP contains only the commands that BetterCP/M requires to be available independently of CPXs and transient utilities:

GET JUMP PEEK P POKE GO SAVE

The memory-oriented commands share common address conventions. Addresses are hexadecimal, leading zeroes are optional, and the valid address range is **0000H** through **FFFFH**.

The commands are designed to fail before partially performing an operation when their required operands are malformed.

GET loads a file into memory without executing it. With no explicit address, loading begins at **0100H**; an address operand permits loading elsewhere where the requested range is safe.

PEEK, abbreviated **P**, displays memory. It supports individual starting addresses, inclusive ranges, and forward or backward paging.

POKE modifies memory using hexadecimal byte values, literal text, or a combination of the two.

GO establishes the normal transient execution environment and transfers control to **0100H**.

JUMP performs the corresponding operation at an explicitly specified address.

SAVE writes a specified number of 256-byte pages beginning at **0100H** to a file.

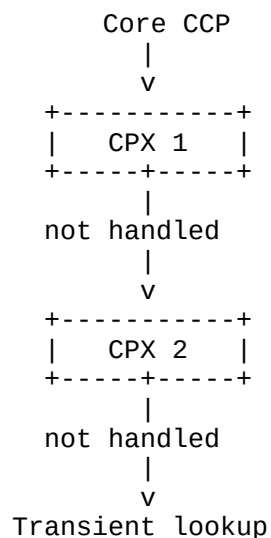
These commands provide useful low-level capabilities while remaining small enough to justify their position in the CCP. Their user-visible syntax is documented in the User's Guide.

4.6 Command Processor Extensions

A Command Processor Extension, or CPX, extends command processing without becoming part of the protected resident system-service environment.

CPXs form an ordered command-dispatch chain. Each installed CPX is offered a command only after drive/user navigation and core CCP commands have declined ownership.

Conceptually:



A CPX that handles the command terminates dispatch. A CPX that does not recognize the command passes it onward.

The default BetterCP/M resident command package is **RCP.CPX**. Its command inventory is:

DIR	ERA	TYPE	REN	USER
CLS	VER	COPY	MOVE	

These commands appear resident to the user but do not need to remain protected while a transient application executes. The entire CPX/CCP command environment can be reclaimed and reconstructed.

This distinction is important to BetterCP/M's memory architecture. Adding a CPX consumes command-environment space while the prompt is active but does not by itself lower the TPA available to a transient application.

CPXs can be loaded and unloaded dynamically. Their configured order is explicit and is preserved by reconstruction.

The CPX architecture is described in detail in Chapter 13.

4.7 Transient Commands

If a command is not drive/user navigation, is not a core command, and is declined by every installed CPX, BetterCP/M attempts to execute it as a transient program.

Transient commands normally correspond to **.COM** files stored on disk.

The Command Environment determines that transient lookup is required, but the resulting operation crosses into the Program Execution Environment and System Services. BetterCP/M must locate the program, establish its compatibility-visible initial state, load it into the TPA, and transfer control to it.

The conventional CP/M invocation environment is preserved. This includes the command tail, default FCBs, DMA state, page-zero vectors, and other program-visible conditions required by compatible software.

The use of CPXs does not remove the transient command model. A command can be supplied residently for convenience while a transient counterpart provides additional functions or serves as a fallback when the corresponding CPX is absent.

Conversely, not every resident command requires a transient equivalent. Command placement is determined by function and system requirements rather than by a requirement for symmetrical resident and transient implementations.

4.8 Command Dispatch Order

BetterCP/M 1.0 defines command dispatch in the following order:

- 1.drive/user navigation;
- 2.core CCP commands;
- 3.installed CPXs in configured order;
- 4.transient-program lookup.

This ordering is part of the command-environment architecture.

It ensures that fundamental navigation remains available independently of extensions. It protects core command behavior from accidental CPX interception. It allows multiple CPXs to coexist with deterministic precedence. Finally, it

preserves ordinary CP/M transient execution as the fallback when no resident facility claims a command.

The CPX order is explicit rather than derived from a dependency graph. BetterCP/M 1.0 does not define general module-carried **BEFORE**, **AFTER**, **REQUIRES**, **CONFLICTS**, or similar relationships for CPXs.

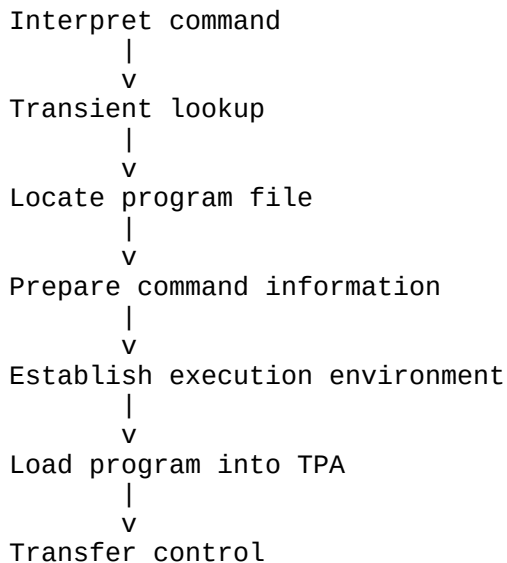
Where two CPXs could recognize the same command, configured chain order determines which receives the opportunity first.

This behavior makes extension order observable and therefore something configuration tools must preserve rather than silently rearrange.

4.9 Program Invocation

Once transient lookup identifies an executable program, BetterCP/M establishes the environment in which the program will run.

Conceptually, invocation proceeds as follows:



The Command Environment coordinates the transition but does not privately duplicate the file system or program-execution machinery.

The application receives the CP/M-compatible environment rather than an interface exposing BetterCP/M's private command-processing structures.

When the application terminates, control returns through the operating-system termination path. BetterCP/M can then reconstruct the command environment and resume the command-processing cycle.

The Program Execution Environment is described in Chapter 5.

4.10 Command-Line Editing and History

BetterCP/M extends the basic CP/M command-input experience with line editing and bounded command history.

Command editing operates while a command is being constructed. BetterCP/M supports cursor movement and editing conventions appropriate to its command environment, including the retained WordStar-style editing behavior.

Command history permits previously entered commands to be recalled, edited, and executed again.

The history facility is persistent across ordinary transient execution and warm boot. It is not stored in reclaimable CPX or CCP memory, because that memory can be destroyed while an application executes.

Instead, BetterCP/M 1.0 assigns command history the fixed 192-byte Persistent Data Segment. Command history is the sole owner of the 1.0 PDS.

A true cold boot initializes a valid empty history. Warm boot and command-environment reconstruction preserve valid history. Invalid history state is discarded safely rather than being interpreted as commands.

The fixed history facility is deliberately bounded. BetterCP/M 1.0 does not implement a general persistent-data allocator merely to support possible future command-environment features.

4.11 SUBMIT and XSUB

BetterCP/M retains the CP/M batch-command environment provided by **SUBMIT** and **XSUB**.

SUBMIT permits a sequence of commands to be supplied from a command file rather than entered interactively one at a time. **XSUB** extends batch operation to programs that obtain input through the conventional console-input path.

These facilities are retained because they form part of established CP/M operating practice and because existing command files can depend upon their behavior.

Batch processing remains integrated with the ordinary command-dispatch environment. Commands encountered during **SUBMIT** processing are resolved according to the same drive/user, core-command, CPX, and transient lookup rules used for interactive commands.

The command environment must also recover cleanly when a batch operation terminates, fails, or is abandoned. A warm boot caused during batch execution is not merely equivalent to returning normally from one command; termination behavior must preserve the established CP/M semantics of the command sequence.

The syntax and use of **SUBMIT** and **XSUB** are described in the User's Guide.

4.12 Reentry and Restoration

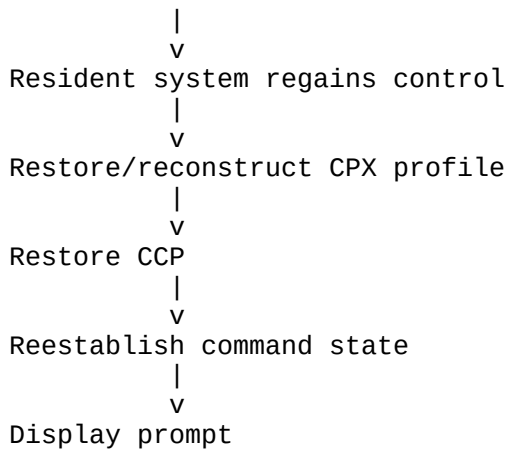
Transient execution can overwrite the reclaimable command environment. BetterCP/M must therefore be able to restore a usable command processor whenever control returns from an application.

The CCP and configured CPXs are reconstructed together from the active command configuration. Their current runtime addresses are not persistent configuration.

RSXs are different. They belong to the protected system-service environment and remain installed independently of whether a particular CCP or CPX image is currently present.

Ordinary reentry therefore has the conceptual form:

Transient program terminates



State whose lifetime extends across transient execution is preserved by its proper owner rather than by assuming the old command-environment memory remains intact.

This applies particularly to command history, current system state, and the active extension configuration.

The reconstruction mechanism allows BetterCP/M to reconcile two competing goals: a richer command environment at the prompt and a large TPA while applications execute. The system need not choose between keeping the entire command processor permanently resident and abandoning resident command extensions altogether.

5. Program Execution

BetterCP/M preserves the CP/M 2.2 transient-program execution model. An ordinary CP/M program is loaded into the Transient Program Area beginning at 0100H, receives the conventional page-zero environment and command information, accesses operating-system services through the standard CP/M interface, and returns control to the operating system when execution terminates.

The application does not need to know how BetterCP/M internally implements the command environment, system services, resident extensions, or hardware abstraction. These mechanisms exist behind the compatibility boundary.

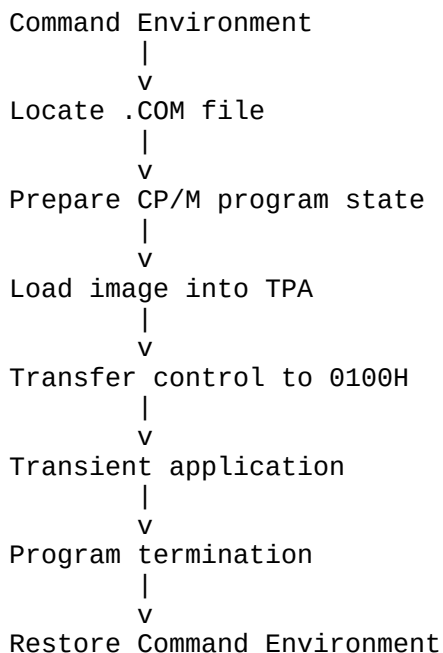
BetterCP/M also takes advantage of the transient nature of the command environment. The reclaimable CCP and CPXs need not remain intact while an application executes. Their memory can therefore be made available to the program and reconstructed when control returns to the operating system.

5.1 The CP/M Execution Model

BetterCP/M 1.0 is a single-tasking system. At most one transient application executes at a time.

A program normally begins as a .COM file containing a directly loadable memory image. BetterCP/M establishes the CP/M-compatible execution environment, loads the program, and transfers control to its entry point.

Conceptually:



Once control has been transferred, the program owns the application portion of the TPA until it terminates.

BetterCP/M does not impose a new executable format upon ordinary CP/M software. Existing **.COM** programs continue to use the conventional CP/M loading and execution model.

5.2 Transient Program Loading

A conventional transient program is loaded beginning at address **0100H**.

The contents of the **.COM** file correspond directly to the bytes placed in memory. No executable header, relocation table, or BetterCP/M-specific loader information is required for an ordinary CP/M application.

The program image must fit within the TPA available under the current resident configuration. BetterCP/M verifies the load against the actual application-memory boundary rather than permitting a large image to overwrite protected system memory.

The default BetterCP/M 1.0 configuration supports a complete-record **.COM** ceiling of 53 KiB. Optional resident facilities that consume protected memory can lower the available TPA and therefore reduce the largest transient image that can be loaded.

Program loading uses the normal storage and file-system services. The Command Environment initiates the operation, but it does not contain a private file loader independent of the rest of the operating system.

The same underlying execution machinery can also be used when control is transferred to an image already present in memory, as with the resident **GO** and **JUMP** commands.

5.3 Initial Program State

Before transferring control to a transient application, BetterCP/M establishes the processor and memory state required by the CP/M execution environment.

This includes:

- the page-zero compatibility environment;
- the BDOS entry mechanism;
- the current drive;
- the default DMA address;
- the command tail;
- the default File Control Blocks;
- the transient execution stack; and
- other state required by the adopted CP/M compatibility contract.

The program begins execution at **0100H** for a normally loaded **.COM** file.

BetterCP/M does not allow stale state from a previously executed program to become the initial state of the next program merely because the corresponding bytes remain in memory.

For example, a command invoked with no command tail receives a valid empty command tail rather than whatever text happened to be supplied to an earlier application.

Initial execution state is therefore deliberately established as part of invocation.

5.4 Page Zero

Addresses 0000H through 00FFH form the CP/M-compatible page-zero area.

Page zero contains the vectors and program information through which transient applications interact with the operating environment. BetterCP/M preserves the locations and semantics required by compatible CP/M software.

Among the most important are the warm-boot vector at the bottom of memory and the system-service entry mechanism associated with address 0005H.

Page zero also contains application-visible command information, including the default FCBs and command tail.

Because page zero is part of the compatibility interface, BetterCP/M cannot treat it as ordinary application scratch storage or reorganize it solely for internal convenience.

At the same time, the contents established for one transient invocation need not remain meaningful after that invocation ends. BetterCP/M recreates the required page-zero state when preparing another execution environment.

5.5 Command Tail and Default FCBs

The command text following the transient program name is supplied using the conventional CP/M command-tail representation.

This allows a command such as:

```
A>PROGRAM INPUT.DAT
```

to provide INPUT.DAT to PROGRAM.COM through the interface expected by CP/M software.

BetterCP/M also constructs the conventional default File Control Blocks from eligible command operands. Programs written to use the traditional CP/M default-FCB convention therefore continue to receive the expected initial file specifications.

The command tail and default FCBs are compatibility interfaces, not BetterCP/M's preferred general argument-passing mechanism. Their representation is preserved because existing software depends upon it.

BetterCP/M extensions can define additional interfaces where needed, but ordinary CP/M applications are not required to use them.

When no command tail is supplied, BetterCP/M creates a valid empty tail. A previous program's command information is never implicitly reused.

5.6 Program Memory and the TPA

During transient execution, the application can use the TPA from 0100H up to the boundary published by BetterCP/M.

The resident operating system remains protected above that boundary.

The TPA can contain:

- the loaded program image;
- program data;
- application stacks;

- dynamically managed application workspace; and
- other memory used by the transient program.

BetterCP/M does not attempt to preserve the reclaimable command environment within the TPA merely because that environment existed before the application was loaded. CCP and CPX memory that can safely be reconstructed is available for reuse as part of the execution model.

Protected resident facilities are different. Core system services, required hardware-dependent code and state, active RSXs, and other facilities needed during application execution remain outside the TPA.

Consequently, installing an RSX can reduce the TPA, while installing a CPX does not impose the same permanent application-memory cost.

The current TPA boundary, rather than a fixed historical assumption about system size, defines the memory available to the program.

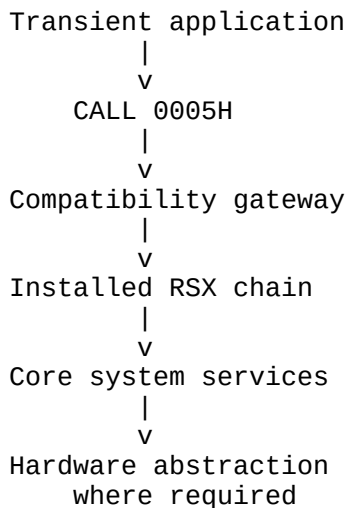
5.7 System-Service Access

Transient applications request operating-system functions through the CP/M-compatible system-service interface.

For ordinary CP/M software, the principal interface is the conventional call through address **0005H**. BetterCP/M preserves the calling conventions, function semantics, and result behavior required by its CP/M compatibility contract.

Internally, a request can pass through BetterCP/M's resident-extension and system-service architecture before reaching the core implementation.

Conceptually:



An ordinary CP/M application need not know whether an RSX is installed or how BetterCP/M organizes the core service implementation.

BetterCP/M-specific services can extend the system-service environment without changing the meaning of existing CP/M functions. The extended service architecture is described in Chapter 6.

5.8 Program Termination

A transient application can terminate through the conventional CP/M mechanisms.

Termination transfers control back to the operating system rather than directly to a private copy of the command processor. BetterCP/M then performs whatever restoration is required before interactive command processing resumes.

Termination can occur normally or as the result of an abort or warm-boot path. The exact state preserved across termination depends upon the ownership and lifetime rules for that state.

Application-owned state in the TPA ceases to be valid when the program terminates.

Resident operating-system state remains under the ownership of the appropriate subsystem. Valid command history, active resident-extension configuration, current system state, and other information defined to survive the transition are not discarded merely because a transient application has ended.

The operating system must not depend upon application memory continuing to contain useful system information after termination.

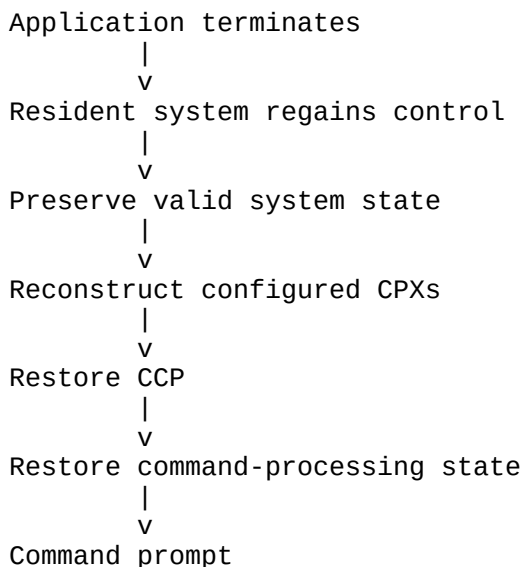
5.9 Return to the Command Environment

When transient execution ends, BetterCP/M reestablishes a usable Command Environment.

Because the CCP and CPXs can be overwritten during application execution, returning to the prompt can require reconstruction rather than simply branching back into an untouched command processor.

The active CPX reconstruction profile identifies the command extensions that must be restored. BetterCP/M reconstructs the configured CPX chain and reloads or restores the CCP beneath it.

Conceptually:



The protected RSX environment remains independent of this process. An RSX does not depend upon a particular CCP or CPX image continuing to exist.

This arrangement allows BetterCP/M to make command-environment memory available to applications without giving up dynamically configurable resident commands.

5.10 Execution Compatibility

BetterCP/M's program-execution architecture is designed so that an ordinary CP/M 2.2 application does not need to know that it is running under BetterCP/M.

Compatibility includes the required relationships among:

- program load address;
- page-zero contents;
- system-service entry;
- command tail;
- default FCBs;
- DMA state;
- current drive and user area;
- program termination;
- TPA boundary; and
- other adopted execution conventions.

BetterCP/M is free to reconstruct the command environment, reorganize private resident memory, implement the BDOS differently, or use a different hardware-abstraction structure because those mechanisms are behind the application compatibility boundary.

Software that bypasses the CP/M interfaces and depends directly upon machine-specific hardware or implementation-private operating-system structures presents a different compatibility problem. Such software can require a particular hardware or historical implementation environment in addition to CP/M compatibility.

BetterCP/M does not silently convert those machine-specific assumptions into requirements for the operating system as a whole.

The objective is therefore not binary identity with Digital Research CP/M. It is preservation of the execution environment on which compatible CP/M software depends, while allowing BetterCP/M's internal architecture to evolve independently.

6. BDOS and System Services

System Services provide the general operating-system functions used by transient applications, the Command Environment, and other BetterCP/M components. For CP/M-compatible applications, they present the service environment historically provided by the Basic Disk Operating System, or BDOS.

BetterCP/M preserves the application-visible CP/M 2.2 BDOS interface without requiring its internal service implementation to duplicate Digital Research's BDOS. Applications continue to use the established entry mechanism, function numbers, calling conventions, data structures, and return behavior. Behind that interface, BetterCP/M can organize services according to its own architectural boundaries.

BetterCP/M also provides services beyond those defined by CP/M 2.2. These extensions use explicitly allocated interfaces rather than changing the meaning of existing CP/M functions. Resident System Extensions can intercept or extend the system-service path, while a separate resident-service facility allows applications and system components to discover callable services supplied by resident providers.

The result is a service architecture with a stable CP/M compatibility boundary and a defined path for BetterCP/M-specific facilities.

6.1 System-Service Model

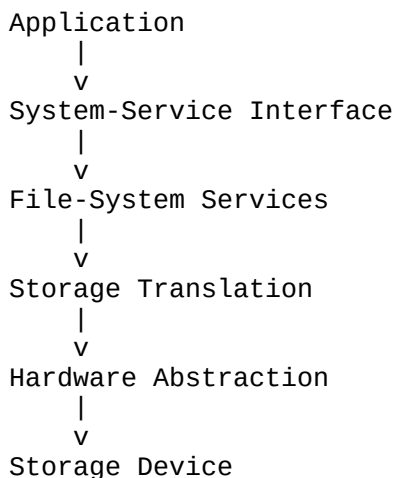
A system service represents an operating-system operation rather than the hardware mechanism by which that operation is performed.

Examples include:

- reading or writing console characters;
- opening, closing, reading, or writing files;
- selecting drives and user areas;
- setting the DMA address;
- obtaining operating-system information; and
- terminating a transient program.

The implementation of one of these operations can cross several architectural components.

For example:



The application requests a logical file operation. It does not need to know which disk controller, emulator interface, physical sector layout, or other hardware mechanism ultimately performs the transfer.

Not every system service reaches hardware. Operations involving only operating-system state can be completed entirely within the resident system.

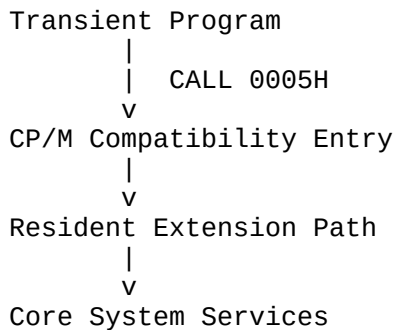
The system-service boundary therefore separates **what operation is requested** from **how the underlying machine performs it**.

6.2 The BDOS Interface

CP/M-compatible applications request system services through the conventional BDOS entry mechanism.

The application places the function number and required parameters in the locations defined by the CP/M calling convention and transfers control through the standard entry at address **0005H**.

Conceptually:



For each CP/M-compatible service, the application-visible interface defines:

- the function identifier;
- input registers or memory structures;
- returned register values;
- memory structures modified by the operation;
- effects upon operating-system state;
- error behavior; and
- conditions that hold when control returns to the caller.

BetterCP/M can use different internal calling conventions between its own components. Internal services do not need to imitate the CP/M application ABI merely because the corresponding external operation is presented through BDOS.

The application-visible BDOS interface and BetterCP/M's internal implementation are therefore deliberately distinct.

6.3 Console and Character Services

System Services provide the console and character-oriented operations required by the CP/M 2.2 environment.

These include the conventional operations for:

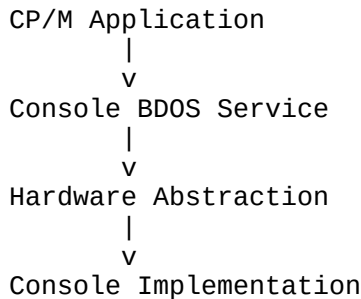
- console input;
- console output;
- direct console interaction;

- console status;
- string output;
- buffered console input; and
- other character-oriented functions defined by the CP/M interface.

The service layer owns the CP/M-visible semantics of these operations.

The physical console itself belongs below the hardware-abstraction boundary. A BetterCP/M system can therefore use a serial terminal, integrated display and keyboard, emulator console, or another suitable device without requiring ordinary applications to distinguish among them.

Conceptually:



Terminal-specific or platform-specific capabilities that go beyond ordinary CP/M console behavior are exposed through separate BetterCP/M facilities rather than silently changing the semantics of existing console functions.

6.4 File and Record Services

BetterCP/M implements the CP/M file and record operations required by compatible applications.

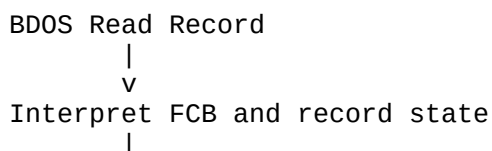
These include operations for:

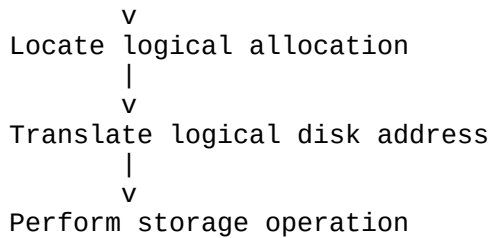
- locating files;
- creating files;
- opening and closing files;
- sequential and random record access;
- reading and writing records;
- renaming files;
- deleting files;
- examining directory information; and
- other operations defined by the CP/M file interface.

The application-visible model remains based upon CP/M File Control Blocks, records, extents, directory entries, and logical drives.

A program using these services does not need to know how the underlying medium is physically organized. The system-service layer translates the CP/M operation into the storage and file-system operations required by the currently selected disk format and device.

For example:





The detailed storage architecture is described in Chapters 8 and 9.

6.5 Drive and User Services

The BDOS environment maintains the logical drive and user-area state expected by CP/M applications.

System Services provide operations for selecting and examining the current logical storage environment and for manipulating drive-related state exposed through the CP/M interface.

Logical drives are operating-system abstractions. BetterCP/M does not require a one-to-one relationship between a CP/M drive letter and a physical storage device.

A logical drive can represent storage supplied by a floppy drive, emulator image, partition or region of another medium, or another backend capable of providing the required logical disk behavior.

Similarly, user areas remain part of the CP/M-visible filesystem environment rather than properties of the physical storage hardware.

The Command Environment can provide more convenient drive/user navigation, but the underlying system state remains authoritative and visible through the normal service interfaces. The CCP does not maintain an independent private notion of the current drive or user area.

6.6 Program-Support Services

Some BDOS operations support the execution environment rather than file or character I/O directly.

These include services concerned with such things as:

- program termination;
- warm-start behavior;
- DMA address management;
- examination or modification of program-related system state; and
- other execution-support functions defined by CP/M.

Responsibility is divided between System Services and the Program Execution Environment.

The Program Execution Environment establishes the conditions under which a transient program begins and runs. System Services provide callable operations that inspect or alter the relevant operating-system state.

For example, the current DMA address is operating-system state manipulated through the BDOS interface, while the initial DMA state supplied to a newly invoked program is part of establishing that program's execution environment.

Where the two areas interact, BetterCP/M defines a single authoritative state rather than maintaining separate execution and BDOS copies that can diverge.

6.7 System State and Service State

System Services necessarily interact with operating-system state.

Examples include:

- current logical drive;
- current user area;
- current DMA address;
- file-system working state;
- storage-selection state; and
- state associated with particular resident services.

Each item of persistent state has a defined owner.

A system service can provide access to state without becoming the owner of that state merely because the state is exposed through a BDOS call. Similarly, multiple services can operate upon the same state without maintaining independent authoritative copies.

Temporary information needed only while a service is executing remains private to that service or subsystem.

The general rule is:

Shared state has one authoritative owner; interfaces provide controlled access to it.

This becomes especially important when the command environment, resident extensions, storage configuration, and applications all interact with the same operating-system state.

System-state ownership is described in detail in Chapter 10.

6.8 Error and Return Behavior

System-service results are part of the compatibility interface.

CP/M does not define one universal error-return mechanism. Depending upon the operation, a result can be represented through:

- register values;
- modified memory structures;
- updated operating-system state;
- console messages;
- program termination;
- warm boot; or
- another service-specific effect.

BetterCP/M preserves the required externally visible behavior rather than forcing every CP/M operation through a newly invented generalized error system.

Internally, a subsystem can use clearer or more systematic status representations where useful. Those internal results are translated into the required application-visible behavior at the compatibility boundary.

This distinction allows BetterCP/M to improve its internal error handling without requiring existing CP/M software to understand a new error model.

BetterCP/M-specific interfaces can define explicit status conventions appropriate to their own ABIs.

6.9 BetterCP/M Extensions

BetterCP/M extends the system-service environment without redefining existing CP/M BDOS functions.

Private BetterCP/M BDOS services occupy a deliberately allocated namespace. BetterCP/M 1.0 reserves functions **176 through 199** for this purpose.

The range is divided as follows:

176-196	Production BetterCP/M services
197	Reserved future escape selector
198-199	Non-public test/conformance functions

BetterCP/M does not interpret every high-numbered BDOS function as its own. Other CP/M-family systems and extensions have historically used portions of the high BDOS namespace, so only explicitly assigned BetterCP/M functions are intercepted as BetterCP/M services.

The 1.0 production assignments are:

176	CPX profile control
177	RSX profile control and reconstruction
178	Console/video cursor support
179	Command-environment printer-echo control
180	Protected subsystem version descriptor
181	Disk configuration and utility I/O
182	Callable resident-service registry
183	FDF service
184-196	Unassigned

Functions **198** and **199** are reserved for internal proofs, diagnostics, and conformance testing and do not form part of the public application ABI.

Function **197** has no callable BetterCP/M 1.0 contract. Its reservation leaves room for a future namespace requirement without prematurely defining what that requirement might be.

BetterCP/M normally assigns one top-level function to a coherent service rather than consuming a separate BDOS number for every operation within that service. Individual operations can then be selected through a versioned request block or service-specific subfunction.

This conserves namespace space and gives each extended service a boundary within which its own ABI can evolve.

6.10 Extended BDOS Namespace

The BetterCP/M private namespace was chosen to avoid known historical allocations where practical.

Functions **176** through **199** were selected as the best-supported available range found during the compatibility investigation. The allocation is nevertheless deliberately narrow: BetterCP/M claims only the functions it has actually assigned.

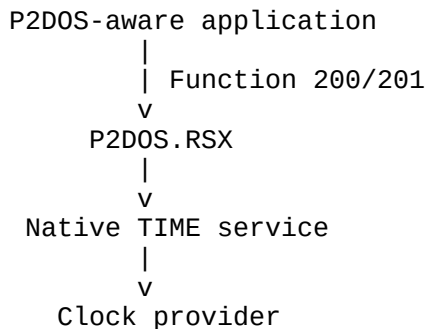
Two important compatibility functions lie immediately above this range:

200 P2DOS-compatible get clock
201 P2DOS-compatible set clock

These are not BetterCP/M-private function numbers. Their meanings are inherited from the P2DOS family.

BetterCP/M implements them through the optional **P2DOS.RSX** compatibility frontend. That RSX translates the historical calls into BetterCP/M's native TIME service.

Thus:



The hardware clock provider itself does not implement P2DOS functions. Historical compatibility is separated from the native BetterCP/M service boundary.

This pattern permits BetterCP/M to support established third-party interfaces without making those interfaces the architecture of the underlying facility.

6.11 Resident Services

Not every BetterCP/M service is best represented as another BDOS function.

RSXs and fixed resident components can publish **callable resident services**. A client discovers such a service through the resident-service registry and then invokes the service through its returned callable entry point.

A service is identified by a four-character identifier independent of the filename or implementation version of its provider.

For example:

TIME
HIST

A provider advertises the service identity together with its ABI version, callable entry offset, and capability information.

The resident-service facility separates **service identity** from **provider identity**. A client asks for a service rather than needing to know which particular RSX filename currently supplies it.

An RSX can advertise more than one service.

BetterCP/M 1.0 permits one active provider for a given service identifier. A proposed resident configuration containing duplicate providers is rejected before publication.

The registry is a logical view of validated advertisements in the active resident environment rather than a manually maintained table of permanent absolute addresses.

6.12 Resident-Service Discovery

BDOS Function 182 provides the BetterCP/M 1.0 resident-service gateway.

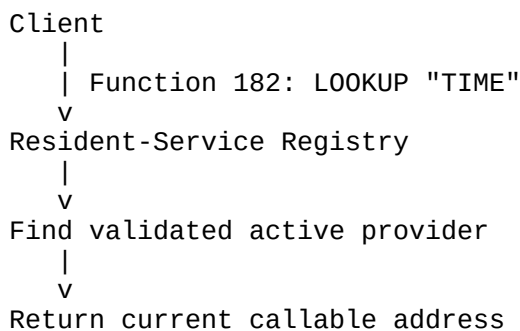
The gateway supports three operations:

- 0 LOOKUP
- 1 ENUMERATE
- 2 DESCRIBE_REGISTRY

LOOKUP requests a service by identifier and required ABI characteristics.

A successful lookup returns the current callable address of a provider satisfying the requested service ID, ABI version, and required capabilities.

Conceptually:



ENUMERATE permits the currently available services to be examined.

DESCRIBE_REGISTRY reports information about the active registry and its implementation capacity.

The registry is derived from validated provider metadata. Ordinary applications and RSXs do not directly insert or remove registry entries. Publication and removal occur as part of the controlled resident-extension reconstruction process.

This prevents a service from becoming visible before its provider is valid and active.

6.13 Service Address Lifetime

A callable resident-service address is not a permanent system identifier.

RSXs can move when the resident layout is reconstructed. If the provider moves, its callable entry point moves with it.

A service address returned by lookup therefore remains valid only while the current resident-layout generation remains unchanged.

Transient applications can safely retain a discovered address during their own execution because BetterCP/M does not reconstruct the resident environment underneath arbitrary transient code.

Longer-lived components must follow the rebinding rules appropriate to their lifetime. In particular, reconstructed CCPs and CPXs rediscover resident services rather than assuming that an address obtained before reconstruction is still valid.

The relationship is therefore:

Service identity	persistent logical identity
Provider filename	implementation identity
Runtime address	current-layout property

Clients should depend upon the first, not the third.

This distinction is essential to BetterCP/M's dynamically reconstructible resident architecture.

6.14 Compatibility and Service Extension

BetterCP/M's service architecture preserves two related but distinct goals.

First, ordinary CP/M software continues to see the service environment it expects. Existing BDOS functions retain their established meanings and calling conventions.

Second, BetterCP/M can provide new facilities without consuming or redefining those interfaces arbitrarily.

New services can be introduced through:

- explicitly allocated BetterCP/M BDOS functions;
- callable resident services;
- RSX interception where a service must extend the BDOS path; or
- compatibility frontends such as **P2DOS.RSX**.

The appropriate mechanism depends upon the nature of the service.

BetterCP/M does not allocate a new BDOS number simply because one appears unused, nor does it introduce a generalized extension mechanism before a concrete requirement justifies it.

Once a BetterCP/M interface is published as stable, however, that interface becomes part of the compatibility contract applicable to software using it.

This permits the system-service architecture to evolve while preserving a clear distinction between the CP/M compatibility environment, BetterCP/M-native facilities, and optional compatibility interfaces inherited from other CP/M-family systems.

7. Hardware Abstraction and the BIOS

BetterCP/M separates hardware-independent operating-system behavior from operations that depend upon the characteristics of a particular machine. This separation performs the general architectural role historically associated with the CP/M BIOS while allowing BetterCP/M to define its internal hardware boundary according to the requirements of the system rather than merely reproducing the implementation of Digital Research CP/M.

Hardware-independent components operate in terms of logical services: console input and output, storage operations, device capabilities, and other functions required by the operating system. Platform-specific code supplies the mechanisms needed to perform those operations on the actual hardware or emulator.

The hardware-abstraction boundary is deliberately modest. BetterCP/M does not attempt to impose a generalized modern device-driver framework upon a small Z80 operating system. A direct call, vector, table, or CP/M-style interface is sufficient where it provides the required separation clearly and economically.

The distinction between the BetterCP/M hardware-abstraction boundary and the compatibility-visible CP/M BIOS is also important. Where the two interfaces have compatible requirements they can share implementation. BetterCP/M does not maintain two complete hardware interfaces merely for architectural purity.

7.1 Hardware-Abstraction Model

A function belongs on the hardware-dependent side of BetterCP/M when its implementation requires knowledge of a particular machine or device.

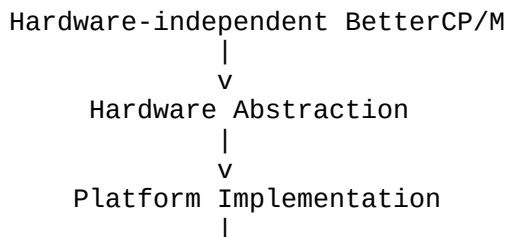
Such knowledge can include:

- I/O port assignments;
- memory-mapped device registers;
- controller protocols;
- hardware status representation;
- physical storage characteristics;
- console implementation;
- machine-specific initialization; and
- other properties that vary among platforms.

Hardware-independent code should not need to know these details in order to perform an ordinary operating-system operation.

For example, the system-service layer can request a console character without knowing whether the character is ultimately displayed by a serial terminal, memory-mapped video display, emulator console, or another device.

Conceptually:



v Hardware

The abstraction ends where machine knowledge becomes necessary. It does not absorb operating-system policy merely because that policy eventually causes hardware to be accessed.

7.2 Hardware-Independent and Hardware-Dependent Boundaries

The hardware-abstraction boundary separates **policy and logical operation** from **machine-specific mechanism**.

For example, deciding which CP/M logical drive is currently selected is operating-system state. Determining how a sector is physically transferred by a particular controller is hardware-dependent mechanism.

Similarly, the CP/M semantics of console input belong to System Services. Reading a keyboard matrix or serial UART belongs to the platform implementation.

The boundary should therefore be placed where it creates a useful separation without adding unnecessary runtime layers.

BetterCP/M does not require that every interaction with hardware pass through a generalized dispatcher. On a Z80 system, a compact assembly-language call or vector can be a perfectly adequate architectural interface.

The important requirement is that hardware-independent components do not unnecessarily depend upon details that should belong to the platform.

This permits a platform implementation to change without forcing corresponding changes throughout the operating system.

7.3 Console and Character Devices

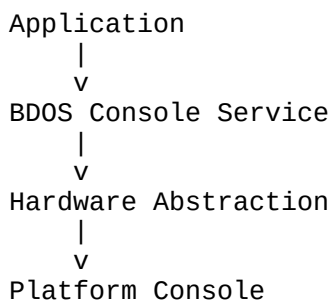
Hardware Abstraction provides the machine-dependent operations needed by BetterCP/M's console and character services.

These operations can include:

- console input;
- console output;
- console-input status;
- logical character-device access; and
- device initialization or control where required.

System Services define what a CP/M console operation means. Hardware Abstraction determines how that operation is carried out on the selected platform.

Thus:



The platform console can be implemented by very different mechanisms without changing the application-visible CP/M service.

BetterCP/M does not assume that every machine has a CRT, serial terminal, memory-mapped display, or particular keyboard arrangement.

The same principle applies to the traditional CP/M logical character devices. **CON:**, **RDR:**, **PUN:**, and **LST:** describe logical destinations or sources rather than permanently defining particular physical hardware.

A platform can expose appropriate physical or virtual providers for these devices.

7.4 Storage Devices

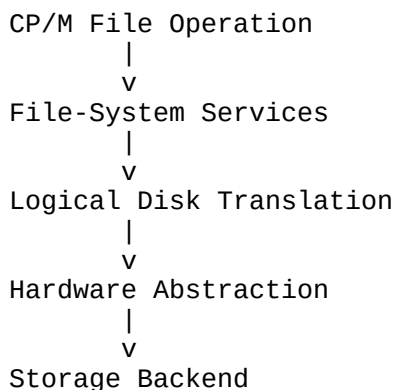
Hardware Abstraction supplies the physical storage operations required by the storage subsystem.

Machine-dependent storage responsibilities can include:

- device initialization;
- device selection;
- physical or backend-level reads;
- physical or backend-level writes;
- device-status handling;
- controller-specific translation; and
- other operations required to access the medium.

The filesystem does not directly program a disk controller merely because it needs a record.

Instead, the operation crosses the storage boundary:



This prevents controller-specific assumptions from spreading into the CP/M filesystem implementation.

The storage backend need not be a traditional floppy disk. An emulator image or a later block-storage implementation can satisfy the same logical requirements through a different physical mechanism.

7.5 Logical and Physical Storage

BetterCP/M makes a deliberate distinction between **logical CP/M storage** and **physical storage**.

A logical CP/M drive does not have to correspond one-to-one with a physical device.

Likewise, a logical CP/M track or sector need not represent an actual physical track or sector on the underlying medium.

The Storage and File-System subsystem owns the interpretation of:

- logical drives;
- CP/M disk formats;
- directories;
- allocation blocks;
- logical tracks and sectors; and
- record organization.

Hardware Abstraction owns the mechanism by which the required storage is physically accessed.

This distinction allows a CP/M disk format to remain meaningful even when the underlying storage device bears little resemblance to the floppy disk for which the format was originally designed.

It also allows several logical drives to be mapped to one physical device where a platform supports such an arrangement.

The logical-to-physical storage architecture is described further in Chapters 8 and 9.

7.6 Hardware Initialization

Each platform defines the initialization required to make its hardware usable by BetterCP/M.

Some initialization occurs during boot before the hardware-independent system can operate. Other initialization can occur when BetterCP/M applies the active system configuration.

Hardware initialization can include such operations as:

- establishing console hardware;
- initializing storage controllers;
- configuring serial devices;
- selecting device operating modes;
- establishing platform-specific vectors or state; and
- preparing other required peripherals.

The platform implementation owns the hardware-specific mechanism.

Where configuration values determine hardware behavior, the hardware implementation should consume the authoritative configuration rather than maintaining an unrelated private copy.

If changing a configuration value is sufficient to alter subsequent behavior, no additional mechanism is necessary.

If a hardware device must be reprogrammed when its configuration changes, the platform implementation can provide a compact operation that applies the current configuration to the device.

Where practical, this operation should reuse the same machine-specific initialization logic needed during startup rather than duplicating it.

7.7 Platform Capabilities and Configuration

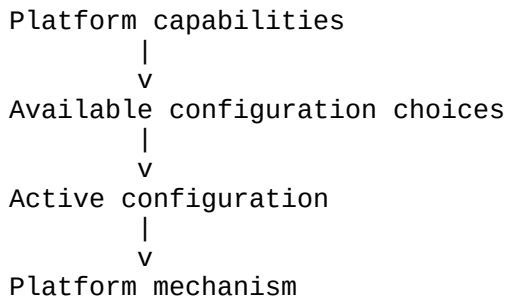
BetterCP/M distinguishes between **what a platform can do** and **what the user has configured it to do**.

The platform implementation publishes the capabilities it actually provides. Configuration selects among or parameterizes those capabilities.

For example, one platform might provide a programmable UART with selectable baud rate, parity, word length, and flow control. An emulator might instead expose a character stream whose physical framing is controlled by the host. Another platform might have no serial device at all.

BetterCP/M should not pretend that these systems have identical configurable hardware.

Conceptually:



This principle applies to console devices, serial ports, printers, storage devices, and other platform facilities.

CONFIG therefore presents only settings that correspond to capabilities exposed by the current platform. Unsupported settings are omitted or identified as unavailable rather than accepted and silently ignored.

Platform-private information such as register addresses and controller-specific bit fields remains behind the hardware boundary.

7.8 The BIOS Interface

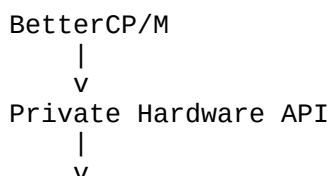
The CP/M BIOS is historically both a hardware interface and an application-visible system interface.

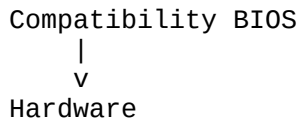
BetterCP/M preserves BIOS behavior where that behavior forms part of the CP/M compatibility contract, but it does not require every internal hardware operation to be structured exactly as a Digital Research BIOS entry.

Where the traditional CP/M BIOS interface provides a suitable internal hardware boundary, BetterCP/M can use or share it directly.

Where BetterCP/M requires an internal operation that does not correspond naturally to a traditional BIOS call, a separate compact internal interface can be used.

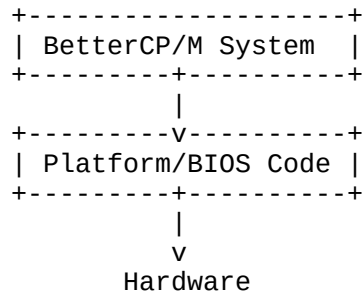
The architecture therefore does **not** require:





merely for the sake of layering.

A more direct organization can be used:



with the required compatibility-visible BIOS entries and BetterCP/M-private hardware entry points sharing implementation and state where appropriate.

This avoids paying a resident-memory and execution-time cost for an abstraction that provides no corresponding benefit.

7.9 Platform Implementation

Each supported BetterCP/M platform supplies the machine-dependent implementation required by the hardware-abstraction boundary.

A platform implementation can contain:

- boot support;
- console-device code;
- character-device code;
- storage-device code;
- hardware initialization;
- reinitialization operations;
- capability descriptions;
- machine-specific configuration data; and
- other hardware-dependent facilities.

The platform defines **how** these operations are performed. It does not redefine the hardware-independent semantics of the operating system.

A new platform should therefore be brought up principally by implementing the required hardware operations and capability descriptions rather than by modifying unrelated portions of the BDOS, command environment, or filesystem.

This is an architectural objective rather than a claim that every possible Z80 machine can be supported without changes to the hardware interface. A new machine can reveal a requirement not anticipated by the existing boundary.

When that happens, the interface can be extended deliberately rather than bypassed with machine-specific dependencies scattered throughout the operating system.

7.10 ROM and RAM Considerations

The hardware architecture also affects where BetterCP/M code and state can reside.

BetterCP/M 1.0 defines a ROM-qualified system profile in which immutable system code can execute directly from write-protected memory.

This imposes a strict distinction between immutable system material and live writable state.

ROM can contain:

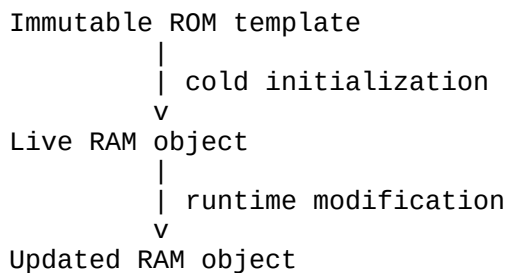
- executable code;
- constants;
- immutable tables;
- immutable descriptors;
- messages;
- cold-initialization code; and
- default templates.

RAM contains live mutable objects such as:

- BDOS and BIOS state;
- disk bindings and selected formats;
- configuration state;
- stacks;
- temporary workspaces;
- extension state; and
- other values changed during operation.

Where a live structure begins with constant default values, the constants can reside in ROM while cold initialization creates the writable live instance in RAM.

Conceptually:



A system does not qualify as execute-in-place merely by storing an image in ROM, copying it to RAM during boot, and then executing the RAM copy.

The ROM profile requires immutable code to execute from protected memory and requires runtime operation, including reconstruction, not to modify it.

The memory architecture is discussed more fully in Chapter 2.

7.11 Hardware Compatibility

Hardware compatibility is distinct from CP/M operating-system compatibility.

Most CP/M applications interact with hardware through BDOS and BIOS interfaces. Such programs can remain portable across machines whose physical hardware differs substantially.

Other software accesses machine-specific hardware directly.

BetterCP/M therefore distinguishes among:

System-service compatibility. The program depends upon the CP/M BDOS environment.

BIOS compatibility. The program depends upon compatibility-visible BIOS behavior.

Platform compatibility. The program depends directly upon the hardware of a particular machine.

A program that writes directly to a Model 4 video address, for example, imposes a Model 4 hardware requirement rather than merely a CP/M requirement.

BetterCP/M does not generalize such dependencies into requirements for every supported platform.

A platform can of course emulate or reproduce particular machine hardware where that compatibility is desired, but this is a platform property rather than a fundamental property of BetterCP/M.

7.12 Portability Boundaries

The hardware-abstraction architecture is intended to make BetterCP/M practical to bring up on additional Z80-family systems without turning portability into a generalized driver framework.

A port should concentrate machine-specific knowledge in:

- boot support;
- platform initialization;
- console and character devices;
- storage backends;
- hardware capability descriptions; and
- other explicitly hardware-dependent operations.

Hardware-independent system components should remain unchanged where the new platform can satisfy their existing requirements.

The architecture also avoids assuming that future storage will continue to resemble floppy disks. SD cards, CompactFlash devices, emulator images, and other block-oriented media can be accommodated by mapping BetterCP/M's logical storage environment onto their capabilities.

Future machines can likewise introduce different console, peripheral, boot, or memory mechanisms.

The hardware-abstraction boundary can evolve when a concrete platform demonstrates a new requirement. BetterCP/M does not reserve large tables, generalized driver managers, or unused resident interfaces merely to anticipate hypothetical hardware.

The goal is narrower and more practical: **machine dependence should remain identifiable, bounded, and replaceable without making the rest of BetterCP/M machine-specific.**

8. Storage and File-System Architecture

BetterCP/M preserves the logical storage environment expected by CP/M software while separating that environment from the physical devices used to contain it. Applications continue to see CP/M drives, files, directories, records, extents, allocation blocks, and File Control Blocks. The operating system translates those logical structures into operations appropriate to the configured disk format and storage backend.

This separation is fundamental to the BetterCP/M storage architecture. A logical drive need not correspond directly to one physical drive, a logical CP/M sector need not correspond to one physical sector, and a CP/M disk format need not describe the native organization of the underlying device.

BetterCP/M therefore distinguishes three related concepts:

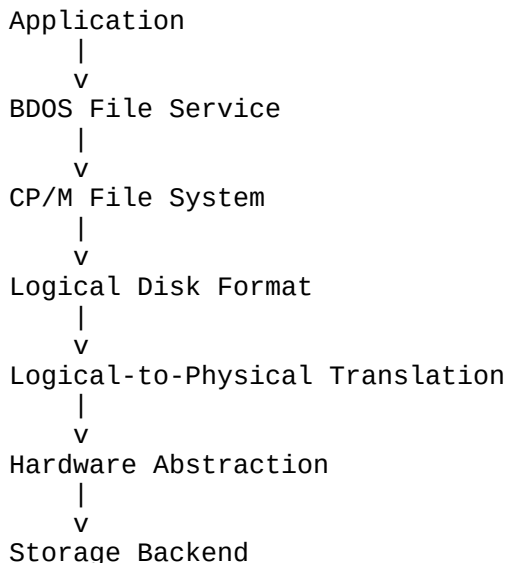
- the CP/M file-system model presented to software;
- the logical disk format used to interpret a volume; and
- the physical or emulated storage mechanism on which that volume resides.

The file-system layer owns CP/M semantics. Disk-format information describes the logical organization required to interpret a volume. Hardware Abstraction performs the device-specific operations needed to access the underlying storage.

8.1 Storage Model

Applications access storage principally through CP/M file and record services.

A typical operation passes through several architectural levels:



Each level has a distinct responsibility.

The application deals with files and records. The file system interprets directories, extents, and allocation. The disk-format layer describes the logical organization of the selected volume. Translation maps that organization onto the storage operations supplied by the platform.

This organization permits BetterCP/M to support conventional floppy formats while avoiding the assumption that floppy geometry is a permanent property of the operating system.

8.2 Logical Drives

BetterCP/M presents the conventional CP/M logical-drive namespace **A:** through **P:**.

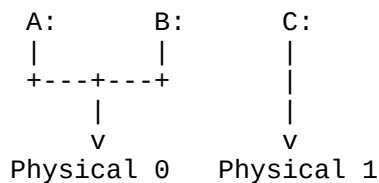
A logical drive represents a configured CP/M storage volume. It does not necessarily represent a separate physical device.

Depending upon the platform, several logical drives can be backed by different physical devices, different media, separate image files, or distinct regions supplied by one storage device.

BetterCP/M therefore treats the relationship between a drive letter and its backing storage as configuration rather than as an intrinsic property of the drive letter.

Conceptually:

CP/M Logical Drives



A platform is not required to provide all sixteen logical drives. Gaps in the configured drive namespace are permitted.

Each configured logical drive also has an associated logical disk format. Two logical drives backed by similar physical devices need not use the same format.

The currently selected drive remains ordinary CP/M system state and is independent of how that drive is physically implemented.

8.3 CP/M Disk Formats

A CP/M disk format defines the logical organization required to interpret a volume.

Important format characteristics include such things as:

- logical sector size;
- sectors per track;
- reserved tracks;
- allocation-block size;
- directory capacity;
- extent organization;
- logical disk capacity; and
- translation information required by the format.

These parameters cannot safely be assumed to have one universal value. Historical CP/M systems used many mutually incompatible disk formats even when the physical media appeared similar.

BetterCP/M therefore avoids embedding one disk geometry throughout the filesystem implementation.

Instead, the selected format supplies the parameters required to interpret the logical disk.

The format describes the disk that the filesystem sees. Device-specific code separately describes how the platform accesses the storage on which that logical disk resides.

Chapter 9 describes BetterCP/M's disk-format description and configuration architecture in detail.

8.4 File-System Model

BetterCP/M retains the CP/M file-system model.

Files are represented through directory entries and are accessed through File Control Blocks. File data is organized as logical records, with larger files represented through the CP/M extent mechanism.

The filesystem provides the semantics required for operations such as:

- create;
- open;
- close;
- search;
- sequential read and write;
- random read and write;
- rename;
- erase; and
- examination or modification of file-related metadata.

These operations are expressed in CP/M terms regardless of the underlying storage device.

The file system is therefore concerned with questions such as:

Which directory entries describe this file?

Which allocation blocks belong to this extent?

Which logical record contains the requested data?

It is not concerned with questions such as:

Which controller register selects this drive?

Which host file contains this emulated disk?

Those belong below the filesystem boundary.

8.5 Directory and Allocation

A CP/M directory contains fixed-size entries describing the files and extents stored on a logical disk.

BetterCP/M interprets these entries according to the selected disk format and the CP/M compatibility requirements.

Directory entries identify such information as:

- user area;
- filename and file type;

- file attributes;
- extent information;
- record count; and
- allocated storage blocks.

The number of available directory entries and the representation of allocation blocks depend upon the disk format.

BetterCP/M maintains the allocation information required to determine which portions of the logical disk are free and which belong to existing files.

Directory capacity and data capacity are distinct resources. A disk can therefore exhaust its available directory entries even though unallocated data blocks remain.

The filesystem must also prevent conflicting allocation. A block allocated to one live file cannot simultaneously be treated as free or assigned independently to another file.

Working allocation state is valid only for the disk interpretation from which it was derived. A media or format change invalidates state whose assumptions no longer describe the selected volume.

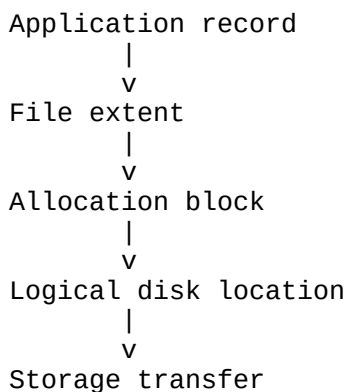
8.6 Records and Extents

CP/M file I/O is based upon **128-byte logical records**.

Applications normally request file operations in terms of these records rather than physical disk sectors.

A file larger than the amount represented by one directory extent uses additional extents. BetterCP/M preserves the extent behavior required by CP/M software while translating the resulting record operations through the configured logical disk format.

The layers therefore remain distinct:



A physical transfer can be larger than a 128-byte CP/M record. Conversely, a logical sector arrangement can require translation before the appropriate physical data is accessed.

The application does not need to know which case applies.

Random and sequential file operations share the same underlying file organization while differing in how the requested logical record is selected.

8.7 Logical-to-Physical Translation

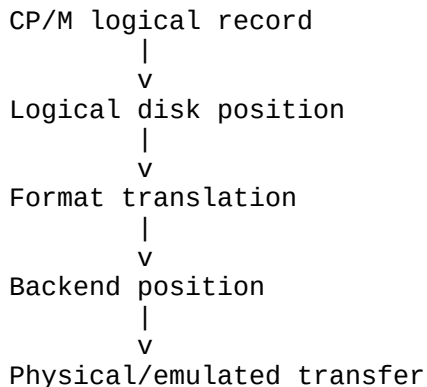
BetterCP/M does not assume that the logical sector numbers used by a CP/M disk format correspond directly to physical sector positions.

Translation can be required for several reasons:

- historical sector skew;
- differing logical and physical sector sizes;
- controller-specific sector numbering;
- emulator image organization;
- partition or region offsets; or
- storage whose native organization does not resemble a floppy disk.

The logical disk format supplies the information needed to interpret CP/M-visible storage. The platform supplies the operations required to reach the underlying medium.

Conceptually:



This boundary permits the same CP/M file-system logic to operate with different disk formats and storage implementations.

It also prevents historical floppy conventions from becoming assumptions embedded throughout BetterCP/M.

8.8 File Attributes

BetterCP/M 1.0 supports the ordinary CP/M read-only, system, and archive attribute bits.

The **read-only** attribute is enforced. Operations that would modify or erase a protected file must respect its read-only state according to the applicable filesystem semantics.

The **system** attribute is preserved as file metadata. BetterCP/M 1.0 does not extend the attribute to provide cross-user-area visibility.

The **archive** attribute is likewise preserved and can be inspected or changed. In BetterCP/M 1.0 it is metadata only; the operating system does not impose an automatic archive or backup policy upon it.

Attributes are properties of the CP/M directory representation rather than characteristics of the physical storage backend.

File operations that preserve a file's metadata should consequently preserve the applicable attributes regardless of the underlying device.

Additional file metadata, including filesystem timestamps, is outside the BetterCP/M 1.0 filesystem architecture.

8.9 Media State and Media Changes

The interpretation of a disk depends upon assumptions about both the selected format and the medium currently present.

BetterCP/M therefore distinguishes configured disk state from observations about removable media.

Changing media can invalidate:

- directory information;
- allocation information;
- cached disk state;
- assumptions about the currently mounted volume; and
- other working information derived from the previous medium.

Such state must not continue to be treated as valid merely because its bytes remain in memory.

The general rule is:

A state object remains valid only while the disk and format assumptions from which it was derived remain valid.

BetterCP/M preserves the CP/M safety behavior needed to prevent stale information about one disk from being used to modify another.

A media observation does not, however, silently select a different disk format. The configured logical format remains authoritative until configuration explicitly changes it.

This distinction prevents accidental media detection from becoming an uncontrolled source of system configuration.

8.10 Storage State

Several kinds of state participate in storage operation, and BetterCP/M assigns them to explicit owners.

These include:

- logical-drive bindings;
- selected disk formats;
- live disk-parameter information;
- directory working state;
- allocation/check vectors;
- media observations;
- physical-device state; and
- saved startup configuration.

These objects do not all have the same lifetime.

Some describe the active logical configuration. Some are derived working state valid only while a particular disk interpretation remains active. Some belong to the hardware backend. Others are saved configuration used to reconstruct the environment at cold boot.

BetterCP/M does not treat all storage-related information as one monolithic "disk state."

In particular, a format catalogue entry can establish the provenance of a selected format without remaining a runtime dependency. Once a format has been normalized into the live logical-drive state, ordinary disk I/O depends upon that live state rather than repeatedly consulting the catalogue from which it originated.

The normalized disk-state architecture is described in Chapter 9.

8.11 Storage Compatibility

BetterCP/M's storage architecture is designed to preserve CP/M-visible behavior without requiring the underlying device implementation to resemble the original CP/M hardware.

Compatibility applies principally to:

- file-system semantics;
- FCB behavior;
- records and extents;
- directory representation where visible to software;
- logical drive behavior;
- disk-reset and media-change behavior;
- file attributes; and
- other adopted CP/M storage conventions.

A CP/M application using ordinary BDOS file services should not need to know whether its logical disk resides on a physical floppy, emulator image, or another supported backend.

Software that bypasses BDOS and directly examines disk structures presents a stronger compatibility requirement. Such software can depend upon the exact logical disk format.

Software that accesses a physical controller directly imposes an additional platform-hardware requirement.

BetterCP/M therefore distinguishes:

```
File-system compatibility
|
Logical disk-format compatibility
|
Physical hardware compatibility
```

A program can require one, two, or all three.

This distinction allows BetterCP/M to preserve historical CP/M disk formats where needed without making historical storage hardware a permanent architectural requirement.

8.12 Storage Evolution

The storage architecture deliberately leaves room for media that differ substantially from the floppy-disk systems on which CP/M was originally developed.

Potential backends can include:

- additional floppy formats;
- emulator disk images;

- hard-disk-like devices;
- SD storage;
- CompactFlash storage; and
- other block-oriented media.

Supporting such devices does not require changing the CP/M file-system model merely because their physical organization differs.

Instead, BetterCP/M maps the required logical storage environment onto the capabilities of the backend.

Future storage extensions can also introduce additional logical disk formats or mappings between logical drives and physical devices.

Such development must preserve the separation among filesystem semantics, disk-format interpretation, and hardware access.

BetterCP/M 1.0 does not attempt to anticipate every future storage technology with a generalized storage framework. It establishes the boundaries needed by the formats and platforms actually supported while avoiding assumptions that would unnecessarily tie the operating system to traditional floppy hardware.

9. Disk Formats and Disk State

BetterCP/M separates the logical format of a CP/M disk from the physical device that contains it. The operating system therefore requires an explicit description of each logical disk format and an authoritative runtime representation of the format currently assigned to each configured drive.

This is more than a catalogue of floppy geometries. Disk-format information participates directly in filesystem operation: it determines logical capacity, directory organization, allocation, translation, and the memory required for filesystem working state. An incorrect or incomplete format description can therefore corrupt a volume.

BetterCP/M 1.0 uses **Disk Format Definitions (FDFs)** to describe available formats and normalized live disk state to represent the formats actually in use. Catalogue identity records where a configuration originated; it is not required for ordinary I/O once the selected format has been normalized into the active drive state.

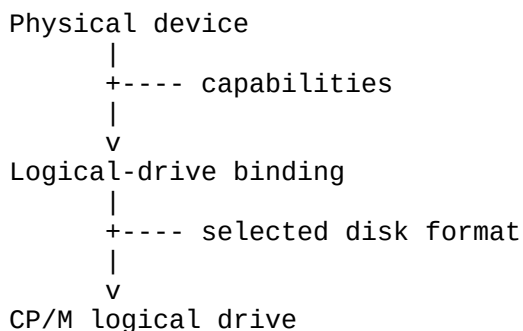
The architecture consequently distinguishes among format definition, physical-device capability, logical-drive binding, active runtime state, saved startup configuration, and observations about removable media.

9.1 Physical Drives and Logical Bindings

A physical storage device and a CP/M logical drive are separate objects.

The platform describes the physical devices it can access and the capabilities of those devices. BetterCP/M configuration then binds logical drive letters to appropriate physical storage and assigns a logical disk format to each binding.

Conceptually:



A physical device can potentially support more than one logical format, and the same format can be used on more than one suitable device.

Logical drives are independently configured. BetterCP/M does not require all drives attached to similar hardware to use the same disk format.

The **A:** through **P:** namespace can contain gaps. An unconfigured logical drive does not need a dummy physical binding or format merely to make the namespace contiguous.

A binding is valid only when the physical backend can support the requirements of the selected logical format.

9.2 Disk Geometry

A disk format describes the logical geometry and filesystem parameters needed to interpret a CP/M volume.

Relevant values include such characteristics as:

- logical sector organization;
- sectors per logical track;
- reserved system tracks;
- disk capacity;
- allocation-block organization;
- directory size;
- extent-related parameters; and
- sector translation where required.

These values describe the **logical disk**. They need not reproduce the native geometry of the underlying device.

For a traditional floppy format, logical and physical geometry can be closely related. For an emulator image or block-oriented storage backend, the relationship can be largely synthetic.

BetterCP/M therefore does not infer filesystem parameters merely from the apparent type of physical device.

The complete logical format is selected explicitly and validated as a unit.

9.3 Disk Format Definitions

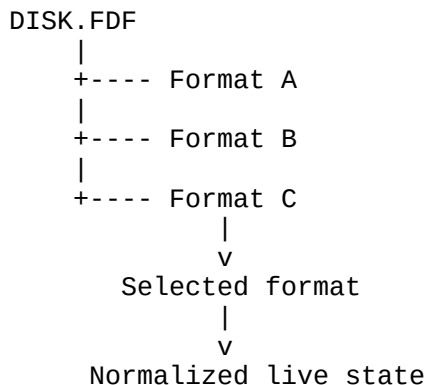
BetterCP/M stores disk-format descriptions in **Disk Format Definition** form.

An FDF contains the information needed to describe a supported logical disk format independently of the particular drive to which it will eventually be assigned.

Format definitions can be supplied through a format catalogue such as **DISK.FDF**.

This allows BetterCP/M to maintain a library of known formats without embedding every supported geometry directly into executable filesystem code.

Conceptually:



The catalogue is a source of format definitions. It is not itself the live disk-control structure used for every subsequent I/O operation.

This distinction allows the active system to remain self-contained after a format has been selected and normalized.

9.4 Format Selection and Provenance

A configured logical drive retains enough information to identify the format from which its active configuration was derived.

BetterCP/M distinguishes this **provenance** from runtime authority.

A format catalogue entry can answer:

Which named format did this configuration originally come from?

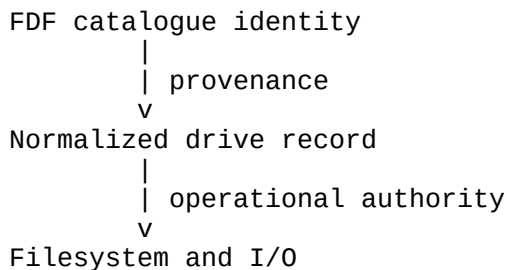
The normalized live drive state answers:

What parameters does the running system actually use to access this drive?

Ordinary disk I/O depends upon the latter.

The distinction is important because a catalogue can change, become unavailable, or contain a later definition under the same catalogue identity. The running system must not reinterpret an already validated active disk merely because an external catalogue has changed.

Thus:



Catalogue identity is useful for configuration, display, editing, and reconstruction of saved settings. It is not a substitute for the complete active format state.

9.5 Format Validation

A disk-format definition must be validated before BetterCP/M adopts it as authoritative state.

Validation ensures that the format is internally coherent and can be represented within the capabilities and resource limits of the running system.

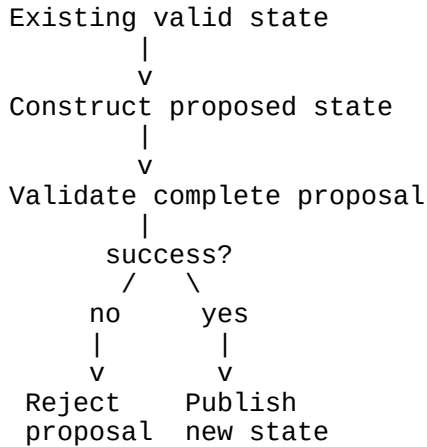
This can include checking:

- parameter ranges;
- logical capacity;
- allocation relationships;
- directory requirements;
- translation information;
- compatibility among related format fields;
- physical-device capability; and
- RAM required by the resulting filesystem structures.

Validation applies to the complete resulting record rather than merely to whichever field the user most recently edited.

BetterCP/M does not publish a partially changed disk configuration and then attempt to repair it after other components have begun using it.

The intended transition is:



Failure leaves the existing authoritative configuration unchanged wherever the operation permits.

9.6 Normalized Disk State

Once a disk format has been selected and validated, BetterCP/M converts the required information into a **normalized, self-contained runtime representation**.

This normalized state is the operational authority for the logical drive.

Filesystem and disk-I/O code should not repeatedly return to the FDF catalogue to rediscover values already established for the active configuration.

The normalized representation contains the information required for operation regardless of how that information was originally expressed in the catalogue.

This provides several advantages:

- runtime operation does not depend upon continued catalogue access;
- configuration provenance is separated from operational state;
- different source representations can produce the same normalized state;
- validation occurs before publication rather than during arbitrary I/O; and
- filesystem code operates upon a stable representation.

Live pointers required by disk operation refer to the complete live runtime structures rather than to temporary catalogue or configuration buffers.

9.7 Active and Saved Disk Configuration

BetterCP/M distinguishes the disk configuration used by the running system from the configuration saved for the next cold boot.

The **active configuration** describes the current logical-drive bindings and formats.

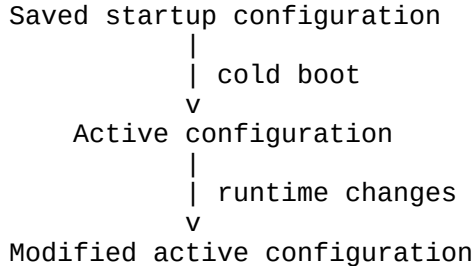
The **saved configuration** supplies the defaults from which a later cold boot constructs its initial active state.

A configuration utility can therefore change a setting in one of several ways depending upon the nature of the change:

- apply it to the current session;
- save it for a future cold boot; or
- record it as a pending cold-boot change when immediate application is inappropriate.

A warm boot does not silently replace the active disk configuration with the saved defaults.

This distinction follows the general BetterCP/M configuration model:



Saving the current or pending state is a deliberate operation.

Disk configuration and system installation are also distinct. **CONFIG** defines the system's storage policy; **SYSGEN** installs or copies a system image onto appropriate media.

9.8 Runtime Authority and Provenance

Several pieces of information can describe aspects of the same logical drive, but only one can be authoritative for a particular purpose.

BetterCP/M therefore assigns explicit roles:

Physical-device records describe the operational characteristics of the available storage backend.

Normalized logical-drive bindings describe the active logical disk and are authoritative for filesystem and disk operation.

FDF/FDB identity records the provenance of a selected format.

Saved configuration describes what should be reconstructed at a future cold boot.

Media observations describe what the system has observed about removable media.

These objects are related but not interchangeable.

In particular:

FDF identity is not runtime disk state.

and:

Media observation is not configuration authority.

This prevents a disk from silently changing interpretation because the system noticed different media or because a catalogue entry happened to change.

A format change occurs only through the configuration mechanisms that own such a change.

9.9 Disk Capabilities

A physical backend publishes the capabilities relevant to disk configuration and operation.

A format can be assigned only where those capabilities are sufficient to implement it.

Capability information can describe such matters as:

- whether the device is present;
- whether it is writable;
- supported transfer characteristics;
- removable-media behavior;
- capacity constraints;
- formatting capabilities; and
- other platform-specific limitations.

The configuration interface uses these capabilities to determine which operations and formats are meaningful.

Unsupported operations are not treated as successful no-ops.

For example, a backend that can read and write an image but cannot physically format media should not advertise a formatting operation merely because the logical disk format itself is understood.

The same principle used elsewhere in BetterCP/M applies here:

The platform describes capability; configuration selects among capabilities that actually exist.

9.10 Filesystem Working Memory

Disk configuration affects not only storage interpretation but also RAM consumption.

CP/M filesystems require working structures such as allocation and check vectors whose sizes depend upon the configured logical disk.

A larger logical volume can therefore require more resident RAM than a smaller one.

BetterCP/M accounts for this relationship when validating disk configuration. A format is not acceptable merely because its logical geometry is internally valid; the running system must also be able to provide the memory required to operate it.

This is particularly important in BetterCP/M because resident memory competes with the TPA.

CONFIG can therefore report the memory consequence of a proposed disk configuration before it is committed.

The resulting RAM belongs to the disk subsystem rather than to the Persistent Data Segment. Disk state does not become PDS state merely because it persists while the disk configuration remains active.

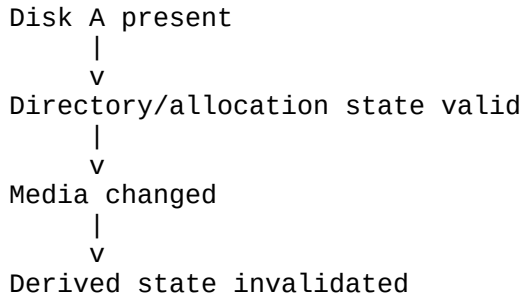
9.11 Media Observation

Removable media introduces information that is inherently observational.

The system can discover that a medium has changed, that a drive is not ready, or that another media-related condition has occurred.

Such information affects the validity of working filesystem state.

For example:



What the observation does **not** do is select another logical disk format.

If drive **A:** is configured for a particular format, inserting media that appears different does not authorize the operating system to reinterpret **A:** under another format automatically.

The system can reject access, invalidate working state, report an error, or require explicit reconfiguration as appropriate.

This separation protects the distinction between **what the system observes** and **what configuration declares**.

9.12 Configuration Changes

Changing a logical-drive binding or disk format is a state transition rather than merely an edit to one field.

BetterCP/M constructs and validates the resulting complete state before making it authoritative.

A successful change can require:

- invalidating directory and allocation state;
- rebuilding live disk structures;
- updating logical-to-physical translation;
- reallocating or rebuilding filesystem working memory;
- updating configuration provenance; and
- notifying or unconfiguring dependent bindings where required.

A synchronous configuration operation does not return success while leaving some of these required changes pending invisibly.

If a change cannot be applied safely to the current session, it can instead become a pending cold-boot change.

The user-facing configuration mechanism is described in the User's Guide. The important architectural property is that configuration changes preserve one coherent authoritative disk state.

9.13 Unsupported Formats and Operations

BetterCP/M distinguishes between a format it understands and an operation the current platform can actually perform.

A format definition can be valid while a particular physical backend lacks the capabilities required to use it.

Likewise, a backend can support ordinary I/O without supporting operations such as low-level formatting.

BetterCP/M does not silently approximate unsupported behavior.

An unsupported format, impossible binding, invalid parameter combination, or unavailable operation is rejected explicitly.

The old valid state remains authoritative where the attempted operation can be rejected before publication.

This is particularly important for storage because accepting a configuration that cannot actually be honored can lead to filesystem corruption rather than merely loss of a convenience feature.

9.14 Disk-State Invariants

Several rules summarize the BetterCP/M 1.0 disk-state architecture.

A logical drive and a physical storage device are distinct objects.

Each configured logical drive has a complete, authoritative logical format.

The active normalized disk state, not the external format catalogue, governs ordinary runtime I/O.

Format-catalogue identity records provenance rather than replacing operational state.

Media observation never silently selects another disk format.

A disk-format change invalidates working state derived from the former format.

Configuration changes are validated as complete resulting states before publication.

Unsupported operations fail explicitly rather than being silently ignored.

Filesystem working memory belongs to the storage subsystem and is accounted for as part of the resident-memory budget.

Saved startup configuration and active runtime configuration remain distinct.

These rules allow BetterCP/M to support multiple CP/M disk formats and varied storage backends without allowing the interpretation of a volume to become ambiguous.

The objective is not merely format flexibility. It is **one authoritative interpretation of each configured logical disk at any given time**, with every other piece of disk-related information assigned a clearly defined role.

10. System State and Configuration

BetterCP/M distinguishes between the code that implements the operating system and the mutable state that describes its current operating environment. The selected drive, user area, DMA address, disk bindings, device assignments, extension profiles, and other runtime values are state: they can change while the underlying system code remains unchanged.

Every item of system state has a defined owner, representation, and lifetime. BetterCP/M avoids maintaining independent authoritative copies of shared information, since such copies can diverge and leave different parts of the operating system operating under different assumptions.

Configuration is a related but narrower concept. Configuration selects operating-system policy and parameterizes capabilities supplied by the platform and system components. BetterCP/M distinguishes the configuration active in the current session from saved startup defaults and from changes that have been prepared for the next cold boot.

This allows the running system to be configured without making every setting immediately mutable, and allows a warm boot to preserve the current environment rather than silently restoring saved defaults.

10.1 System-State Model

System state is information retained by BetterCP/M beyond the immediate execution of a single instruction or service call.

Examples include:

- current drive;
- current user area;
- current DMA address;
- active disk configuration;
- device assignments;
- active CPX and RSX profiles;
- command history;
- startup state;
- hardware-dependent state; and
- filesystem working state.

Not all state has the same lifetime.

Some values survive only while a particular operation is in progress. Others survive command execution, transient programs, warm boot, or resident reconstruction. Saved configuration can survive a complete shutdown and be used to construct a new runtime environment at the next cold boot.

BetterCP/M therefore treats state according to ownership and lifetime rather than simply according to where the bytes happen to reside.

The continued presence of a value in memory does not establish that the value remains valid.

10.2 State Ownership

Every authoritative item of BetterCP/M state has an owner.

The owner is the subsystem responsible for defining:

- what the state means;
- where its authoritative representation resides;
- which components may examine it;
- which components may modify it;
- when it becomes valid;
- when it becomes invalid; and
- how it is initialized, preserved, reconstructed, or discarded.

For example, the Command Environment can select the current drive, but that does not make the CCP the private owner of drive state. Applications and System Services must observe the same current drive.

Similarly, **CONFIG** can modify disk configuration without becoming the owner of the live disk structures used by the filesystem.

The architectural rule is:

A component that edits or exposes state does not necessarily own that state.

Shared state should have one authoritative representation. Cached or derived copies are permitted where useful, but their dependence upon the authoritative state and the conditions under which they become invalid must be defined.

10.3 Global System State

Some state describes the operating environment as a whole rather than one transient application or subsystem operation.

Examples include:

- current drive;
- current user area;
- active system configuration;
- active resident-extension configuration;
- logical-device assignments; and
- other values whose effects are visible across commands or applications.

Such state must remain coherent when control moves between the Command Environment, applications, System Services, and resident extensions.

A transient application can alter some global state through the normal CP/M interfaces. When the application terminates, the resulting valid state remains part of the operating environment unless its contract says otherwise.

The Command Environment therefore does not reconstruct its own independent version of global state when it returns after transient execution.

Reconstruction restores command-processing code and command-private state; it does not imply that the entire operating system has returned to its previous values.

10.4 Command and Program State

The Command Environment and the currently executing transient program each maintain state appropriate to their roles.

Command-environment state can include:

- the line currently being edited;
- parsing state;

- command-dispatch state;
- CPX state;
- command history; and
- information needed to initiate a transient program.

Much of this state is reclaimable. The CCP and CPXs can be overwritten while a transient program executes and reconstructed afterward.

State that must survive that destruction cannot depend upon the continued presence of the old CCP or CPX image.

Command history is the principal BetterCP/M 1.0 example. It survives transient execution and warm boot and therefore resides in the bounded Persistent Data Segment rather than in reclaimable command memory.

Application-private state, by contrast, belongs to the transient program while it executes. BetterCP/M does not treat arbitrary data left in the TPA after termination as persistent system state.

10.5 Storage State

Storage operation produces several kinds of system state.

These include:

- logical-drive bindings;
- selected disk formats;
- live disk structures;
- directory working state;
- allocation and check vectors;
- physical-device state; and
- media observations.

Their ownership and lifetime differ.

The normalized logical-drive state is authoritative for current disk operation. Directory and allocation working information is derived from that environment and remains valid only while the assumptions from which it was constructed remain true.

A media change, disk reset, format change, or binding change can therefore invalidate derived storage state.

Saved disk configuration has another lifetime again: it describes what a future cold boot should construct rather than serving directly as the live I/O structure.

The disk-state architecture is described in Chapter 9.

10.6 Configurable System State

BetterCP/M represents settings that can reasonably vary without rebuilding the operating system as configuration data where practical.

Configuration can include such things as:

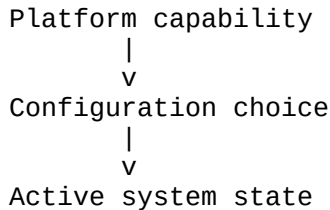
- logical-device assignments;
- platform-supported keyboard settings;
- printer/list-device policy;
- serial settings;
- logical-drive bindings;

- disk formats;
- CPX startup profile;
- RSX startup profile; and
- the startup command.

Not every hardware characteristic is configurable.

The platform defines the capabilities it actually provides. Configuration selects among or parameterizes those capabilities where runtime or startup selection is meaningful.

Thus:



A setting that the platform cannot implement is not meaningful configuration merely because another platform supports it.

This capability-driven model prevents **CONFIG** from becoming a collection of controls whose effects depend upon assumptions about one particular machine.

10.7 Configuration States

BetterCP/M distinguishes three configuration states:

Active configuration.

The settings currently used by the running system.

Saved startup defaults.

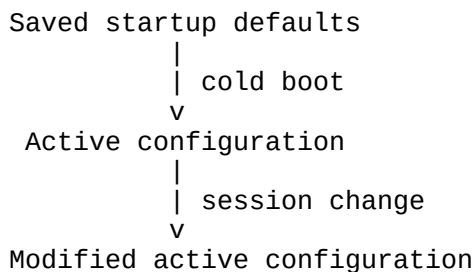
The settings from which the next cold boot constructs its initial configuration.

Pending cold-boot changes.

Changes that have been selected but cannot or should not alter the current running environment.

These states are deliberately separate.

For example:



A runtime change does not automatically become a permanent startup default.

Similarly, editing a saved value does not necessarily force the running system to reconstruct itself immediately.

Every configurable setting should make clear whether a change:

- takes effect immediately;
- applies only to the current session;
- is saved for future startup; or
- requires a cold boot.

A warm boot does not silently replace active settings with the saved startup configuration.

10.8 State Lifetime

State lifetime describes the transitions across which a value remains meaningful.

Important BetterCP/M transitions include:

- completion of a system-service call;
- completion of a command;
- transient-program invocation;
- transient-program termination;
- warm boot;
- command-environment reconstruction;
- resident-extension reconstruction;
- media change;
- configuration change; and
- cold boot.

Different state survives different subsets of these transitions.

For example:

Command input under construction
-> discarded when command completes

Application workspace
-> discarded when transient execution ends

Command history
-> survives transient execution and warm boot
-> reset by cold boot

Saved startup configuration
-> survives shutdown
-> used to construct the next cold-boot environment

State lifetime is therefore an architectural property, not an accidental consequence of whether RAM happened to retain its contents.

10.9 State Transitions

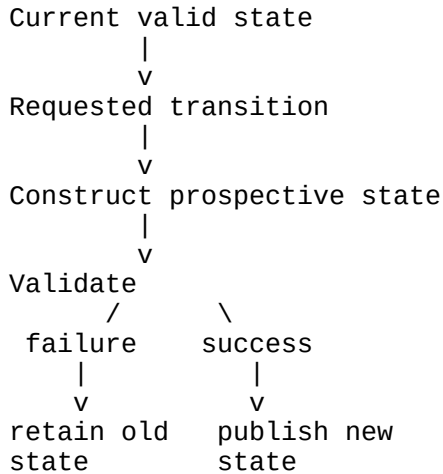
A state transition can make previously valid information invalid.

BetterCP/M requires such transitions to be explicit.

For example, changing the format assigned to a logical drive invalidates directory and allocation information derived from the former format. Reconstructing an RSX can invalidate previously discovered callable addresses.

A cold boot invalidates session state and constructs a new runtime environment from saved defaults.

The general pattern is:



Where possible, BetterCP/M constructs and validates the prospective state before replacing the existing authoritative state.

This transactional approach is particularly important for resident-extension and disk-configuration changes, where publication of a partially valid configuration could make the system unusable.

A transition that cannot preserve the old state must instead have a defined recovery path.

10.10 Active, Pending, and Saved Configuration

The distinction among active, pending, and saved state allows BetterCP/M to handle settings whose application costs differ.

Some settings can be changed synchronously. A logical device assignment, for example, can potentially take effect immediately if the selected provider is already available.

Other settings affect memory layout or startup construction and can be more appropriately applied at cold boot.

CONFIG therefore does not present every edit as though it were an immediate mutation of the running system.

A user can also choose between two save scopes:

- save pending cold-boot changes; or
- save the current configuration as the startup defaults.

Before writing saved configuration, BetterCP/M validates the complete resulting startup configuration.

Saving configuration is not system installation. The saved settings belong to the currently installed BetterCP/M system; **SYSGEN** performs the separate operation of installing or copying a system image.

10.11 Device Configuration

BetterCP/M retains the CP/M logical character-device model while allowing the available physical destinations to vary by platform.

The logical devices include:

CON:
RDR:
PUN:
LST:

Applicable IOBYTE compatibility is preserved.

A platform publishes the providers available for these logical devices. Depending upon the machine, these can include:

- console devices;
- serial ports;
- parallel ports;
- emulator endpoints;
- host-file endpoints; or
- a null device.

The configuration system selects among providers that actually exist.

Device-specific settings remain capability-driven. A real serial interface can expose baud rate, parity, word length, stop bits, and hardware flow-control settings. An emulator endpoint need expose only those settings that genuinely affect its operation.

BetterCP/M does not simulate meaningless configuration fields merely to make every platform's CONFIG display identical.

10.12 Disk Configuration

Disk configuration is one of the more substantial parts of BetterCP/M configuration.

It includes:

- physical-drive capabilities;
- logical-drive-to-device bindings;
- independent logical formats;
- logical-drive capacity;
- format catalogue selection;
- saved disk configuration; and
- the memory consequences of the selected configuration.

Disk state has its own authoritative runtime representation, as described in Chapter 9.

CONFIG edits the policy that produces or modifies that state. It does not maintain a second private disk catalogue or duplicate the filesystem's operational structures.

A complete proposed disk configuration is validated before publication or saving.

Where a change affects another binding or makes an existing non-system binding impossible, the configuration operation reports the consequence explicitly rather than silently leaving an invalid mapping.

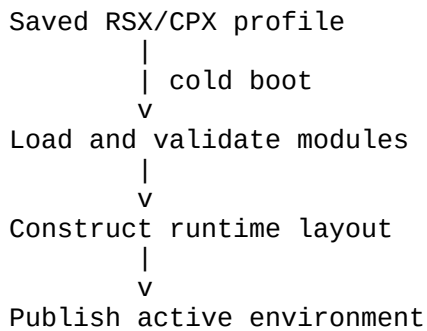
10.13 Resident Startup Profiles

BetterCP/M can save CPX and RSX profiles for construction during cold boot.

These profiles identify the extensions that should form part of the initial command and resident-service environments and preserve their configured ordering.

The profile is configuration. The current runtime addresses of those modules are not.

Thus:



Changing a profile can affect memory use. An RSX consumes protected resident memory and can lower the TPA ceiling. A CPX consumes reclaimable command-environment space but does not impose the same cost during transient execution.

Configuration tools can therefore display the memory consequences of the selected profiles before they are saved or applied.

The extension architecture is discussed in Chapters 12 through 15.

10.14 Startup Command

BetterCP/M 1.0 supports one optional saved startup command.

The saved record contains an enabled/disabled state and one ordinary CCP command line. An empty command disables startup execution.

For a longer startup sequence, the command can invoke a SUBMIT file, for example:

SUBMIT STARTUP

The startup command executes **once** after a successful true cold initialization, after the devices, drives, resident extensions, services, and command environment required to execute it have been established.

It does not execute after:

- ordinary warm boot;
- transient-program termination;
- command-environment reconstruction; or
- resident reconstruction that occurs within the running session.

BetterCP/M records that startup execution has begun before dispatching the command. A failed command therefore does not cause an automatic retry loop.

A documented recovery mechanism can suppress startup execution for one boot without erasing the saved command.

This ensures that a faulty startup command cannot permanently prevent the user from reaching the command prompt.

10.15 Configuration Ownership

CONFIG is the principal user-facing configuration utility, but it does not own every item it can inspect or change.

This distinction follows the general state-ownership rule.

For example:

CONFIG edits disk policy

-> Disk subsystem owns live disk state

CONFIG selects RSX profile

-> Extension subsystem owns active RSX state

CONFIG selects device routing

-> Appropriate system/platform state remains authoritative

CONFIG saves startup command

-> Startup lifecycle machinery owns execution state

Provider-specific state remains with the provider.

Stacks and temporary workspaces remain with their natural subsystem owners.

The resident-service registry remains derived extension-subsystem state.

The BetterCP/M 1.0 Persistent Data Segment remains owned solely by command history.

This prevents CONFIG from becoming a central database through which unrelated subsystems must route their private runtime state.

10.16 Configuration Recovery

Configuration mechanisms must preserve a path back to a usable system.

A proposed configuration is therefore validated before it is committed wherever possible.

Invalid extension profiles, impossible disk bindings, unsupported platform settings, or other detectable errors are rejected before becoming authoritative.

Saved startup configuration receives the same treatment. BetterCP/M does not deliberately write a startup configuration known to be internally invalid.

Some failures can nevertheless occur only when the system is subsequently booted or when hardware is actually accessed.

BetterCP/M therefore provides recovery behavior that permits optional startup configuration to be bypassed or reduced sufficiently to regain control.

The startup command can be suppressed for a recovery boot. Optional resident configuration must likewise not turn a recoverable configuration mistake into permanent loss of the system disk.

Detailed recovery procedures belong in the User's Guide. Architecturally, the important requirement is:

Configurable facilities may fail, but the configuration architecture must preserve a defined route to a usable BetterCP/M environment.

This principle, together with explicit ownership and lifetime, allows BetterCP/M to provide substantially more configurable behavior than CP/M 2.2 without making the running system's state ambiguous or uncontrolled.

11. Persistent System State

Some BetterCP/M state must survive transitions that destroy or reconstruct other parts of the operating environment. Command history, for example, must remain available after a transient program overwrites the reclaimable command environment and after the CCP and CPXs are subsequently reconstructed.

BetterCP/M provides a **Persistent Data Segment**, or **PDS**, for state whose ownership contract specifically requires PDS discovery, versioning, and preservation across command-environment reconstruction and ordinary warm boot.

The PDS is deliberately not a general-purpose area for writable resident data. BetterCP/M 1.0 has exactly one PDS owner: the built-in command-history facility. Other persistent or resident state remains with the subsystem that naturally owns it.

This narrow definition is important. Persistence is a property of a state's required lifetime, not merely of its physical location in high memory.

11.1 Persistent-State Model

BetterCP/M distinguishes several classes of writable resident state:

Bounded PDS state.

Versioned persistent objects whose lifetime and discovery requirements explicitly require the PDS.

Fixed subsystem state.

Ordinary mutable state belonging to the BDOS, BIOS, disk subsystem, extension subsystem, gateway, or another fixed component.

Stacks.

Execution storage with explicitly defined ownership and size.

Shared system workspaces.

Temporary storage that can be overlaid where lifetime analysis demonstrates that the users cannot be active simultaneously.

These categories can occupy adjacent physical memory without acquiring the same ownership semantics.

In particular:

Writable does not mean PDS.

and:

Persistent does not automatically mean PDS.

An object belongs in the PDS only when its architectural contract requires the services and lifetime associated with the PDS.

11.2 The Persistent Data Segment

BetterCP/M 1.0 provides one fixed 192-byte Persistent Data Segment.

Its address range is:

D504H - D5C3H

The segment is protected from transient-program use and remains available while the reclaimable command environment is destroyed and reconstructed.

The PDS is discovered through the BetterCP/M extension-control environment rather than by requiring clients to assume undocumented memory placement. The version-1 Extension Control Block publishes the information required by the 1.0 PDS contract.

The PDS does not contain:

- a heap;
- an allocation directory;
- a general owner table;
- dynamically allocated objects;
- configurable sizing;
- runtime growth; or
- a migration facility.

Those mechanisms are not required by BetterCP/M 1.0.

The complete 1.0 PDS is one fixed object owned by command history.

11.3 BetterCP/M 1.0 PDS Layout

The 192-byte PDS contains the version-1 command-history object.

Its layout is:

Offset	Size	Meaning
0	2	Signature "BH"
2	1	Structure version
3	1	Packed-record count
4	2	Packed bytes used
6	4	Owner-private working state
10	182	Packed command records

The signature identifies a BetterCP/M history object.

The structure version is currently 1.

The record count identifies the number of complete history records currently stored, while the used-length field records the number of bytes occupied by packed history data.

The four bytes beginning at offset 6 belong privately to the history implementation. They are not general PDS metadata and do not form part of a future allocation directory. Their values need not possess persistent semantics.

The remaining 182 bytes contain length-prefixed packed command records.

The representation is intentionally compact because every byte of protected high memory competes with transient-program capacity.

11.4 Command-History State

Command history is the sole owner of the BetterCP/M 1.0 PDS.

When a command is accepted into history, it is stored as a complete packed record.

As additional commands are inserted, the history owner updates the record count, used length, working state, and packed records within its fixed allocation.

The history never grows beyond the 192-byte PDS boundary.

If there is insufficient room for a new command, the implementation can discard the oldest complete records until sufficient space becomes available.

A record is not partially retained merely to use every remaining byte.

Conceptually:

```
+-----+
| newest command      |
+-----+
| previous command   |
+-----+
| previous command   |
+-----+
| ...                |
+-----+
| oldest retained command |
+-----+
```

As new records require space, complete old records are evicted.

Failure to add one command must leave the PDS valid and must not impair continued operation of the CCP.

11.5 Cold-Boot Initialization

A true cold boot establishes a new BetterCP/M runtime instance and therefore initializes command history to a valid empty version-1 object.

Cold initialization establishes:

- the "BH" signature;
- structure version 1;
- zero retained command records;
- zero packed bytes used; and
- valid owner-private working state.

A cold boot does not treat apparently valid bytes left by a previous execution as a reason to preserve the former history.

This distinction is architectural rather than dependent upon whether the underlying RAM physically retained its contents.

Conceptually:

```
Cold boot
  |
  v
Initialize PDS
  |
  v
Valid empty history
```

The 1.0 architecture requires unconditional cold initialization of the history object.

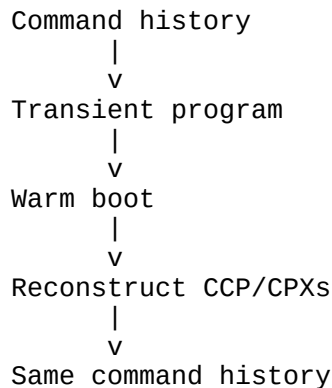
11.6 Warm-Boot Preservation

Ordinary warm boot does not create a new BetterCP/M runtime instance.

A valid PDS therefore survives warm boot.

The same applies to reconstruction of the CCP and CPXs and to resident-layout changes that preserve the running system instance.

Conceptually:



The history object is preserved byte for byte across these transitions unless the history owner itself legitimately modifies it.

This permits command history to remain available even though the memory containing the command processor has been overwritten by the transient application.

The PDS consequently provides persistence independently of command-environment residency.

11.7 Validation and Recovery

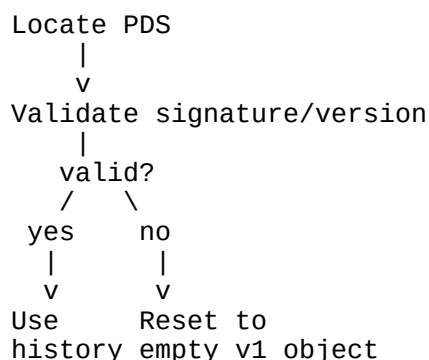
The history owner validates the PDS before using its contents.

At minimum, the signature and structure version must identify a valid history object.

If the header is invalid or identifies an unsupported version, BetterCP/M does not attempt to interpret the record area as command history.

Instead, it resets the PDS to a valid empty version-1 object.

Thus:



Validation occurs before command records are consumed.

This provides a simple recovery rule: damaged or unknown history state results in loss of history, not loss of the command environment.

The history facility is therefore optional state in the practical sense that corruption of its contents must not prevent BetterCP/M from reaching a usable prompt.

11.8 PDS Discovery

BetterCP/M does not require command-history code to depend solely upon a private hard-coded address.

The existing Extension Control Block provides the BetterCP/M 1.0 PDS discovery contract.

It identifies the validity and lower boundary of the fixed PDS, while the system gateway supplies the corresponding upper boundary.

For BetterCP/M 1.0, the current and boot PDS sizes are both 192 bytes.

The discovery mechanism provides a defined interface even though the allocation itself is fixed.

This distinction allows software to depend upon:

the PDS published by BetterCP/M

rather than:

whatever bytes happen to be at D504H.

The current physical address remains useful when describing the 1.0 memory map, but the architectural identity of the object comes from its published contract.

11.9 PDS Ownership Exclusions

Several BetterCP/M structures survive long enough that they might superficially appear to belong in the PDS. They do not.

The BetterCP/M 1.0 PDS explicitly excludes:

- the RSX reconstruction profile;
- the CPX reconstruction profile;
- startup-command lifecycle state;
- the resident-service registry;
- normalized disk state;
- configuration state;
- clock-provider state;
- stacks; and
- temporary or overlaid workspaces.

These objects retain their natural owners.

For example:

Command history
-> PDS

RSX profile
-> extension subsystem

CPX profile
-> gateway/command reconstruction state

Disk bindings
-> disk subsystem

Clock-provider state
-> provider RSX

Stacks
-> owning execution subsystem

Physical relocation of one of these structures in a future memory layout does not change its ownership.

An object does not become PDS state merely because it moves next to the PDS.

11.10 Persistent State and RSXs

Optional feature state normally belongs to the RSX that implements the feature.

An installed RSX can remain resident across transient execution and warm boot and can carry its own mutable state according to its reconstruction contract.

This makes the RSX itself the natural owner of such state.

For example, a clock provider can maintain provider-specific state inside its resident allocation without placing that state in the PDS.

If the RSX is explicitly unloaded, its ordinary private state is discarded with it.

This creates a useful ownership distinction:

State that should exist while the module exists belongs to the module.

State that must survive removal or replacement of the module might justify a different persistence mechanism.

BetterCP/M 1.0 has no accepted optional feature requiring such independent persistence.

The PDS is therefore not used as a convenient dumping ground for RSX state.

11.11 Persistent State and ROM Systems

ROM qualification reinforces the distinction between persistence and writability.

Immutable system code and constant templates can reside in ROM. Live persistent state must reside in writable RAM.

For command history:

```
Immutable initialization knowledge
      |
      v
Cold initialization
      |
      v
192-byte live PDS in RAM
      |
      v
Runtime history changes
```


The same principle applies to other system state.

A constant default can be stored in ROM, but the live object produced from that default belongs in RAM if runtime code modifies it.

The PDS is therefore part of the writable system-state environment even when most BetterCP/M executable code is running directly from ROM.

ROM qualification does not turn ordinary mutable subsystem state into PDS state merely because that state must now be separated physically from executable code.

11.12 PDS and the TPA

The PDS occupies protected memory and therefore contributes directly to the BetterCP/M resident-memory budget.

For the BetterCP/M 1.0 layout, the PDS lower boundary is:

D504H

and the exclusive TPA ceiling is:

D501H

A transient program beginning at **0100H** therefore has 54,273 addressable bytes below that ceiling.

Because CP/M **.COM** files are loaded in complete 128-byte records, the largest complete-record image is:

54,272 bytes

or exactly:

53 KiB

This is the accepted BetterCP/M 1.0 default floor.

The fixed PDS design is consequently not only simpler than a general allocator; it also makes the memory cost of BetterCP/M 1.0 persistence explicit and bounded.

Future changes that reduce this accepted complete-record capacity require an explicit release-scope decision rather than occurring accidentally as persistent facilities accumulate.

11.13 Shared Workspaces Are Not Persistent State

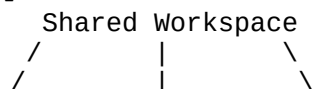
BetterCP/M can save resident RAM by allowing temporary workspaces to share storage when their lifetimes cannot overlap.

Potential users include:

- directory buffers;
- file-loader FCBs;
- module-transfer buffers;
- CONFIG transfer buffers;
- reconstruction scratch space; and
- BDOS call scratch space.

This is a static lifetime optimization, not persistent allocation.

Conceptually:



Directory operation	Reconstruction operation	CONFIG operation
------------------------	-----------------------------	---------------------

only one live at a time

Before two facilities can share the same memory, their entry, exit, and exclusion conditions must establish that simultaneous use cannot occur.

A workspace does not belong in the PDS merely because it resides in protected RAM.

Likewise, a stack does not become persistent data merely because its storage must remain reserved for a resident subsystem.

This distinction prevents the PDS concept from expanding until it becomes indistinguishable from all writable high memory.

11.14 BetterCP/M 1.0 Persistence Boundary

The BetterCP/M 1.0 persistence architecture is deliberately narrow:

```
PDS
|
+-- HISTORY v1
```

and nothing else.

The system does **not** provide in 1.0:

- a general PDS allocator;
- configurable PDS size;
- owner allocation tables;
- dynamic PDS growth;
- PDS migration;
- runtime PDS release;
- named-directory state;
- PATH state;
- **HISTORY.RSX**; or
- a generalized persistent-extension store.

Some of these facilities can be investigated for later BetterCP/M releases.

A future owner-scoped PDS allocator could define allocation metadata, migration, release policy, configurable cold-boot sizing, and controlled expansion through the resident reconstruction machinery.

Such a design would begin from demonstrated consumers rather than from the assumption that every writable resident object should be allocated through one central mechanism.

No allocator format, allocation identifier, granularity, or maximum size is promised by BetterCP/M 1.0.

The 1.0 architecture therefore establishes a useful principle for future development:

Persistence is granted because a specific state object requires a defined persistent lifetime, not because a general persistent-memory facility happens to exist.

For BetterCP/M 1.0, command history is that object, and 192 bytes are sufficient to satisfy its requirement while preserving the accepted 53 KiB transient-program capacity.

12. Extension Architecture

BetterCP/M begins with a CP/M 2.2-compatible operating environment but is designed to support additional facilities without requiring every new capability to become part of the fixed operating-system core.

BetterCP/M 1.0 provides two principal forms of loadable resident extension: **Command Processor Extensions (CPXs)** and **Resident System Extensions (RSXs)**. CPXs extend the command environment. RSXs extend or mediate the resident system-service environment. The distinction reflects ownership and lifetime rather than merely module format.

Extensions are optional additions to a defined base system. They must preserve CP/M-compatible behavior where they coexist with the ordinary CP/M environment, operate through explicit architectural boundaries, and justify their use of memory and other limited system resources.

The extension architecture is deliberately bounded. BetterCP/M does not attempt to provide a generalized dynamic-module framework, dependency manager, package system, or modern driver architecture. The mechanisms implemented for 1.0 exist because specific BetterCP/M facilities require them.

12.1 Extension Model

An extension adds functionality not required by the basic CP/M 2.2-compatible environment.

Extensions can provide such things as:

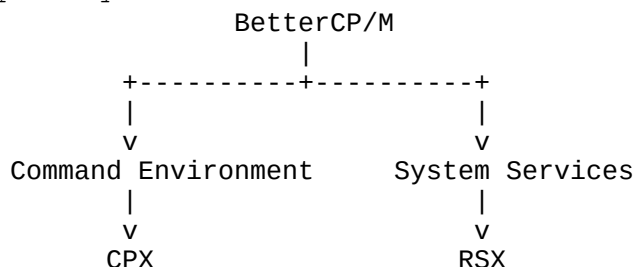
- additional command functions;
- additional or intercepted system services;
- resident callable services;
- compatibility interfaces;
- optional hardware or storage facilities; and
- other BetterCP/M-specific capabilities.

The architecture distinguishes extensions according to the system responsibility they extend.

A command-oriented extension belongs to the Command Environment and is implemented as a CPX.

A service-oriented extension that must remain available while transient applications execute belongs to the resident system environment and is implemented as an RSX.

Conceptually:



A module does not become an RSX merely because it is loaded into memory, nor does a command facility become a CPX merely because it can be invoked from the prompt.

The module type follows architectural responsibility.

12.2 Extension Principles

BetterCP/M extensions follow the same design principles as the rest of the operating system.

Compatibility. An extension operating within the normal CP/M environment must not silently change behavior required by compatible software.

Ownership. New functionality belongs with the subsystem whose responsibilities naturally include it.

Minimal mechanism. Extension interfaces should be no more general than their actual requirements justify.

Resource awareness. Resident memory, reclaimable memory, code size, execution cost, and complexity remain relevant.

Defined state. Extension state has an explicit owner and lifetime.

Explicit interfaces. Extensions interact with the rest of BetterCP/M through defined contracts rather than undocumented knowledge of private structures.

Recoverability. Invalid or failed extension configuration must not knowingly be published as a partially constructed system environment.

The existence of an extension architecture does not imply that every BetterCP/M component is dynamically replaceable.

12.3 CPXs and RSXs

CPXs and RSXs occupy different parts of the BetterCP/M memory and execution architecture.

The intended high-to-low organization is conceptually:

```
+-----+
| BIOS / hardware-dependent state |
+-----+
| Core BDOS / System Services     |
+-----+
| PDS and installed RSXs          |
+-----+
| CP/M compatibility gateway      |
+-----+
| Installed CPXs                  |
+-----+
| Reloadable CCP                  |
+-----+
| Transient Program Area          |
+-----+
```

An installed **RSX** is protected while transient programs execute. It therefore consumes resident memory and can lower the TPA ceiling.

An installed **CPX** belongs to the reclaimable command environment. It can be destroyed together with the CCP when a transient program executes and reconstructed afterward. Consequently, a CPX does not impose the same permanent TPA cost as an RSX.

This distinction is central to the extension architecture.

It permits frequently used command facilities to appear resident without requiring all of their code to remain protected during application execution.

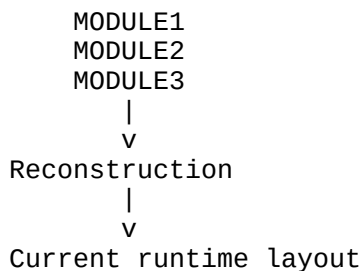
12.4 Extension Profiles

BetterCP/M represents the configured CPX and RSX environments through explicit ordered profiles.

A profile describes **which modules should be present and in what order**. It does not assign permanent runtime addresses to those modules.

Conceptually:

Configured profile



The runtime addresses are consequences of the current memory layout and can change when that layout is reconstructed.

CPX and RSX profiles remain distinct because the two module classes have different lifetimes and reconstruction requirements.

Profiles can also be saved as part of the cold-boot configuration so that BetterCP/M reconstructs the desired extension environment when a new system instance is initialized.

BetterCP/M 1.0 preserves explicit profile order. It does not infer order from a general dependency graph.

12.5 Extension Loading and Validation

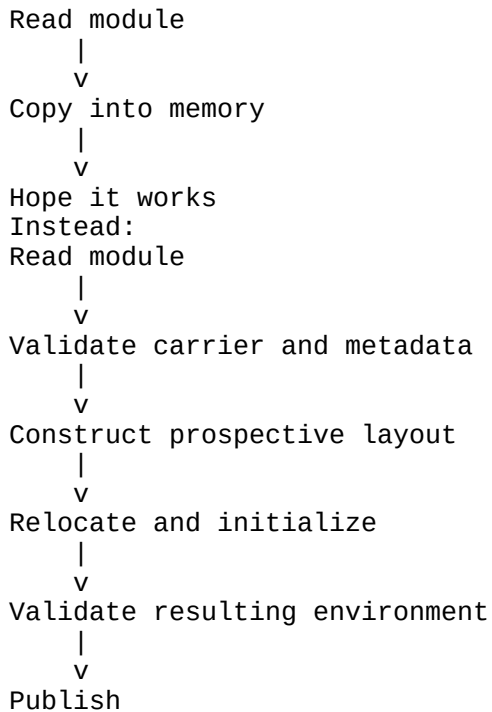
An extension is validated before it becomes part of the active environment.

Validation can include such matters as:

- module format and version;
- image integrity;
- memory requirements;
- metadata structure;
- relocation information;
- service advertisements;
- reconstruction class;
- interface compatibility; and
- resource limits.

A module that cannot satisfy the applicable contract is rejected before publication.

Loading is therefore not simply:



Until publication occurs, the old active environment remains authoritative where the operation permits it to be retained.

This prevents applications or system components from observing a partially installed extension.

12.6 Extension Ordering

Ordering has different consequences for CPXs and RSXs.

CPXs form a forward command-dispatch chain. After the core CCP has declined a command, installed CPXs are offered the command in configured order. The first CPX to handle it terminates command dispatch.

RSXs form an ordered resident chain through which applicable system-service requests can pass.

Order is therefore semantically meaningful.

BetterCP/M 1.0 records order explicitly rather than defining general module metadata such as:

REQUIRES
BEFORE
AFTER
CONFLICTS

Such facilities would require a broader relationship and dependency model than the 1.0 system needs.

Where order matters, the configured profile supplies it directly.

12.7 Resource Accounting

Extensions consume different classes of memory according to their role.

An RSX occupies protected resident memory. Its installation can therefore lower the application TPA.

A CPX occupies reclaimable command-environment memory and is absent from the application's usable memory calculation once the command environment has been reclaimed.

This distinction means that extension cost cannot be represented by a single "resident module size."

For example:

RSX

- > protected during application execution
- > reduces TPA

CPX

- > reclaimed during application execution
- > does not reduce application TPA

Extension management must also account for metadata, state, reconstruction information, and other memory required by the applicable subsystem.

BetterCP/M does not permanently reserve the maximum conceivable RSX or CPX space. Memory is charged according to the modules actually configured.

Resource use remains visible because optional functionality should not silently erode one of CP/M's most important resources: application memory.

12.8 Transactional Configuration

Changes to the protected resident layout are treated as reconstruction transactions.

This is necessary because inserting or removing an RSX can move other modules.

Suppose the active chain is:

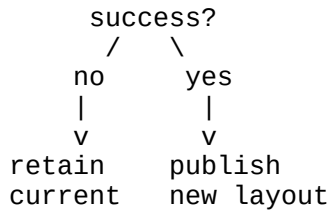
HELLO.RSX
FDF.RSX
OTHER.RSX

If HELLO.RSX is removed, the modules below it can move to new addresses.

BetterCP/M therefore does not merely delete the selected image and leave the remaining resident environment untouched. It constructs the prospective new layout, validates and reconstructs the affected modules, and publishes the new environment only when the transaction can be completed coherently.

Conceptually:

```
Current environment
|
v
Requested change
|
v
Construct prospective layout
|
v
Validate / relocate / restore
|
```



Publication of the new resident head and generation occurs only after the prospective environment is valid.

This same transaction provides the basis for keeping resident-service discovery consistent with the active RSX set.

12.9 Failure and Rollback

A failed extension operation must not knowingly expose an invalid partial configuration.

Failures can result from such conditions as:

- malformed modules;
- unsupported carrier versions;
- insufficient memory;
- invalid relocation information;
- unsupported reconstruction classes;
- duplicate resident services;
- incompatible metadata;
- failed initialization; or
- resource-capacity limits.

Where failure occurs before publication, the existing environment remains authoritative.

No half-installed provider is entered into the resident-service environment, and no stale callable address is deliberately left published after its provider has been removed or moved.

Where the former environment cannot simply be retained, BetterCP/M follows the defined resident recovery path rather than continuing with a configuration known to be inconsistent.

Extension configuration therefore follows the same broader BetterCP/M principle used for disk and system configuration:

Validate prospective state before making it authoritative.

12.10 Extension Discovery and Identification

Different forms of extension require different kinds of discovery.

CPXs are primarily command-environment modules. Their available commands can be enumerated through the CPX interface.

RSXs can advertise callable resident services through the resident-service mechanism described in Chapter 15.

A callable service has an identity independent of the filename or current address of the provider supplying it.

BetterCP/M does not require applications to infer capability merely from the BetterCP/M version number where an explicit service can instead be discovered.

Likewise, a module's runtime address is not its identity.

The distinction is:

Module identity	-> what provider is installed
Service identity	-> what capability is available
Runtime address	-> where it happens to reside now

These concepts must not be conflated in a reconstructible resident environment.

12.11 Extension State

Extension state normally remains with the extension that owns it.

A CPX cannot assume that its private memory survives transient execution because the command environment is reclaimable.

State that must survive command reconstruction therefore requires an owner whose lifetime actually spans that transition.

The BetterCP/M 1.0 command-history facility, for example, uses the PDS rather than CPX-private storage.

An RSX remains protected while installed and can therefore own live resident state. When reconstruction can move that RSX, however, the module must declare how its state survives movement.

BetterCP/M distinguishes several RSX reconstruction classes:

STATELESS
STATEFUL
STATE_PRESERVING
COLD_ONLY

These determine whether a module can simply be reloaded, can have its complete live image moved under defined pointer rules, requires an explicit save/restore contract, or cannot safely move while active.

The detailed reconstruction contracts are described in Chapter 14.

12.12 Extension Compatibility

Extensions intended to coexist with the ordinary CP/M environment must preserve that environment.

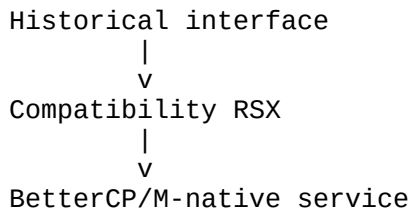
An RSX can inspect or intercept BDOS calls, but it cannot silently redefine an established CP/M operation merely because it lies in the service path.

A CPX can add commands, but core compatibility commands retain precedence over CPX dispatch.

BetterCP/M-specific system services use explicitly allocated interfaces rather than opportunistically claiming existing CP/M functions.

Historical compatibility frontends can also be implemented as extensions. P2DOS.RSX, for example, provides the historical P2DOS clock-call interface while translating those calls into BetterCP/M's native TIME service.

This arrangement separates:



The native architecture therefore does not have to be shaped around every historical compatibility interface it supports.

12.13 Extension Recovery

Because extensions can participate in startup configuration, an invalid extension profile must not permanently prevent access to the system.

Cold-boot and reconstruction mechanisms therefore preserve a defined recovery path in which optional extension configuration can be bypassed or reduced sufficiently to restore a usable command environment.

An extension profile is configuration, not an unconditional instruction to execute arbitrary modules regardless of validity.

Modules are validated when the profile is constructed.

A failure is reported rather than knowingly publishing an invalid chain.

This principle is particularly important for RSXs because a malformed protected resident layout could otherwise prevent ordinary applications or even the command environment from operating correctly.

Recovery behavior allows optional extensions to remain optional in practice as well as in name.

12.14 BetterCP/M 1.0 Extension Boundary

BetterCP/M 1.0 implements extension mechanisms required by concrete system facilities.

It does not provide a generalized package manager, dynamic linker, module dependency solver, or arbitrary plugin framework.

In particular, the 1.0 architecture does not define general module-carried:

- dependencies;
- conflicts;
- relative-order constraints;
- relationship graphs; or
- automatic dependency resolution.

Nor does extension architecture imply that every operating-system subsystem can be replaced dynamically.

The 1.0 mechanisms are deliberately narrower:

CPX
-> command-environment extension

RSX

-> resident system/service extension

Resident-service registry

-> discovery of callable resident services

Reconstruction transaction

-> controlled resident-layout changes

These facilities solve requirements that BetterCP/M actually has while preserving room for later evolution.

Future releases can broaden the extension architecture when concrete uses justify doing so. Such expansion should begin from measured requirements rather than from a desire to make BetterCP/M resemble a much larger modern operating system.

The objective remains the same as elsewhere in BetterCP/M: provide enough architecture to solve the real problem clearly, while keeping the resulting Z80 system small enough to understand.

13. Command Processor Extensions

Command Processor Extensions, or **CPXs**, extend the BetterCP/M command environment without becoming part of the protected resident system-service environment.

A CPX can provide additional commands, aliases, or other command-processing behavior. To the user, commands supplied by a CPX appear resident: they can be entered directly at the command prompt without loading an ordinary transient **.COM** program. Architecturally, however, CPXs belong to the reclaimable command environment.

This distinction allows BetterCP/M to provide a substantially richer command environment without permanently reducing the memory available to transient applications. When an application is executed, the CCP and CPXs can be overwritten. When control returns to BetterCP/M, the configured CPX environment is reconstructed along with the CCP.

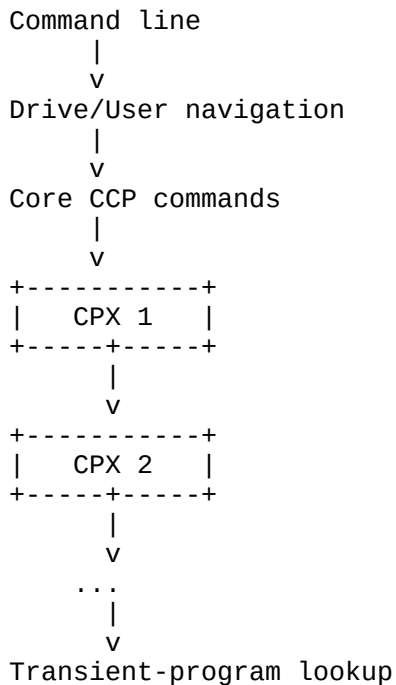
CPXs are therefore fundamentally different from RSXs. A CPX extends **command processing**. An RSX extends or participates in the **resident system-service environment**.

13.1 CPX Architecture

The BetterCP/M command environment consists of a small core CCP supplemented by zero or more CPXs.

The CCP handles drive/user navigation and the small set of commands that must remain part of the core command processor. Commands not handled there are offered to the installed CPX chain.

Conceptually:



A CPX either accepts a command or declines it.

If a CPX accepts the command, it performs the requested operation and terminates command dispatch.

If it declines the command, the next CPX is given the opportunity to handle it.

If every installed CPX declines, BetterCP/M proceeds to ordinary transient-program lookup.

The CPX mechanism is therefore a command-dispatch chain rather than a generalized shell plugin system.

13.2 BCPX Modules

BetterCP/M CPXs are distributed in the **BCPX** module format.

The BCPX carrier contains the information required for BetterCP/M to recognize, validate, load, relocate, initialize, and dispatch the module.

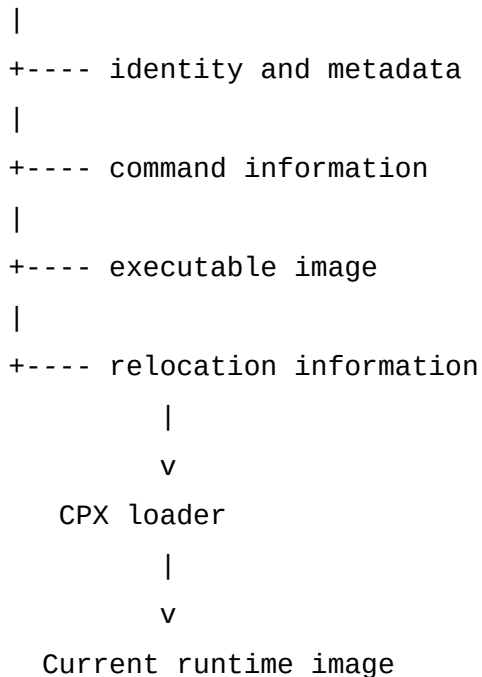
A BCPX module is not an ordinary **.COM** program. It is loaded as part of the command environment and must satisfy the CPX module contract.

The module format separates information that identifies and describes the CPX from the runtime address at which a particular reconstruction places it.

This is important because CPXs are movable.

Conceptually:

BCPX module



BetterCP/M 1.0 uses BCPX version 1 for the CPX module contract.

The precise binary structure of BCPX files belongs in the Programmer's Guide or ABI reference. Architecturally, the important point is that BCPX provides a validated, relocatable representation from which the command environment can be reconstructed.

13.3 Command Enumeration

A CPX identifies the commands it supplies.

This permits BetterCP/M and its management utilities to determine the command inventory of the active CPX environment without requiring commands to be discovered by trial execution.

The default BetterCP/M command package, RCP.CPX, supplies:

DIR	ERA	TYPE	REN	USER
CLS	VER	COPY	MOVE	

Command enumeration describes the interface offered by the CPX. It does not change the normal dispatch order.

A command supplied by a CPX remains subordinate to drive/user navigation and core CCP commands.

This prevents a CPX from replacing a fundamental core operation simply by advertising the same command name.

Enumeration also permits tools to report where resident commands originate and allows the configured command environment to be inspected without relying upon undocumented knowledge of individual CPX modules.

13.4 Command Dispatch and Chaining

The CPX dispatch contract is deliberately small.

A CPX receives the normalized command line and its length. The command line supplied to the CPX is uppercase.

The CPX then returns one of two logical results:

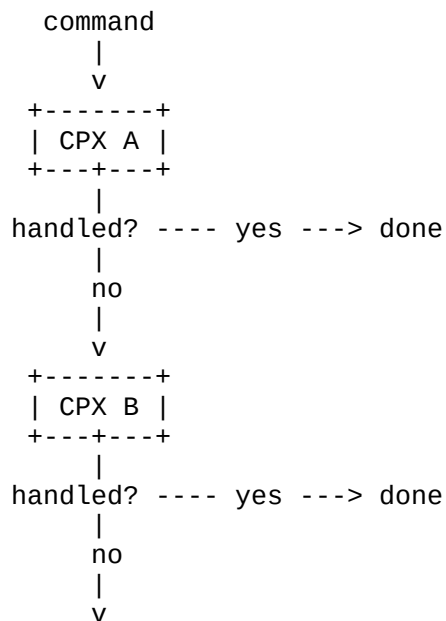
handled
declined

In the CPX calling convention, carry indicates that distinction.

A CPX that handles the command returns with the command considered complete.

A CPX that declines permits dispatch to continue.

Conceptually:



transient lookup

Carry has this meaning because the CPX ABI defines it. BetterCP/M does not assign a universal system-wide meaning to the carry flag.

Likewise, register preservation and clobbering follow the CPX interface rather than an assumed global convention.

The dispatch mechanism remains intentionally narrow. More elaborate functionality can be implemented within CPXs where required without enlarging the core CCP dispatch contract.

13.5 Initialization and Shutdown

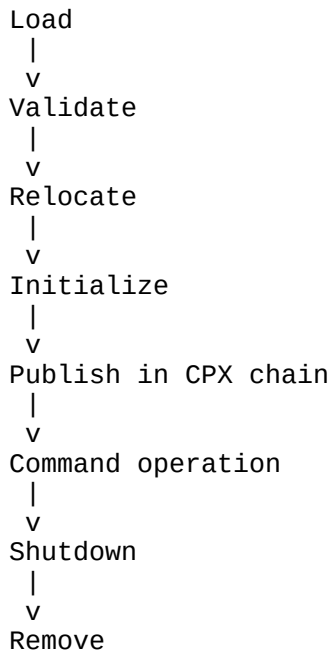
A CPX has a lifecycle extending beyond individual command invocations.

When BetterCP/M constructs a CPX environment, the module is loaded, relocated, validated, and initialized before becoming part of the active command chain.

Initialization permits the CPX to establish any command-environment state required for operation.

Correspondingly, removal can invoke the shutdown behavior defined by the CPX contract before the module ceases to be active.

The lifecycle is conceptually:



Initialization and shutdown do not give a CPX ownership of general system state.

A CPX that manipulates files, devices, configuration, or other operating-system facilities uses the corresponding BetterCP/M services rather than creating a private competing implementation.

13.6 CPX Profiles

The active CPX environment is represented by an ordered profile.

The profile identifies which CPXs are configured and their order in the dispatch chain.

For example:

RCP.CPX
TOOLS.CPX
DEBUG.CPX

describes a different command-dispatch environment from:

DEBUG.CPX
RCP.CPX
TOOLS.CPX

if more than one module can recognize the same command.

The profile records logical configuration rather than current runtime placement.

It therefore contains the information needed to reconstruct:

which CPXs
+
in what order

not:

which CPXs
+
at which permanent addresses

CPX runtime addresses can change each time the command environment is reconstructed.

Explicit profile order is therefore part of the configuration and must be preserved.

BetterCP/M 1.0 does not derive CPX order from a general dependency or priority graph.

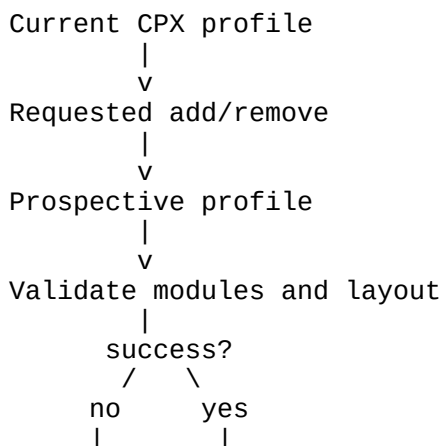
13.7 Loading and Unloading CPXs

CPXs can be added to or removed from the active command environment.

A requested change produces a new CPX profile and a corresponding reconstructed command environment.

Before publication, BetterCP/M validates the modules and verifies that the proposed environment can be constructed.

Conceptually:



v	v
retain	construct
current	new chain
profile	

A malformed or unsupported CPX is rejected rather than knowingly being inserted into the live dispatch chain.

The exact management interface used to request these operations is separate from the module's own command-dispatch ABI.

BetterCP/M's ABI conventions deliberately distinguish management operations from the runtime interface of the component being managed.

13.8 Warm-Boot Reconstruction

The most important property of a CPX is that it need not survive transient-program execution physically.

When BetterCP/M invokes a transient application, the CCP and CPXs can be overwritten as part of making the maximum practical memory available to the application.

The active CPX profile survives separately.

When the application terminates and BetterCP/M performs warm-boot reentry, the command environment is reconstructed.

Conceptually:

```

    Command environment

    CPX A
    CPX B
    CCP
    |
    | transient execution
    v
memory overwritten
    |
    | warm boot
    v
read active CPX profile
    |
    v
reload / relocate CPXs
    |
    v
restore CCP

```

|
v

command prompt

This is why a CPX can appear resident to the user without imposing the same TPA cost as an RSX.

Its **configuration** is persistent across the transition; its **current executable image** need not be.

13.9 CPX State

The reclaimable nature of CPXs places an important restriction upon CPX-owned state.

A CPX must not be the sole owner of indispensable state that must survive transient execution.

Private state needed only while the current CPX image exists can remain with the CPX.

State whose required lifetime extends beyond destruction of that image must belong elsewhere under an explicit ownership contract.

Command history demonstrates the distinction.

It would be natural from a user-interface perspective to regard history as part of the command environment. But history must survive when the CCP and CPXs are overwritten. BetterCP/M 1.0 therefore places the history object in the Persistent Data Segment rather than in a CPX.

Thus:

CPX-private temporary state

-> may disappear with CPX

Command history

-> PDS owner

Global operating-system state

-> owning system subsystem

Resident-service state

-> owning RSX or fixed subsystem

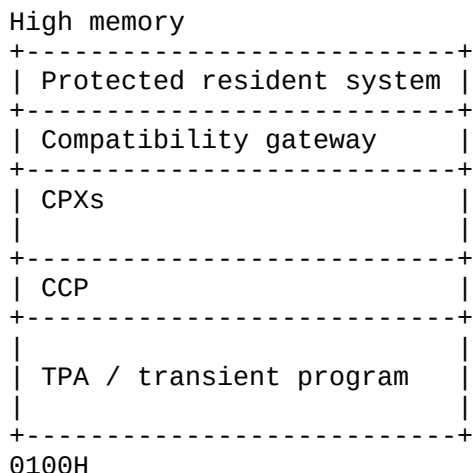
A CPX can use or modify state owned elsewhere through the appropriate interface, but it does not acquire ownership merely because one of its commands exposes that state to the user.

13.10 CPX Memory Use

CPXs occupy memory while the Command Environment is active.

They are placed above the reloadable CCP and below the protected resident system.

Conceptually:



When a transient program is executed, reclaimable CPX and CCP memory becomes available to the application environment.

This means CPX size affects how much memory is required to construct the interactive command environment, but does not reduce the ordinary transient TPA in the same manner as protected resident extensions.

BetterCP/M therefore does not need to restrict useful command functionality as severely as it would if every resident command permanently occupied protected memory.

Nevertheless, CPX size still matters. The command environment must fit in the available reclaimable region, reconstruction has a cost, and unnecessary command-environment complexity remains contrary to BetterCP/M's design principles.

13.11 The Standard RCP.CPX

The default BetterCP/M resident command package is **RCP.CPX**.

It provides the frequently used commands:

```
DIR
ERA
TYPE
REN
USER
CLS
VER
COPY
MOVE
```

These commands are separated from the core CCP because none requires permanent presence while an arbitrary transient program executes.

The distinction is architectural rather than a judgment about how frequently a command is used.

For example, **DIR** can be one of the most frequently invoked commands on the system while still belonging naturally in **RCP.CPX**.

By contrast, **SAVE** remains in the CCP because implementing it as an ordinary transient program would destroy the memory image it is intended to save, and a

bare command environment must retain the complete resident memory-monitor sequence.

The standard RCP therefore provides convenience without turning every common command into permanently protected system code.

Some RCP commands also have transient counterparts. These can provide fallback operation when **RCP.CPX** is absent or extended functionality beyond the resident command.

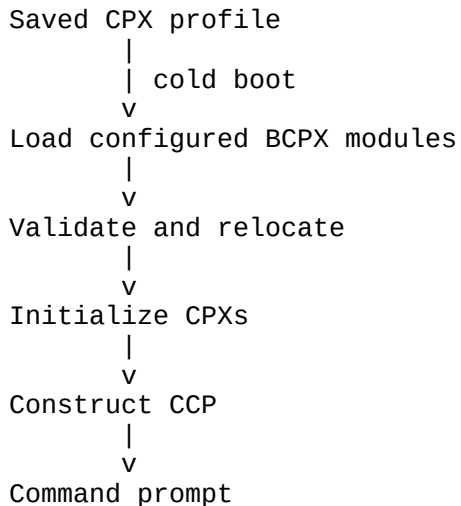
The resident/transient relationship is a user-interface decision and does not alter the CPX architecture itself.

13.12 CPX Configuration and Startup

The CPX profile can be saved as part of the BetterCP/M cold-boot configuration.

During cold initialization, BetterCP/M loads and validates the configured profile and constructs the initial command environment.

Thus:



The active profile can subsequently differ from the saved startup profile if CPXs are loaded or unloaded during the session.

A warm boot reconstructs the **active** profile. It does not silently restore the saved cold-boot profile.

This follows BetterCP/M's general distinction between active configuration and saved startup defaults.

CONFIG can manage the saved startup profile without becoming the owner of the active CPX environment.

13.13 CPX Interface Conventions

The CPX ABI follows BetterCP/M's general interface rules.

Persistent module identity is distinct from runtime address.

A normal BetterCP/M-specific callable interface restores the caller's stack and preserves **IX** and **IY** unless the CPX contract explicitly states otherwise.

Flags have only the meanings assigned by the CPX interface. In particular, the handled/declined meaning of carry belongs specifically to CPX dispatch.

Interfaces define their own:

- inputs;
- outputs;
- preserved registers;
- clobbered registers;
- flag meanings;
- state effects;
- error behavior;
- pointer lifetimes; and
- reentrancy rules.

A BetterCP/M release number is not itself a CPX ABI version.

Likewise, successful execution of a module does not by itself make an interface stable. A published stable ABI requires its binary format, calling conventions, lifecycle, version behavior, failure rules, and compatibility requirements to be defined.

The detailed BCPX programming interface belongs in the Programmer's Guide.

13.14 CPX Compatibility and Limits

CPXs extend the BetterCP/M command environment without changing the ordinary CP/M program-execution environment.

Core compatibility behavior retains precedence over CPXs.

An ordinary transient program does not need to know which CPXs are installed.

CPXs do not intercept BDOS merely because they provide commands that use operating-system services.

BetterCP/M 1.0 also deliberately does not define a general CPX dependency framework.

BCPX v1 contains no general declarative:

**REQUIRES
CONFLICTS
BEFORE
AFTER**

relationships.

If a later BetterCP/M release develops concrete requirements for such relationships, the module metadata and management architecture can be revised deliberately.

Similarly, named-directory navigation, NDR, PATH, and broader shell facilities are not implicit consequences of having a CPX mechanism. They remain separate later design work.

The BetterCP/M 1.0 CPX architecture has a narrower purpose:

Provide ordered, relocatable, reconstructible command extensions without permanently charging their code against the transient application environment.

That boundary gives BetterCP/M a richer command processor while retaining the compact execution model that makes CP/M useful on a 64 KiB Z80 system.

14. Resident System Extensions

Resident System Extensions, or **RSXs**, extend or mediate the BetterCP/M resident system-service environment. Unlike Command Processor Extensions, RSXs remain protected while transient programs execute and can therefore provide services continuously to applications and other resident components.

An RSX can intercept BDOS requests, provide BetterCP/M-specific behavior, implement compatibility interfaces, or publish callable resident services. Installed RSXs form an ordered chain between the CP/M compatibility gateway and the core system-service implementation.

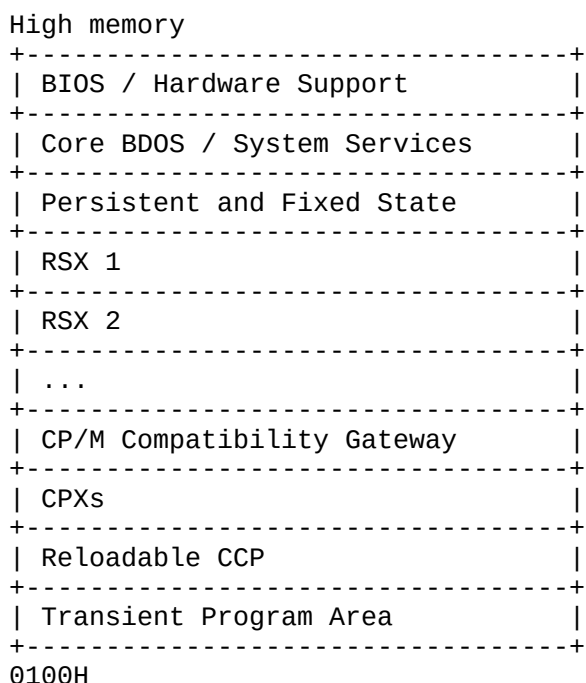
Because RSXs occupy protected memory, they have a direct cost in application space. Loading an RSX can lower the TPA ceiling; unloading one can recover protected memory. BetterCP/M therefore allocates resident space according to the modules actually installed rather than permanently reserving the maximum possible extension area.

RSXs are dynamically loadable and unloadable, but their position in memory is not permanent. Changing the protected resident configuration can require the remaining RSXs to move. BetterCP/M consequently defines reconstruction rules for preserving module state, repairing relocatable references, and publishing the resulting resident environment only after the prospective configuration has been validated.

14.1 RSX Architecture

The RSX environment lies within protected resident memory below the fixed system core and above the reclaimable command environment.

Conceptually:



The exact addresses depend upon the active resident configuration.

The compatibility gateway provides the stable application-facing route into the system-service environment. Behind it, requests can pass through the installed RSX chain before reaching core System Services.

RSXs therefore extend the service path without requiring applications to know the physical location of the core BDOS or of any particular extension.

An RSX can also publish a callable resident service. Such a service is discovered through the resident-service registry rather than by requiring a client to know the RSX's current base address.

The two mechanisms serve different purposes:

BDOS interception

-> participate in the normal system-service path

Callable resident service

-> provide an explicitly discoverable BetterCP/M service

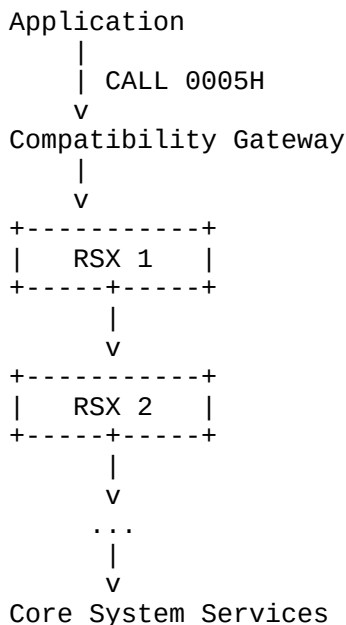
An RSX can use either or both where its function requires them.

14.2 The RSX Chain

Installed RSXs form an ordered chain.

A request entering through the CP/M compatibility gateway can be offered to the RSXs before reaching the core system-service implementation.

Conceptually:



An RSX can recognize a request that belongs to it, perform additional processing, translate the request, or allow normal service processing to continue according to its defined interface.

Chain order can therefore affect behavior and is part of the active RSX configuration.

BetterCP/M 1.0 preserves this order explicitly through the RSX profile. It does not derive ordering from a generalized dependency or priority system.

The chain must also avoid unintended recursion. An RSX that needs to request a service while processing a call must use the bypass or continuation mechanism defined for that interface rather than accidentally reentering itself indefinitely.

RSXs remain part of the protected environment while transient programs execute. They are therefore available independently of whether the current CCP and CPXs are present.

14.3 BRSX Modules

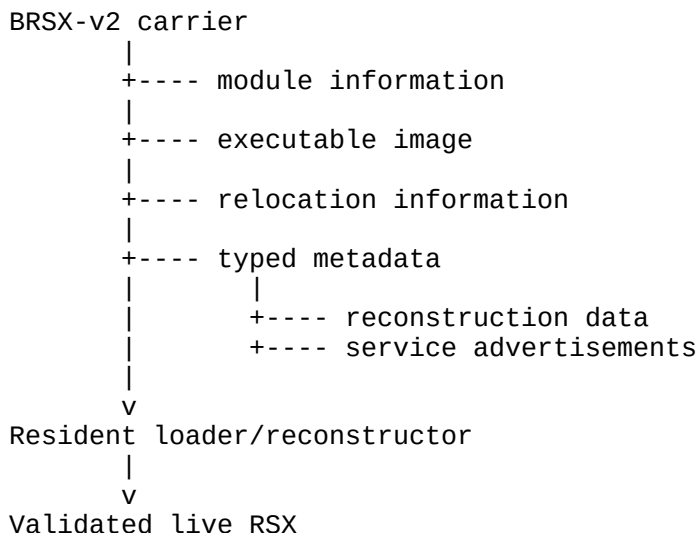
BetterCP/M RSXs are distributed in the **BRSX** module format.

Production BetterCP/M 1.0 uses **BRSX version 2**. BRSX version 1 records earlier format history but is not a supported BetterCP/M 1.0 runtime compatibility path.

A BRSX-v2 carrier contains the information needed to:

- identify and validate the module;
- determine its memory requirements;
- load its executable image;
- relocate linked addresses;
- identify its reconstruction class;
- describe reconstruction metadata;
- advertise callable services;
- initialize the module; and
- participate in controlled resident reconstruction.

Conceptually:



Entry points and other relocatable references in the carrier are expressed relative to the module rather than assuming a permanent runtime base.

Typed metadata permits the loader to distinguish the different kinds of information associated with the module. Unknown metadata types or unsupported versions are rejected rather than guessed at.

The detailed binary layout of BRSX belongs in the Programmer's Guide. The architectural requirement is that the carrier contain enough information for BetterCP/M to validate and reconstruct the module safely.

14.4 RSX Profiles

The active RSX environment is described by an ordered profile.

For example:

```
P2DOS.RSX
CLOCK.RSX
OTHER.RSX
```

identifies both which modules are active and their configured order.

The profile does not assign permanent addresses.

Instead:

```
RSX profile
  |
  v
Resident reconstruction
  |
  v
Current memory layout
```

The current location of a module is a consequence of reconstruction.

This distinction permits the same profile to be rebuilt at different addresses as resident memory requirements change.

An RSX profile can also be saved as part of the cold-boot configuration. The saved profile describes the environment BetterCP/M should construct during a future cold boot; the active profile describes the environment currently in use.

A warm boot does not silently replace the active profile with the saved one.

Changing the active RSX profile can require a reconstruction transaction because inserting or removing one module can move other modules already in the chain.

14.5 Reconstruction Classes

Every BRSX-v2 module declares how its state is to be treated if resident reconstruction requires the module to move.

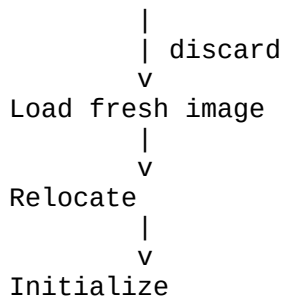
BetterCP/M defines four reconstruction classes:

```
STATELESS
STATEFUL
STATE_PRESERVING
COLD_ONLY
```

STATELESS modules contain no live private state that must survive reconstruction. BetterCP/M can load a fresh image at the new address and initialize it normally.

Conceptually:

```
Old STATELESS image
```



STATEFUL modules preserve their complete live allocation when moved. Runtime state moves with the module, and declared internal absolute pointers are repaired according to the reconstruction ABI.

STATE_PRESERVING modules require their state to be exported and restored through a versioned preservation contract rather than by copying the live allocation directly.

COLD_ONLY modules cannot safely move during the running session. A requested layout change requiring such a module to move is rejected or deferred until cold boot.

The reconstruction class is mandatory metadata. A loader that does not implement the declared class rejects the module before altering the resident layout.

This makes reconstructibility an explicit property of the RSX rather than an assumption made by the loader.

14.6 Stateful RSXs

A **STATEFUL** RSX retains its complete live allocation when resident reconstruction moves it.

This includes both executable material and mutable private state belonging to the module.

A stateful move is valid only when the module satisfies the stateful reconstruction contract.

In particular:

- all state that must survive movement lies within the declared live allocation;
- copying the allocation preserves all non-pointer values;
- runtime references to the module's own image use relocatable offsets or handles, or occupy declared pointer slots;
- the module contains no untracked runtime absolute pointer into itself;
- the module does not depend upon a live register value or active stack frame during movement;
- it contains no undocumented pointer into another movable RSX, CPX, CCP, or transient reconstruction workspace; and
- fixed external resources remain valid independently of the module's address.

Reconstruction occurs only at a quiescent resident-system boundary. BetterCP/M does not move an RSX while arbitrary transient or RSX code is executing.

A module that cannot satisfy these restrictions cannot safely declare itself **STATEFUL**. It must instead use **STATE_PRESERVING** or **COLD_ONLY**.

This distinction is important because ordinary relocation information is not sufficient to preserve arbitrary live state.

14.7 Relocation and Pointer Repair

BetterCP/M distinguishes between **linked-image relocation** and **runtime pointer repair**.

Ordinary BRSX relocation records identify addresses derived from the linked module image. They allow a freshly loaded module to be placed at a runtime address different from its link origin.

Runtime execution can create additional absolute addresses that did not exist in the original image.

For example, an RSX can create a pointer to one of its own mutable buffers:

```
Old module base
|
+---- data buffer
|
+---- runtime pointer ----+
                        |
                        +--> data buffer
```

If the complete live image is moved, copying the bytes preserves the old pointer value. The pointer must therefore be repaired.

For a stateful move, BetterCP/M computes:

delta = new_base - old_base

and examines both:

- ordinary static relocation slots; and
- declared runtime absolute-pointer slots.

A current word is adjusted by **delta** only when its value points inside the old module allocation:

old_base <= value < old_base + allocation_size

Values outside that range remain unchanged.

This preserves:

- null values;
- sentinels;
- fixed-memory addresses; and
- external addresses that should not move with the RSX.

The reconstructor then rewrites loader-owned runtime words and rebuilds the RSX chain.

Movement between overlapping source and destination ranges has normal **memmove** semantics.

The central rule is:

Relocation records describe addresses derived from the linked image; they do not automatically describe every absolute pointer created at runtime.

A stateful RSX must therefore make its runtime internal pointers reconstructible explicitly.

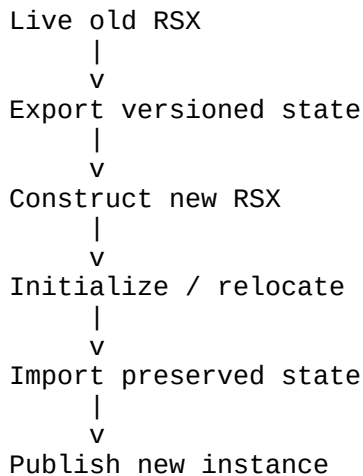
14.8 State Preservation

Not every RSX can safely preserve its state by moving its complete live image.

A module can maintain state whose representation should not be copied directly, can depend upon external objects requiring rebinding, or can contain internal relationships that are better reconstructed through a defined serialization contract.

Such a module belongs to the **STATE_PRESERVING** class.

The intended model is:



The state representation must itself be versioned so that the loader does not assume that two implementation versions have identical private layouts.

BetterCP/M 1.0 reserves this reconstruction class in the architecture, but the complete export/import ABI is deferred until a concrete module requires it.

An implementation that does not support the class therefore rejects a **STATE_PRESERVING** module rather than treating it as stateless or copying it as though it were stateful.

Ordinary RSX-private state is discarded when the RSX is explicitly unloaded. State that must survive removal or replacement of the module requires a separately defined persistence contract.

14.9 Loading and Unloading RSXs

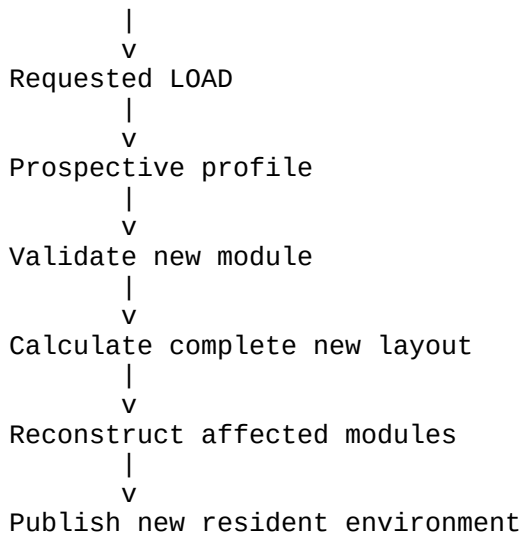
RSXs can be loaded and unloaded during a BetterCP/M session.

Because an RSX occupies protected memory, this operation can affect the memory layout of every resident component below it and can alter the TPA ceiling.

Loading therefore involves more than copying one module into an unused hole.

Conceptually:

Current RSX profile



Unloading follows the same architectural model.

If the chain is:

```

RSX A
RSX B
RSX C

```

and RSX A is removed, RSX B and RSX C can move. Their reconstruction contracts therefore participate in the unload operation even though those modules themselves are not being removed.

An explicit unload invokes the module's defined shutdown behavior where applicable and discards its private state.

The remaining modules are reconstructed according to their own declared classes.

After the operation, the command environment can also require reconstruction because changing the protected resident boundary changes the space available below the compatibility gateway.

14.10 Transactional Reconstruction

Resident-layout changes are performed as transactions.

The transaction has two broad phases:

```

PREPARATION
  |
  +-- read required carriers
  +-- validate metadata
  +-- classify modules
  +-- calculate new layout
  +-- verify memory
  +-- preserve required live state
  +-- complete required disk I/O
  |
  v
COMMIT
  |
  +-- reconstruct resident memory
  +-- repair pointers
  +-- rebuild chain

```

```

    +-- establish services
    |
    v
PUBLISH
    |
    +-- new RSX head
    +-- new boundaries
    +-- new service generation
    +-- new gateway state

```

Potentially failing work is performed during preparation wherever possible.

Only after preparation succeeds does BetterCP/M alter the protected resident environment.

Externally visible descriptors are published last.

This ordering is essential because clients must never be told that a new RSX environment exists while part of that environment still describes the old layout.

The resident-service registry follows the same transaction. It is derived from the validated service advertisements in the prospective RSX set and becomes visible with the new resident generation.

A validation or preparation failure leaves the active environment unchanged.

14.11 Failure and Rollback

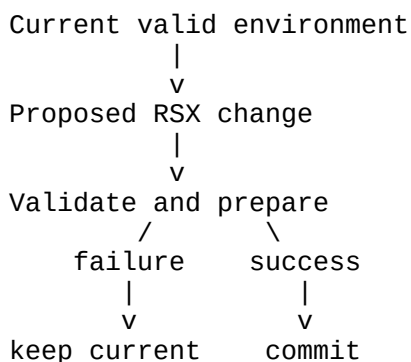
Resident reconstruction can fail for reasons including:

- malformed BRSX carriers;
- unsupported carrier or metadata versions;
- unsupported reconstruction classes;
- insufficient protected memory;
- invalid relocation information;
- invalid runtime-pointer metadata;
- duplicate service providers;
- service-capacity limits;
- missing module images;
- impossible memory layouts; or
- failed module initialization.

Such failures are detected before commit wherever possible.

A failed prospective configuration is not partially published.

Thus:



environment new layout

The loader validates the complete prospective memory map and all required module metadata before committing the new resident layout.

Unknown reconstruction classes, malformed metadata, or unsupported module formats are errors. BetterCP/M does not silently reinterpret them as **STATELESS**.

If a failure occurs during the short commit phase, BetterCP/M follows the defined resident recovery path rather than reporting success with a partially linked chain.

The resident-service environment obeys the same rule. A provider does not become discoverable until its module is valid, reconstructed, and published. When a provider is removed or moved, stale callable addresses are not retained as registry state.

This transactional model gives BetterCP/M dynamically configurable resident services without making the integrity of the running system depend upon the success of a sequence of partially visible memory edits.

15. Resident Services

BetterCP/M permits RSXs and, where useful, fixed resident components to publish **callable resident services**. These services provide facilities that need to remain available while transient programs execute but do not naturally belong in the standard CP/M BDOS interface.

A resident service has a stable logical identity independent of the filename, implementation version, or current address of the module that provides it. Clients discover a required service through the BetterCP/M resident-service gateway and then call the returned entry point according to that service's own ABI.

The resident-service facility defines **discovery and binding**. It does not define the operations performed by individual services. A clock service, for example, defines its own requests and results; the resident-service architecture merely provides a standard means to discover a compatible provider.

Because RSXs can move during resident reconstruction, callable addresses are deliberately temporary. The persistent identity of a service is its service identifier and ABI, not the address at which its current provider happens to reside.

15.1 Resident-Service Model

A resident service represents a callable capability supplied by the protected resident environment.

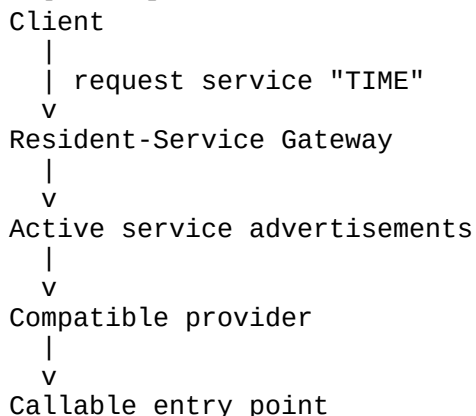
Examples include services such as:

TIME
HIST
NDR
SECU

where those services are present in a particular configuration.

A client does not locate the providing RSX and calculate an entry point from private knowledge of its format. Instead, it asks BetterCP/M for a service having the required identity and interface characteristics.

Conceptually:



After successful discovery, the client invokes the returned entry point using the ABI defined by that particular service.

The registry therefore separates two operations:

Discovery

-> Which compatible service is available,
and where is its current entry point?

Invocation

-> Perform an operation defined by that
service's own ABI.

BetterCP/M does not route every service invocation repeatedly through the registry. Once a client has obtained a valid callable address, it can call the service directly for the lifetime of that binding.

15.2 Service Identification

Every resident service has an exact four-byte uppercase ASCII identifier.

Shorter identifiers are space-padded.

Examples include:

TIME
HIST
NDR_
SECU

where the underscore above represents the padding space in NDR rather than a literal underscore character.

Service identity is independent of provider identity.

Thus, a client interested in clock services asks for:

TIME

rather than asking for a particular clock-provider filename.

This permits different RSXs or fixed resident implementations to provide the same logical service without requiring clients to know which implementation is installed.

Identifiers beginning with X are reserved for experimental or private use until BetterCP/M defines a stable third-party service-ID allocation policy.

A service ID identifies the kind of service. It does not by itself establish interface compatibility. ABI version and capabilities participate separately in service selection.

15.3 Service Advertisements

A provider advertises each callable service it supplies.

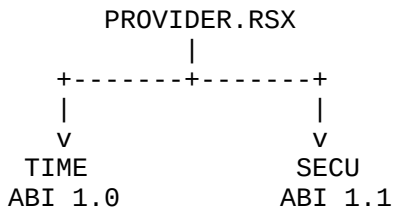
An advertisement identifies:

4 bytes Service ID
1 byte ABI major version
1 byte ABI minor version
2 bytes Callable entry offset
2 bytes Capability flags

The callable entry is expressed as an offset from the RSX base rather than as a permanent absolute address.

An RSX can advertise more than one service.

For example, conceptually:



BetterCP/M 1.0 permits only one active provider for a given service identifier, regardless of ABI major version.

A proposed configuration containing duplicate providers is rejected before publication.

This avoids requiring BetterCP/M 1.0 to define provider priorities, stacking rules, or selection among multiple implementations of the same service.

Parallel providers and more elaborate selection policies are deferred.

15.4 The Resident-Service Registry

The **resident-service registry** is the logical view of callable services available in the current resident environment.

It is not a permanent flat table of absolute service addresses.

Instead, validated service advertisements reside with their active providers. The resident-service gateway walks the validated provider information in the active RSX chain and derives the current callable address from:

RSX base + advertised entry offset

Conceptually:

Active RSX chain

```

|
+-- RSX A
|   +-- service advertisement
|
+-- RSX B
|   +-- service advertisement
|   +-- service advertisement
|
+-- RSX C
|   +-- service advertisement
|       |
|       v

```

Logical registry view

This organization has several useful consequences.

There is no second authoritative table of service addresses that can become stale when an RSX moves.

Discovery does not require disk I/O because validated advertisements are materialized within the live provider allocations.

An unloaded provider consumes no separate persistent registry allocation.

The BetterCP/M 1.0 Persistent Data Segment is not used for the resident-service registry.

Registry control and reconstruction state belong to the fixed extension subsystem.

15.5 Service Publication

An RSX does not register itself by modifying the registry directly.

Service publication is controlled by the resident loader and reconstructor.

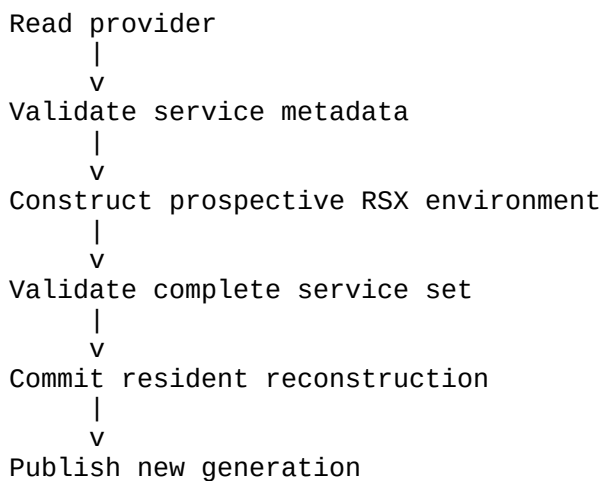
When a BRSX-v2 provider is loaded, BetterCP/M validates its service metadata before the provider becomes active.

Validation includes such matters as:

- service identifier;
- metadata version;
- ABI information;
- capability information;
- callable entry bounds;
- provider validity;
- duplicate service IDs; and
- registry capacity.

Only after the complete prospective resident environment has been validated and reconstructed are its services published.

Thus:



Transient programs and RSXs cannot directly insert or remove registry entries.

This prevents a service from becoming visible before its provider is fully usable.

15.6 Service Discovery

BetterCP/M private BDOS Function **182** is the resident-service gateway.

The caller places:

C = 182
DE = address of request block

The gateway provides three operations:

0 LOOKUP
1 ENUMERATE
2 DESCRIBE_REGISTRY

LOOKUP is the normal binding operation.

A client supplies:

- the requested service ID;
- required ABI major version;
- minimum acceptable ABI minor version; and
- required capability bits.

A provider satisfies the request when:

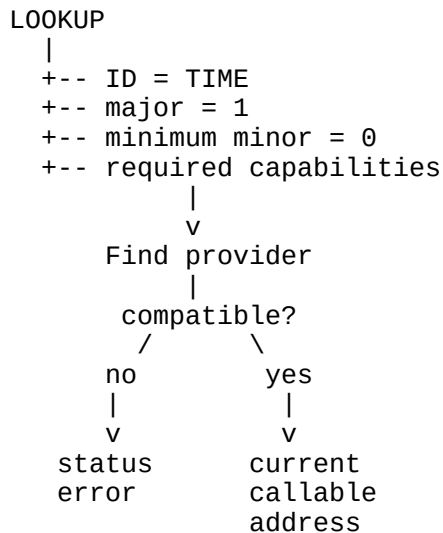
```

service ID matches
      AND
ABI major matches
      AND
provider minor >= required minor
      AND
all required capability bits are present

```

On successful lookup, BetterCP/M returns the current callable entry point and information describing the selected service.

Conceptually:



Discovery therefore tests **presence**, **ABI compatibility**, and **capability compatibility** separately.

A client need not infer any of them merely from the BetterCP/M release number.

15.7 Enumeration and Registry Description

ENUMERATE allows software to examine the services currently published by the resident environment.

The caller supplies a zero-based service ordinal. On success, BetterCP/M returns information describing that service and its current provider.

Enumeration order is not a persistent property.

Likewise, provider indices identify providers only within the current resident layout. They are not stable module identifiers and must not be saved as though they remain meaningful after reconstruction.

DESCRIBE_REGISTRY reports the current registry size and implementation capacities.

BetterCP/M 1.0 supports:

```

4   active RSXs

```

2 service advertisements per RSX
8 resident services total

These values are discoverable through the interface and are **not client constants**.

Software should therefore query the registry description when it actually requires capacity information rather than embedding the 1.0 implementation limits into its own ABI assumptions.

15.8 Service Versioning

Resident services use major and minor ABI versions where compatible evolution is expected.

The **major version** identifies an incompatible ABI generation.

The **minor version** identifies compatible extension within that major version.

A request therefore specifies:

required major
minimum acceptable minor

A provider with the wrong major does not satisfy the request.

A provider with the correct major but an older minor than the caller requires likewise does not satisfy it.

Capability flags permit optional functionality within an otherwise compatible service ABI to be tested independently.

This distinguishes several questions that should not be conflated:

Is the service present?

Does it implement the ABI major I understand?

Is its ABI minor new enough?

Does it provide the optional capabilities I require?

A BetterCP/M system version cannot answer these questions reliably because individual resident services can vary independently of the operating-system release.

15.9 Service Calling Conventions

The resident-service registry discovers a service but does not define that service's operation set.

Each service-specific ABI defines:

- operations;
- arguments;
- results;
- status values;
- register use;
- flag meanings;
- memory effects;
- pointer lifetimes;
- reentrancy;
- callback behavior; and
- bypass rules where applicable.

A normal BetterCP/M callable service returns with the caller's stack restored and preserves **IX** and **IY** unless its own ABI explicitly states otherwise.

No universal meaning is assigned to carry or other flags.

BetterCP/M 1.0 uses one dispatcher entry per logical resident service rather than requiring a separate callable address for every operation supplied by that service.

A complex service can therefore use a service-specific operation selector or versioned request block beneath its one advertised entry point.

This keeps service discovery independent of the number of operations a particular service implements.

15.10 Service Address Lifetime

A callable service address is a property of the current resident layout.

It is not the persistent identity of the service.

If reconstruction moves the provider:

Before reconstruction

```
TIME
 |
 v
Provider at E200H
 |
 v
entry at E240H
```

After reconstruction

```
TIME
 |
 v
Provider at E000H
 |
 v
entry at E040H
```

the logical service remains **TIME**, but the former callable address is no longer valid.

BetterCP/M associates resident discovery with the current **resident-layout generation**.

A returned callable address remains valid only while that generation remains unchanged.

Transient applications can normally retain a discovered address throughout their own execution because BetterCP/M does not reconstruct the resident environment underneath arbitrary transient code.

Components whose lifetime spans resident reconstruction must rediscover the service.

In particular, reconstructed CCPs and CPXs bind again rather than retaining raw addresses obtained before reconstruction.

The architectural rule is:

Service identity and ABI are persistent concepts; a callable address is a current-layout property.

15.11 Reconstruction and Rebinding

Resident reconstruction builds the complete prospective logical service environment from the advertisements belonging to the prospective RSX set.

If a provider moves, BetterCP/M does not patch a separate persistent registry address.

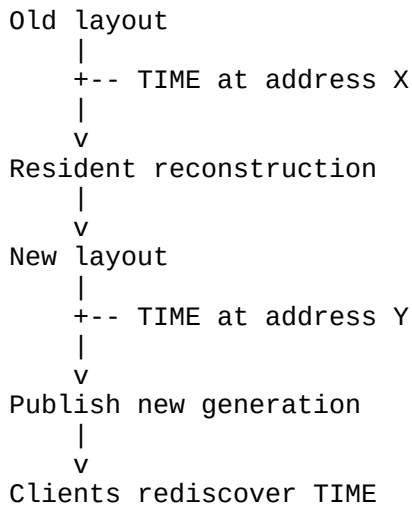
Instead, after reconstruction the callable address is again derived from:

new RSX base + advertised entry offset

The new RSX chain and resident generation are published only after the reconstructed environment is valid.

Clients surviving the transition subsequently rediscover the service.

Conceptually:



This arrangement avoids attempting to identify and patch every service pointer that might exist throughout the system.

Instead, BetterCP/M defines pointer lifetime explicitly and requires rebinding at the appropriate lifetime boundary.

15.12 Service Removal

When an RSX is unloaded, every service advertisement belonging to that provider disappears with it.

The reconstructed resident environment is published without those services.

No independent registry entry remains behind.

Thus:

Before unload

CLOCK.RSX
|
+-- TIME

After unload

CLOCK.RSX absent

|
+-- TIME absent

A later lookup reports that the service is unavailable unless another valid provider has been installed as part of the new configuration.

An old callable address obtained before the unload is invalid because the resident-layout generation has changed.

Explicit unload also discards the provider's ordinary private state according to its RSX lifecycle contract.

The service registry does not provide persistence for feature state.

15.13 Failure and Registry Integrity

The resident-service gateway never deliberately follows an unvalidated provider pointer.

Malformed service metadata is rejected during resident reconstruction before the proposed configuration is published.

A prospective configuration can fail because of:

- invalid service identifiers;
- unsupported metadata versions;
- invalid callable-entry offsets;
- duplicate providers;
- unsupported providers;
- service-capacity limits;
- incompatible module metadata; or
- another resident-reconstruction failure.

Where preparation fails, the old resident environment and its service set remain authoritative.

If registry corruption is detected in the running environment, BetterCP/M does not attempt to continue by trusting suspect addresses. It reconstructs from validated active metadata where possible or enters the defined resident recovery path.

Service discovery therefore participates in the same transactional publication rules as the RSX environment itself.

15.14 BetterCP/M 1.0 Service Boundary

The resident-service facility is intentionally small.

It provides:

- service identity
- ABI version matching
- capability matching
- lookup
- enumeration
- registry description
- current callable binding

It does not provide:

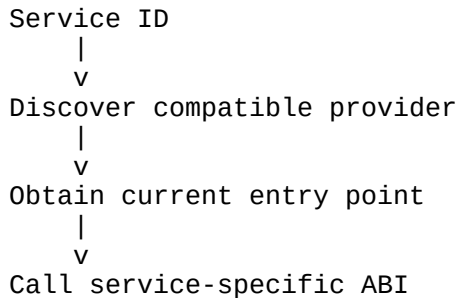
- a general dynamic linker;
- arbitrary application registration;
- multiple simultaneous providers for one service ID;

- provider priorities;
- service stacking;
- automatic dependency resolution;
- permanent callable addresses; or
- a general persistent-service database.

Those facilities are not required by BetterCP/M 1.0.

The registry exists to solve a specific architectural problem: movable resident providers need a stable way to advertise callable capabilities without making their current addresses permanent system interfaces.

This produces a simple relationship:



The architecture therefore preserves both extensibility and the reconstructible resident-memory model.

A BetterCP/M component depends upon **the service it needs**, not upon the filename or current address of the module that happens to provide it.

16. Date and Time Architecture

BetterCP/M provides date and time through a layered clock architecture that separates hardware access, the native BetterCP/M clock service, native consumers, and optional historical compatibility interfaces.

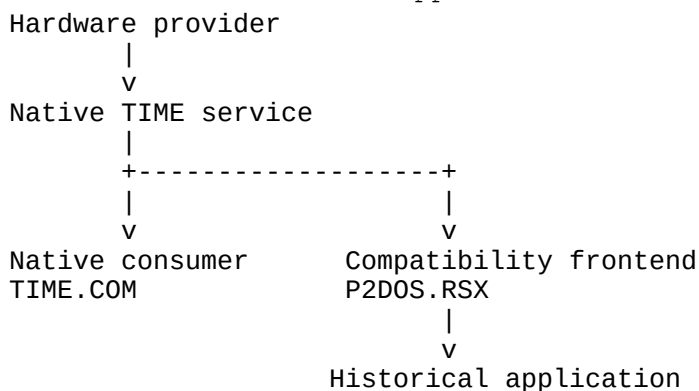
The central service is the callable resident **TIME** service. A clock provider implements the native TIME ABI and translates between its particular hardware representation and BetterCP/M's canonical clock record. Native programs such as **TIME.COM** discover and call this service directly.

Historical interfaces are implemented separately. For example, **P2DOS.RSX** provides the P2DOS Functions 200 and 201 interface while using the same native TIME service underneath. It does not contain another clock driver.

This separation allows new clock hardware to be supported without modifying every clock-aware application or compatibility frontend, and allows additional historical interfaces to be supported without duplicating the hardware implementation.

16.1 Clock Architecture

BetterCP/M divides clock support into four conceptual layers:



The **hardware provider** understands the physical or emulated clock.

The **TIME service** defines BetterCP/M's hardware-independent clock interface.

A **native consumer** uses the TIME service directly.

A **compatibility frontend** translates another operating-system interface into the native TIME service.

The resident-service registry provides discovery between these layers but is not itself another clock layer.

In the BetterCP/M 1.0 implementation, the hardware provider and TIME service can reside in the same BRSX-v2 module. They remain conceptually separate responsibilities even when one module implements both.

Two architectural rules follow:

Adding another clock API does not require another hardware driver.

and:

Adding another clock device does not require changing every clock frontend.

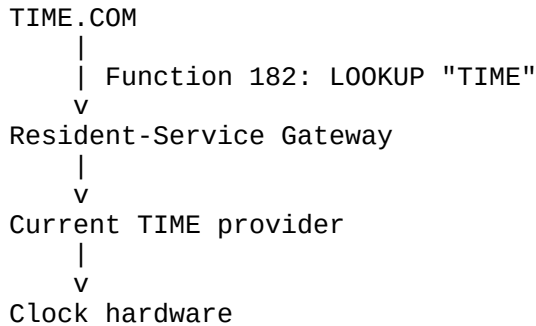
16.2 The TIME Service

A BetterCP/M clock provider advertises:
TIME

as resident service ABI 1.0.

Clients discover the current provider through the resident-service gateway at BDOS Function 182.

Conceptually:



The TIME service supports two operations:

```
0  GET
1  SET
```

GET obtains the current date and time.

SET requests that the provider update the clock.

Not every provider is writable. A read-only provider advertises no SET capability and returns the defined **SET_UNSUPPORTED** status if a SET operation is attempted.

The TIME service owns clock-service semantics. It does not own command syntax, human-readable formatting, or historical API conventions.

16.3 Canonical Clock Record

TIME ABI 1.0 represents date and time using a compact five-byte record:

Offset	Size	Meaning
0	2	Day count, little-endian
2	1	Hour, packed BCD
3	1	Minute, packed BCD
4	1	Second, packed BCD

The day count begins with:

Day 1 = 1 January 1978

and the 16-bit range extends through:

Day 65535 = 5 June 2157

Hours range from packed-BCD 00H through 23H, minutes and seconds from 00H through 59H.

This representation originated with P2DOS, but BetterCP/M adopts it as its native record because it is compact, historically established, adequate for the required range, and efficient for common real-time clocks.

Its use as the native representation does not make P2DOS the underlying BetterCP/M clock architecture. P2DOS compatibility is one useful consequence of choosing the same representation.

The record represents the machine's ordinary **local civil time**.

TIME ABI 1.0 does not define:

- time zones;
- daylight-saving policy;
- subsecond time;
- alarms;
- scheduling; or
- network-time synchronization.

These concepts are outside the 1.0 clock contract.

16.4 Reading the Clock

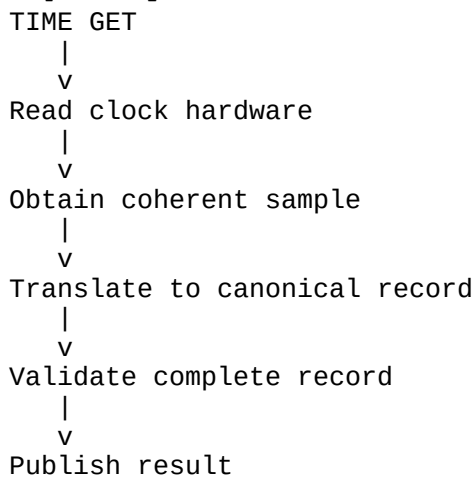
A GET request asks the provider for one coherent clock sample.

The provider must not construct a returned record from mutually inconsistent hardware values obtained across a clock rollover.

For example, reading separate hardware registers carelessly near midnight could produce a result combining the previous day's date with the following day's time.

The provider is responsible for whatever device-specific procedure is necessary to obtain a coherent observation.

Conceptually:



The caller receives the clock record only after the complete sample has been obtained and validated.

A hardware failure, invalid sample, or unavailable device produces a service failure rather than a plausible fabricated time.

This keeps hardware-specific coherence rules inside the provider rather than requiring every clock consumer to understand them.

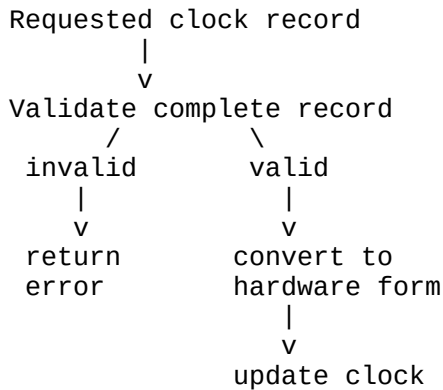
16.5 Setting the Clock

SET requests use the same canonical five-byte record.

Before modifying the hardware, the provider validates the complete requested record.

An invalid date or time must therefore fail before the clock is partially changed.

Conceptually:



A provider advertises capability bit **0001H** when SET is supported.

A read-only provider omits this capability and returns **SET_UNSUPPORTED**.

BetterCP/M 1.0 does not require every supported hardware clock to be writable. The required providers can therefore supply complete native date/time reading even where their device interface does not safely support clock setting.

The native service nevertheless defines SET so that writable providers can implement it without requiring a new clock ABI.

16.6 Hardware Clock Providers

A hardware provider owns the device-specific portions of clock operation.

These include:

- coherent hardware reads;
- optional hardware writes;
- device-specific errors;
- bounded waits where required;
- conversion from the device representation to the canonical TIME record; and
- conversion from the canonical record when SET is supported.

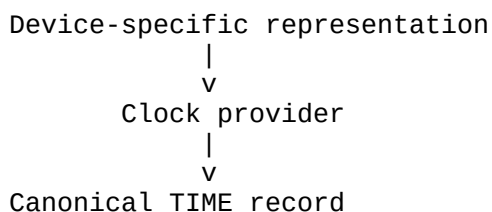
Hardware clocks need not use the BetterCP/M representation internally.

A device might expose:

- BCD calendar registers;
- binary calendar fields;
- an epoch counter;
- host-supplied emulator time; or
- another representation.

The provider absorbs these differences.

Thus:



Native clients see the same TIME ABI regardless of the provider.

BetterCP/M 1.0 includes qualified providers for the FreHD environment and for z80pack/cpmsim. **FREHDCLK.RSX** accesses the FreHD clock interface, while **ZPRTC.RSX** accesses the z80pack real-time clock interface.

Both advertise the same logical **TIME** service.

A client therefore does not require separate FreHD and z80pack versions merely to display the date.

16.7 Provider Discovery

Clock providers use the ordinary BetterCP/M resident-service architecture.

A client requests service ID:

TIME

with the required ABI major, minimum minor, and capability mask.

Only zero or one **TIME** provider can be active under the BetterCP/M 1.0 registry rules. A prospective configuration containing duplicate **TIME** providers is rejected before publication.

The service identity is stable across provider implementations.

The following are therefore different concepts:

TIME	service identity
ABI 1.0	service interface
FREHDCLK.RSX	provider identity
current address	runtime binding

Clients should depend upon the first two.

The callable address returned by discovery remains valid only for the resident-layout generation in which it was obtained.

A client whose lifetime crosses resident reconstruction must rediscover **TIME** afterward.

16.8 Native Consumers

TIME.COM is the standard native BetterCP/M clock utility.

It discovers and invokes the **TIME** service directly.

The minimum BetterCP/M 1.0 interface includes:

TIME
TIME /PROVIDER

The first displays the current local date and time.

The second identifies the installed clock provider.

The utility owns presentation concerns such as:

- command syntax;
- input parsing;
- day-count/calendar conversion;
- display formatting;
- help text; and
- human-readable error reporting.

These operations do not belong in every clock provider.

The TIME service deliberately exposes the compact canonical record rather than Gregorian structures, formatted strings, locale information, or display policy.

This keeps the resident provider small and leaves human presentation with the transient consumer.

16.9 P2DOS Compatibility

BetterCP/M provides P2DOS clock-call compatibility through the optional P2DOS.RSX frontend.

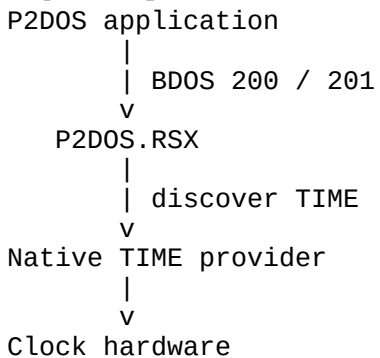
P2DOS defines:

Function 200 GET TIME
Function 201 SET TIME

with DE addressing the caller's five-byte clock record.

P2DOS.RSX intercepts these functions and translates them into calls to the native TIME service.

Conceptually:



The frontend contains no hardware driver.

Because the P2DOS and BetterCP/M native clock records have the same representation, ordinary record conversion is unnecessary. The frontend nevertheless remains responsible for translating interface behavior and result conventions.

For each request, P2DOS.RSX resolves TIME, invokes the returned provider immediately, translates the result, and discards the callable address.

It does not retain the address across invocations or resident reconstruction.

This deliberately avoids creating another long-lived rebinding mechanism.

16.10 P2DOS Result Behavior

BetterCP/M freezes a compact result convention for the P2DOS compatibility frontend.

Successful Functions 200 and 201 return:

A = 00H
HL = 0000H

and preserve the caller's DE buffer address.

If P2DOS.RSX is installed and claims the request but the operation fails:

A = FEH
HL = 00FEH

The **FEH** result is BetterCP/M-defined deterministic behavior rather than historical P2DOS behavior.

Native failures such as:

- unavailable clock;
- hardware failure;
- invalid record; or
- unsupported SET

collapse into this compatibility-level failure result.

If no frontend handles Function 200 or 201, final BDOS fallback returns:

```
A = FFH
HL = 00FFH
```

This means the P2DOS compatibility interface is not installed.

It does **not** mean that BetterCP/M has no clock.

A native TIME provider can be fully operational while **P2DOS.RSX** is absent.

Conversely, **P2DOS.RSX** can be installed while no TIME provider is available; in that case the frontend claims the request and returns its documented unavailable result.

This preserves the architectural separation between native clock service and historical compatibility.

16.11 Read-Only Providers

A read-only provider is a complete TIME provider for GET operations.

It advertises the TIME service without the SET capability.

Native clients can discover this distinction before attempting to modify the clock.

The P2DOS compatibility frontend behaves somewhat differently because Function 201 remains part of the interface whenever **P2DOS.RSX** is installed.

A P2DOS SET request therefore still reaches the frontend. The frontend calls the native TIME service, receives **SET_UNSUPPORTED**, and maps that result to:

```
A = FEH
HL = 00FEH
```

This distinction is deliberate.

The provider reports its actual native capability.

The compatibility frontend preserves the historical selector interface and translates the provider's result into its own public convention.

No special-case clock driver is required.

16.12 Other Historical Clock Interfaces

Other CP/M-family systems used clock interfaces different from P2DOS.

BetterCP/M 1.0 identifies two bounded optional candidates using BDOS Functions 104 and 105:

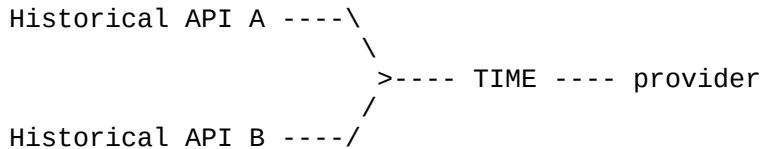
- a CP/M 3 / DOS+ 2.5 clock-call profile; and
- a Z80DOS 2.4 clock-call profile.

These interfaces differ in record size and result behavior and cannot safely be autodetected from an incoming call.

For example, one profile expects a four-byte buffer while another expects five bytes. Writing a five-byte result to a caller that supplied only four bytes could corrupt adjacent memory.

If these interfaces are implemented, they therefore require separate, mutually exclusive compatibility frontends.

Conceptually:



Each frontend owns:

- its historical selector convention;
- record translation;
- public result behavior; and
- validation peculiar to that interface.

None duplicates the hardware provider.

These optional frontends are not required BetterCP/M 1.0 release facilities and are not part of the default resident profile.

16.13 Clock Data and Calendar Conversion

TIME ABI 1.0 exposes only the canonical five-byte clock record.

It does not provide resident operations for:

- converting the day count to year/month/day;
- converting Gregorian dates to day counts;
- formatting dates;
- formatting times;
- localized names;
- locale policy; or
- textual presentation.

Transient consumers such as **TIME.COM** can perform these transformations themselves.

Source code for such transformations can be shared among programs without requiring the corresponding code to become permanently resident.

If future BetterCP/M facilities demonstrate that multiple resident clients need the same neutral conversion operations, a separate optional callable service can be considered.

BetterCP/M 1.0 does not reserve an ABI, service ID, request block, or memory allocation for such a hypothetical facility.

The provider rule remains:

Providers normalize hardware into the smallest stable domain record; consumers perform presentation.

16.14 Clock Services and File Timestamps

The presence of a system clock does not imply filesystem timestamp support.

BetterCP/M 1.0 defines the TIME service independently of CP/M directory timestamp formats.

A clock provider supplies the current date and time.

It does not determine:

- whether files carry timestamps;
- where timestamp data is stored;
- which historical timestamp convention is used;
- when file timestamps are updated; or
- how timestamps interact with directory compatibility.

Likewise, loading **P2DOS.RSX** adds P2DOS clock-call compatibility only. It does not automatically add P2DOS filesystem timestamp behavior.

This distinction is important because several CP/M-family systems and extensions used different timestamp schemes.

Supporting one clock-call convention does not establish compatibility with the corresponding operating system's directory extensions.

Filesystem timestamps therefore remain separate later work rather than an implicit part of BetterCP/M 1.0 clock support.

16.15 Date and Time Invariants

Several rules summarize the BetterCP/M 1.0 date and time architecture.

Clock hardware is hidden behind a provider implementing the native TIME ABI.

Native consumers use TIME directly rather than requiring a historical compatibility frontend.

Historical frontends translate their interfaces into TIME rather than duplicating hardware drivers.

TIME uses one canonical five-byte local-time record.

GET publishes a result only after obtaining and validating a coherent sample.

SET validates the complete requested record before modifying hardware.

Provider capabilities describe whether SET is supported.

Service identity is independent of provider filename and current address.

A callable provider address is valid only for its resident-layout generation.

P2DOS compatibility does not define the native clock architecture merely because the native record uses the same compact representation.

Clock-call compatibility does not imply filesystem timestamp compatibility.

Finally, the resident clock architecture remains deliberately small. BetterCP/M 1.0 provides the date and time information required by the operating system and its applications without turning the clock provider into a calendar library, scheduler, timezone system, or collection of historical APIs.

This preserves the broader BetterCP/M principle: **one small native service owns the stable operating-system concept; hardware providers and compatibility adapters remain separate around it.**

17. Compatibility Architecture

CP/M 2.2 compatibility is a primary requirement of BetterCP/M 1.0. The objective is to provide an operating environment in which ordinary CP/M 2.2 applications can execute without modification and encounter the behavior upon which they depend.

Compatibility is defined principally by **observable behavior**, not by duplication of Digital Research's implementation. BetterCP/M is a new operating system implementation and is free to use different algorithms, private data structures, component boundaries, memory organization, and Z80-specific techniques where those differences are invisible to compatible software.

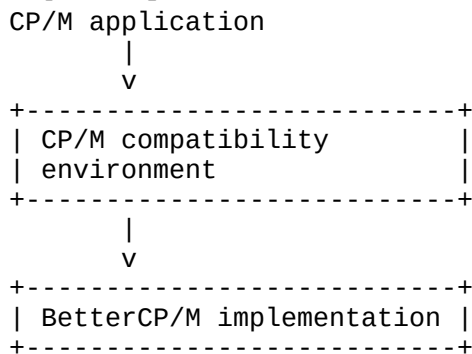
The boundary is therefore behavioral rather than historical. An implementation detail becomes a compatibility concern when software can observe it and useful CP/M software reasonably depends upon it. Conversely, the fact that Digital Research implemented something in a particular way does not by itself require BetterCP/M to reproduce that mechanism.

BetterCP/M treats compatibility as an explicit, testable contract. Documented CP/M interfaces establish the starting point; observed behavior and representative software provide additional evidence where real programs depend upon characteristics not fully specified by the documentation.

17.1 Compatibility Model

The BetterCP/M compatibility model distinguishes between the **environment presented to software** and the **mechanisms used to implement that environment**.

Conceptually:



The upper boundary must behave as required by CP/M-compatible software.

The implementation below it can differ substantially from Digital Research CP/M.

Compatibility can therefore require preservation of:

- calling conventions;
- memory locations;
- data structures;
- command behavior;
- file-system semantics;
- return values;
- side effects;
- initialization state;
- error behavior; and

- other observable characteristics.

It does not automatically require preservation of:

- source-code organization;
- private algorithms;
- internal module boundaries;
- private data structures;
- implementation language conventions; or
- other mechanisms that software cannot observe.

The distinction is central to BetterCP/M. Compatibility constrains what the system presents externally without unnecessarily constraining how the system is built internally.

17.2 Application Compatibility

BetterCP/M 1.0 aims to execute ordinary CP/M 2.2 transient applications without modification.

An application that confines itself to the supported CP/M execution environment and operating-system interfaces should not need to know that BetterCP/M is providing those interfaces.

Application compatibility can depend upon such things as:

- program loading and entry state;
- page-zero contents;
- system-service conventions;
- File Control Blocks;
- command tails;
- default FCB construction;
- DMA behavior;
- console behavior;
- file-system semantics;
- termination; and
- warm-start behavior.

These requirements cross several BetterCP/M architectural components. Compatibility is therefore not the responsibility of one isolated "compatibility module."

For example:

```

Application compatibility
|
+-- Program Execution Environment
+-- BDOS/System Services
+-- Command Environment
+-- File System
+-- BIOS where applicable

```

The components collectively reproduce the environment required by the application.

Application compatibility does not imply hardware compatibility with every historical CP/M machine. Software that bypasses CP/M interfaces and directly accesses machine-specific hardware can impose additional platform requirements.

17.3 BDOS Compatibility

BetterCP/M preserves the application-visible system-service interface required by CP/M 2.2.

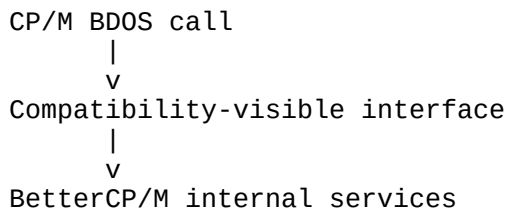
Where required for compatibility, this includes:

- function numbers;
- calling conventions;
- input parameters;
- returned register values;
- memory structures;
- side effects;
- state changes; and
- error behavior.

An application continues to request ordinary CP/M services through the conventional entry mechanism.

Internally, BetterCP/M can translate the request into another representation or divide the implementation among components differently from Digital Research CP/M.

Thus:



The two interfaces need not be identical.

BetterCP/M-specific services occupy explicitly assigned extension interfaces rather than silently redefining standard CP/M functions.

Likewise, optional compatibility frontends such as **P2DOS.RSX** can implement historical interfaces without making those interfaces part of the native BetterCP/M service architecture.

17.4 Command-Environment Compatibility

The BetterCP/M Command Environment preserves CP/M 2.2 command behavior where that behavior forms part of the compatibility environment.

Relevant characteristics include:

- prompt and command-entry conventions;
- command parsing;
- resident commands;
- transient-program lookup;
- default program-file behavior;
- command-tail construction;
- default FCB construction;
- error behavior; and
- return to command processing after execution.

BetterCP/M does not need to reproduce the internal design of the Digital Research CCP.

Its own CCP is smaller, supports CPXs, can be reclaimed during transient execution, and can be reconstructed afterward.

Those architectural differences are permissible because they occur behind the compatibility boundary.

BetterCP/M can also add command-environment facilities such as direct drive/user navigation, line editing, history, and additional resident commands.

Such additions must not silently change the meaning of established CP/M command forms.

An extension is therefore compatible when ordinary CP/M usage retains its expected meaning while the additional behavior is available to users who choose to invoke it.

17.5 Memory and Execution Compatibility

Transient programs receive the memory and execution environment required by CP/M 2.2.

Compatibility-visible elements include such things as:

- low-memory locations;
- entry vectors;
- program origin at 0100H;
- system-service entry;
- warm-start mechanisms;
- command information;
- default FCBs;
- default DMA state; and
- the available TPA boundary.

Some software can also depend upon locations or behavior that were not originally intended as formal application interfaces.

BetterCP/M therefore does not classify a memory location as irrelevant merely because Digital Research regarded it as an implementation detail.

The relevant question is whether useful existing software observes or modifies it and whether preserving that behavior is justified by the compatibility objective.

BetterCP/M-private memory remains free to use different representations where no adopted compatibility requirement applies.

This allows BetterCP/M to use reconstructible command components, different resident organization, a PDS, dynamically configured RSXs, and other internal mechanisms without exposing those differences to ordinary applications.

17.6 File-System and Disk Compatibility

BetterCP/M preserves the CP/M 2.2 logical file-system model required by compatible applications.

This includes applicable behavior involving:

- filenames and file types;
- FCBs;
- directory entries;
- user areas;

- records;
- extents;
- allocation;
- searches;
- sequential access;
- random access;
- file creation;
- file deletion;
- renaming; and
- file attributes.

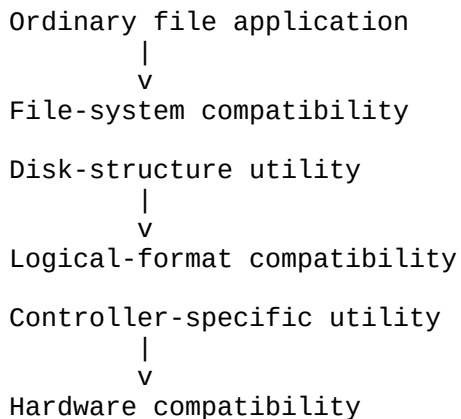
File-system compatibility is distinct from compatibility with a particular disk format.

An application using normal BDOS file services operates upon the logical CP/M filesystem and need not know how its disk is physically represented.

Software that examines directory structures directly can additionally depend upon the logical disk format.

Software that accesses a disk controller directly can additionally depend upon the physical machine.

Thus:



BetterCP/M can support multiple CP/M disk formats without claiming that every historical CP/M format is automatically supported.

17.7 BIOS and Hardware Compatibility

Operating-system compatibility and machine compatibility are separate concerns.

Software using ordinary CP/M System Services should normally remain independent of the underlying machine.

Some programs, however, can:

- call BIOS entry points directly;
- inspect BIOS-related structures;
- depend upon particular BIOS behavior; or
- access physical hardware directly.

Where BetterCP/M adopts a BIOS behavior as part of its compatibility contract, that observable behavior must be provided.

The internal BetterCP/M hardware interface does not have to be identical to the compatibility-visible BIOS interface.

Direct hardware dependencies are narrower still.

A program written for a particular video controller, disk controller, memory map, or machine-specific I/O port cannot be assumed to work on another BetterCP/M platform merely because both systems run CP/M-compatible software.

BetterCP/M does not generalize the hardware peculiarities of individual historical CP/M machines into requirements for every platform.

A platform can emulate such hardware where desired, but that is an additional platform capability rather than part of the general CP/M 2.2 compatibility claim.

17.8 Documented and De Facto Behavior

Digital Research documentation provides the starting point for BetterCP/M compatibility.

Documentation alone, however, does not necessarily describe every behavior upon which existing software came to depend.

Compatibility investigation therefore also considers:

- behavior of reference CP/M 2.2 systems;
- original implementation behavior where relevant;
- representative existing applications;
- established operating procedures; and
- other evidence of real-world dependencies.

This can reveal requirements involving:

- undocumented memory locations;
- register preservation;
- particular return values;
- initialization state;
- command-processing details;
- error behavior; or
- BIOS behavior.

BetterCP/M does not automatically preserve every accidental characteristic observed in one CP/M implementation.

Instead, compatibility evidence is classified conceptually as:

Documented CP/M requirement

Observed reference behavior

Demonstrated software dependency

Unresolved behavior

A de facto behavior becomes part of the BetterCP/M compatibility contract when there is sufficient reason to believe that preserving it materially improves useful CP/M 2.2 compatibility.

Such decisions should be explicit rather than accidental.

Where practical, an adopted compatibility decision is also represented by an executable conformance test so that later implementation changes do not silently remove it.

17.9 Implementation Freedom

Compatibility does not require BetterCP/M to be internally identical to Digital Research CP/M.

Subject to its external contracts, BetterCP/M can:

- reorganize components;
 - use different algorithms;
 - use different private data structures;
 - introduce clearer internal interfaces;
 - separate responsibilities that CP/M combined;
 - combine implementation where separate layering provides no benefit;
 - use Z80 instructions unavailable to an 8080-oriented implementation;
- and
- improve internal consistency.

For example, BetterCP/M's CPX architecture has no requirement to reproduce the internal structure of the Digital Research CCP merely because it provides CP/M-compatible command behavior.

Similarly, the resident-service registry, reconstructible RSX architecture, normalized disk state, and TIME service are BetterCP/M mechanisms rather than historical CP/M structures.

Implementation freedom ends where the difference becomes visible in a way that violates an adopted compatibility requirement.

The converse is equally important:

Compatibility is not a reason to preserve a Digital Research implementation detail for which no compatibility requirement exists.

This gives BetterCP/M room to improve the operating system without turning compatibility into source-code archaeology.

17.10 Compatibility Testing

Compatibility is demonstrated through testing rather than inferred from architectural similarity.

BetterCP/M testing examines both individual interfaces and complete application behavior.

Relevant tests include:

- BDOS function tests;
- memory-environment tests;
- command-environment tests;
- file-system tests;
- program loading and termination tests;
- disk-format tests;
- BIOS-interface tests where applicable;
- representative CP/M application tests; and
- comparison with reference CP/M 2.2 behavior.

A compatibility test should identify what behavior is being preserved and why.

Where BetterCP/M intentionally differs from observed CP/M behavior, the test and documentation should distinguish that decision from an accidental failure.

The compatibility suite therefore serves two purposes.

First, it provides evidence for the current compatibility claim.

Second, it acts as a regression suite as BetterCP/M evolves.

Conceptually:

Compatibility investigation

|

v

Behavioral decision

|

v

Executable test

|

v

Implementation

|

v

Regression testing

This process turns compatibility knowledge into a maintained engineering artifact rather than relying upon the memory of individual developers.

17.11 BetterCP/M Extensions and Compatibility

BetterCP/M provides capabilities beyond CP/M 2.2 while preserving the ordinary CP/M environment.

Examples include:

- CPXs;
- dynamically managed RSXs;
- callable resident services;
- BetterCP/M-private BDOS functions;
- expanded disk configuration;
- command history;
- direct drive/user navigation; and
- the native TIME service.

Ordinary CP/M applications do not need to use these facilities.

Where an extension adds behavior to an existing environment, established CP/M semantics remain intact.

Where BetterCP/M requires genuinely new semantics, it uses an explicitly different interface.

For example:

CP/M BDOS function

-> retains CP/M semantics

BetterCP/M Function 182

-> explicitly BetterCP/M service

TIME callable service

-> explicitly BetterCP/M ABI

P2DOS Functions 200/201

-> optional historical compatibility frontend

This separation prevents extension from gradually redefining what "CP/M compatible" means.

A future BetterCP/M facility that cannot coexist transparently with CP/M behavior should likewise identify itself as an extended or different environment rather than silently changing the compatibility contract.

17.12 Compatibility Limits

BetterCP/M makes deliberately bounded compatibility claims.

The BetterCP/M 1.0 objective is compatibility with the CP/M 2.2 operating-system environment.

It does not automatically imply compatibility with:

- CP/M 3;
- MP/M;
- every vendor-modified CP/M;
- every replacement BDOS;
- every replacement command processor;
- every historical RSX or extension;
- every CP/M disk format; or
- every machine historically capable of running CP/M.

Some interfaces from other CP/M-family systems can be supported separately.

P2DOS clock Functions 200 and 201 are an example. Their presence through P2DOS.RSX does not imply general P2DOS compatibility.

Likewise, implementing one historical clock-call convention does not imply support for that system's filesystem timestamp format.

Software depending upon undocumented or machine-specific behavior must be evaluated individually.

Known incompatibilities should be documented explicitly.

The compatibility objective is therefore:

Make narrow claims that the implementation and conformance evidence can actually support.

A broad assertion of universal "CP/M compatibility" is less useful than a precisely defined CP/M 2.2 compatibility environment whose limits are understood and tested.

17.13 Distribution Utility Compatibility

A complete BetterCP/M distribution includes freely redistributable replacements for the standard commands, tools, and utilities required by the CP/M 2.2 operating environment.

Compatibility for these utilities means more than providing another program with a similar purpose or familiar name.

Where a BetterCP/M utility replaces a standard CP/M utility, applicable externally observable behavior includes:

- command syntax;
- defaults;
- file effects;
- invocation state;
- interaction;
- termination;
- error paths; and
- behavior relied upon by existing command files and operating procedures.

BetterCP/M utilities can add capabilities provided ordinary CP/M syntax retains its established meaning.

For example, BetterCP/M can extend utilities with its drive/user qualification facilities without removing the conventional forms expected by CP/M users.

Shared syntax should be implemented consistently rather than independently reinvented by each utility.

Some historical utilities are inseparable from Digital Research or vendor-specific system images. BetterCP/M replacements for such tools preserve the relevant workflow and purpose where practical without pretending that obsolete image-patching details are portable compatibility requirements.

Original Digital Research binaries are not BetterCP/M release dependencies.

The **Utility Compatibility Ledger** records, for each required utility:

- the historical baseline;
- required observable behavior;
- the BetterCP/M replacement;
- permitted extensions;
- implementation status; and
- conformance evidence.

A replacement is not considered complete merely because a program with the correct filename exists. Its compatibility requirements must be demonstrated through executable tests or explicitly documented manual evidence.

This applies the same principle used throughout BetterCP/M to the distribution itself:

Compatibility is a behavioral contract, not a naming convention.

18. Architectural Constraints and Evolution

BetterCP/M is intended to remain a small, comprehensible operating system for resource-constrained Z80 computers while preserving a useful CP/M 2.2-compatible environment. The architecture therefore defines not only the facilities BetterCP/M provides, but also constraints on how those facilities are implemented and extended.

These constraints prevent local implementation decisions from gradually undermining compatibility, consuming application memory without sufficient benefit, blurring component ownership, or introducing generalized machinery inappropriate to the system.

An architectural invariant is not simply a characteristic of the current source code. It is a property expected to remain true even when the implementation changes. Conversely, ordinary implementation details should not be elevated to invariants unless compatibility, correctness, comprehensibility, or an established architectural boundary requires them.

BetterCP/M can evolve. Its architecture is not intended to freeze the operating system permanently at version 1.0. Architectural change should, however, be deliberate: new requirements can justify revising boundaries and mechanisms, but implementation convenience alone should not silently overturn them.

18.1 System Invariants

Several principles apply throughout BetterCP/M 1.0.

The system provides a CP/M 2.2-compatible operating environment for ordinary transient applications.

Compatibility-visible interfaces retain their required semantics regardless of BetterCP/M's private implementation.

BetterCP/M-specific extensions do not silently redefine established CP/M behavior.

The system remains single-tasking. At most one transient application executes at a time.

Every significant system component has a defined responsibility.

Every persistent item of system state has a defined owner and lifetime.

Shared state has one authoritative value rather than multiple independent authoritative copies.

Hardware-independent code does not unnecessarily depend upon machine-specific implementation details.

Logical CP/M storage remains conceptually distinct from the physical devices used to contain it.

Resident memory remains a constrained resource because it competes directly with transient application memory.

Finally, architectural separation does not require elaborate implementation. A simple call, table, vector, or data structure is sufficient when it preserves the required responsibility and interface boundary.

These principles recur throughout the individual subsystem architectures described in this manual.

18.2 Memory and Resource Constraints

BetterCP/M 1.0 operates within the ordinary 16-bit Z80 address space.

The system does not require:

- bank switching;
- virtual memory;
- memory protection; or
- multiple simultaneous application address spaces.

Memory occupied by protected resident facilities reduces the space available to transient applications and must therefore be justified.

The default BetterCP/M 1.0 configuration preserves a complete-record transient-program capacity of at least:

53 KiB

This accepted floor is an architectural constraint rather than merely an incidental result of the current linker map.

Optional RSXs can reduce the TPA when the user deliberately installs additional resident functionality. CPXs do not impose the same permanent cost because their command-environment memory is reclaimable during transient execution.

Other resources remain constrained as well.

BetterCP/M considers:

- executable code size;
- writable resident RAM;
- disk storage;
- stack requirements;
- workspace requirements;
- reconstruction cost; and
- execution time.

Resource efficiency is not pursued blindly. Saving a few bytes does not justify destroying a useful architectural boundary or making the system unnecessarily difficult to understand.

Conversely, architectural elegance does not justify substantial permanent overhead when a simpler mechanism satisfies the actual requirement.

18.3 Interface Stability

BetterCP/M distinguishes architectural interfaces from implementation-private relationships.

A published interface defines such matters as:

- version;
- inputs;
- outputs;
- preserved and clobbered registers;
- flag meanings;
- memory effects;

- pointer lifetimes;
- error behavior;
- state effects; and
- applicable atomicity or reconstruction rules.

Execution alone does not make an interface stable.

An experimental or provisional interface can exist in working code while remaining subject to revision.

A stable ABI requires its actual contract to be defined.

BetterCP/M release numbers and individual ABI versions are also distinct. Software should not infer compatibility with a particular interface merely because it recognizes the operating-system release number.

Movable objects introduce an additional rule:

Runtime address is not persistent identity.

RSXs, CPXs, and other reconstructible objects can move. Interfaces that survive such movement use stable identifiers, handles, gateways, or rediscovery rather than treating a current absolute address as permanent.

Where an address is returned legitimately, the interface defines how long that address remains valid.

18.4 Compatibility Constraints

CP/M 2.2 compatibility constrains BetterCP/M wherever software can observe the operating environment and reasonably depends upon its behavior.

Compatibility requirements take precedence over internal elegance when the two genuinely conflict.

This does not mean that every characteristic of Digital Research CP/M becomes an architectural constraint.

BetterCP/M distinguishes:

Documented CP/M behavior

Observed reference behavior

Demonstrated software dependency

Implementation-private detail

The first three can provide evidence for a compatibility requirement.

The fourth does not become a requirement merely because Digital Research implemented CP/M that way.

BetterCP/M is therefore free to reorganize its BDOS, CCP, resident memory, storage implementation, hardware interface, and other private mechanisms provided that the adopted compatibility environment remains intact.

Where incompatible behavior is desirable in a future extension, BetterCP/M should expose that behavior explicitly rather than silently changing the CP/M-compatible environment.

18.5 Extension Constraints

The existence of CPXs, RSXs, resident services, and private BetterCP/M system calls does not imply a general requirement that every operating-system component be dynamically extensible.

BetterCP/M 1.0 provides extension machinery for concrete requirements.

It does not provide generalized:

- module dependency resolution;
- package management;
- dynamic linking;
- arbitrary plugin registration;
- driver frameworks;
- service stacking;
- automatic module ordering; or
- persistent extension allocation.

New extension mechanisms should be introduced only when actual requirements demonstrate their value.

Extensions also remain subject to the same resource and compatibility constraints as the core system.

An extension intended to coexist with ordinary CP/M applications must preserve the CP/M environment.

An RSX must justify its protected resident-memory cost.

A CPX must remain reconstructible and cannot be the sole owner of indispensable state that must survive destruction of the command environment.

A callable resident service must define its ABI and address lifetime.

A new private BDOS selector must be allocated deliberately rather than chosen opportunistically.

Extension architecture exists to make useful growth possible, not to maximize theoretical generality.

18.6 Simplicity and Comprehensibility

BetterCP/M favors simple, explicit mechanisms where they satisfy the actual requirements.

This does not mean that the operating system is intended to be a simplified simulation or pedagogical toy. The architecture is driven by the requirements of a practical CP/M-compatible operating system.

Where several designs satisfy those requirements, preference should normally be given to the design that makes the resulting system easier to:

- understand;
- investigate;
- document;
- test;
- maintain; and
- modify.

Educational value follows from this clarity rather than from introducing mechanisms solely for demonstration.

Likewise, BetterCP/M does not adopt facilities merely because they are customary in larger or more modern operating systems.

For example, a compact Z80 calling convention can provide a perfectly adequate architectural boundary without requiring an object model, generalized driver manager, or dynamic linker.

The system should remain possible for an experienced programmer to trace from command input through service dispatch, storage or hardware operation, and back again without requiring a large hidden framework to explain what occurred.

18.7 BetterCP/M 1.0 Architectural Boundary

BetterCP/M 1.0 establishes a specific system boundary.

Within that boundary are the facilities required to provide the defined CP/M 2.2-compatible environment and the BetterCP/M extensions adopted for the release.

These include the established architecture for:

- the command environment;
- CPXs;
- RSXs;
- resident-service discovery;
- system configuration;
- disk-format configuration;
- persistent command history;
- native date and time service;
- hardware abstraction;
- CP/M-compatible execution;
- storage and file services; and
- the BetterCP/M-private system-service namespace.

The 1.0 architecture deliberately does not promise mechanisms merely because they have been discussed as possible future facilities.

A design note, unused metadata field, reserved selector, or conceivable application does not by itself constitute a published BetterCP/M interface.

This distinction is particularly important for future compatibility.

Software written against BetterCP/M 1.0 should be able to distinguish between:

```
published 1.0 contract
  |
  v
safe dependency

implementation detail
  |
  v
not a compatibility promise

future possibility
  |
  v
no current interface
```

This permits later releases to develop new facilities without being constrained by speculative details that were never intended as contracts.

18.8 Deferred Facilities

Several capabilities have been considered but remain outside the BetterCP/M 1.0 architectural contract.

These include, among other possibilities:

- named-directory facilities;
- NDR management;
- PATH semantics;
- generalized persistent-data allocation;
- dynamic PDS sizing;
- bank-switched memory;
- filesystem timestamps;
- additional historical clock frontends;
- generalized RSX dependency metadata;
- multiple providers for one resident-service ID;
- resident-service priority or stacking;
- generalized module relationship graphs;
- additional foreign-filesystem support;
- broader debugging facilities; and
- alternative command environments.

Deferral does not mean that these facilities are rejected permanently.

It means that BetterCP/M 1.0 does not reserve unnecessary runtime machinery or freeze premature ABIs for them.

Some can be added later using boundaries already present in the architecture.

For example, additional clock compatibility interfaces can consume the existing TIME service without modifying clock providers.

Additional logical disk formats can use the established disk-format and storage boundaries.

Future resident facilities can use the RSX and resident-service architecture where those mechanisms fit their requirements.

Other features can eventually demonstrate requirements substantial enough to justify architectural revision.

The distinction is deliberate:

BetterCP/M preserves reasonable room for growth without paying in advance for growth that may never be required.

18.9 Future Architectural Evolution

BetterCP/M's architecture can change when investigation, implementation experience, compatibility testing, or new requirements demonstrate that a change is justified.

Architectural invariants are therefore strong constraints, but they are not immutable historical artifacts.

A proposed change should examine its effects upon:

- CP/M compatibility;

- published BetterCP/M interfaces;
- memory organization;
- component ownership;
- persistent state;
- extension behavior;
- storage architecture;
- hardware ports;
- engineering specifications;
- conformance tests; and
- existing BetterCP/M software.

A requirement applying specifically to BetterCP/M 1.0 can be replaced in a later architecture when the new architecture explicitly defines the replacement behavior.

Such changes should be deliberate and documented.

An implementation should not quietly violate an established invariant merely because doing so simplifies one feature.

CP/M compatibility deserves particular protection. A future BetterCP/M capability should not silently discard established CP/M 2.2 behavior merely because the new facility would be easier to implement without it.

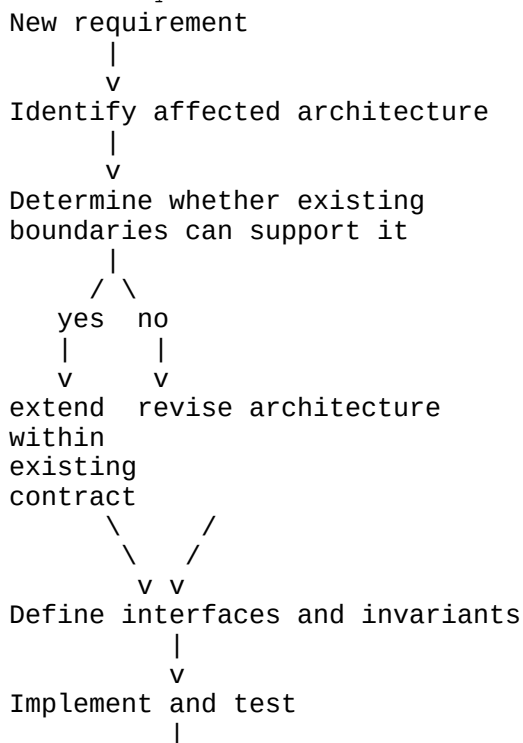
Where genuinely incompatible behavior becomes desirable, BetterCP/M can define an explicitly extended or different environment.

This permits evolution without making the meaning of the existing compatibility contract unstable.

18.10 Architectural Change Process

Architectural evolution should begin with a demonstrated requirement.

A useful sequence is:



Update architectural documentation

The preferred outcome is not always to preserve the existing architecture unchanged.

Sometimes implementation experience demonstrates that an earlier boundary was incorrect or that a mechanism no longer serves the system well.

In that case, retaining the old design solely because it already exists can be as harmful as changing it casually.

The important distinction is between **deliberate architectural revision** and **architectural drift**.

The former evaluates consequences and updates the system's contracts.

The latter allows implementation details to change the architecture without anyone explicitly deciding that the architecture should change.

BetterCP/M permits the first and seeks to prevent the second.

18.11 Long-Term Compatibility

As BetterCP/M evolves, it acquires compatibility obligations of its own.

CP/M 2.2 defines the original compatibility environment, but published BetterCP/M interfaces can also become dependencies of applications, extensions, utilities, and platform implementations.

A stable BetterCP/M ABI should therefore not be changed casually merely because the operating system controls both sides of the original implementation.

Future releases should distinguish among:

- CP/M compatibility contracts;
- stable BetterCP/M contracts;
- provisional or experimental interfaces; and
- implementation-private mechanisms.

This permits experimentation without turning every experiment into a permanent obligation.

It also permits stable BetterCP/M software to depend upon interfaces that have deliberately reached that status.

The architecture should consequently become more explicit, not less, as the system grows.

New facilities should identify which portions are stable contracts and which remain implementation choices.

18.12 Architectural Invariants Summary

The principal BetterCP/M architectural invariants can be summarized as follows.

Compatibility

Ordinary CP/M 2.2 applications receive the environment they require without BetterCP/M-specific modification.

Execution

At most one transient application executes at a time, and termination can restore a usable command environment.

Memory

Every memory region has a defined owner and lifetime. Protected resident memory is charged against application capacity.

Components

Major responsibilities remain explicit, and internal boundaries are preserved where they prevent inappropriate coupling.

Storage

Logical CP/M storage remains distinct from physical storage, and each configured logical disk has one authoritative interpretation.

Hardware

Machine-specific knowledge remains bounded behind the hardware-dependent portion of the system where practical.

State

Every persistent item of state has an owner and lifetime, and shared state has one authoritative representation.

Extensions

Extensions preserve existing compatibility behavior unless they explicitly define another environment, and extension machinery remains proportional to demonstrated requirements.

Resources

Memory, code size, execution cost, and complexity remain architectural concerns.

Evolution

Architectural change is permitted, but it is deliberate, documented, and reflected in the applicable specifications and tests.

Together these constraints define the character of BetterCP/M as much as any individual subsystem does.

The system is intended to grow beyond CP/M 2.2 without losing the properties that make a small Z80 operating system understandable and useful: a clear execution model, explicit interfaces, bounded state, visible resource costs, and a compatibility environment whose behavior can be investigated and tested.

Appendix A. BetterCP/M 1.0 Memory Map

The following map summarizes the default BetterCP/M 1.0 memory organization. Addresses belonging to dynamically reconstructed components can vary with the active configuration; fixed values shown here describe the accepted 1.0 default layout.

FFFFH	+-----+
	BIOS / Hardware Support
	Core BDOS / System Services
	Fixed Resident State
	+-----+
	Persistent Data Segment
D5C3H	
D504H	+-----+
	Installed RSXs
	(configuration dependent)
	+-----+
	CP/M Compatibility Gateway
	+-----+
	Installed CPXs
	(reclaimable)
	+-----+
	Reloadable CCP
	(reclaimable)
	+-----+
D501H	TPA ceiling (exclusive)
	+-----+
	Transient Program Area
0100H	+-----+
	Page Zero
0000H	+-----+

The default complete-record transient-program capacity is **53 KiB**, or 54,272 bytes.

The PDS occupies 192 bytes from **D504H** through **D5C3H** and is owned in BetterCP/M 1.0 solely by command history.

RSX installation can lower the TPA ceiling because RSXs remain protected during transient execution. CPXs and the reloadable CCP occupy reclaimable command-environment memory and therefore do not impose the same permanent TPA cost.

Runtime addresses of reconstructible components are not persistent identities.

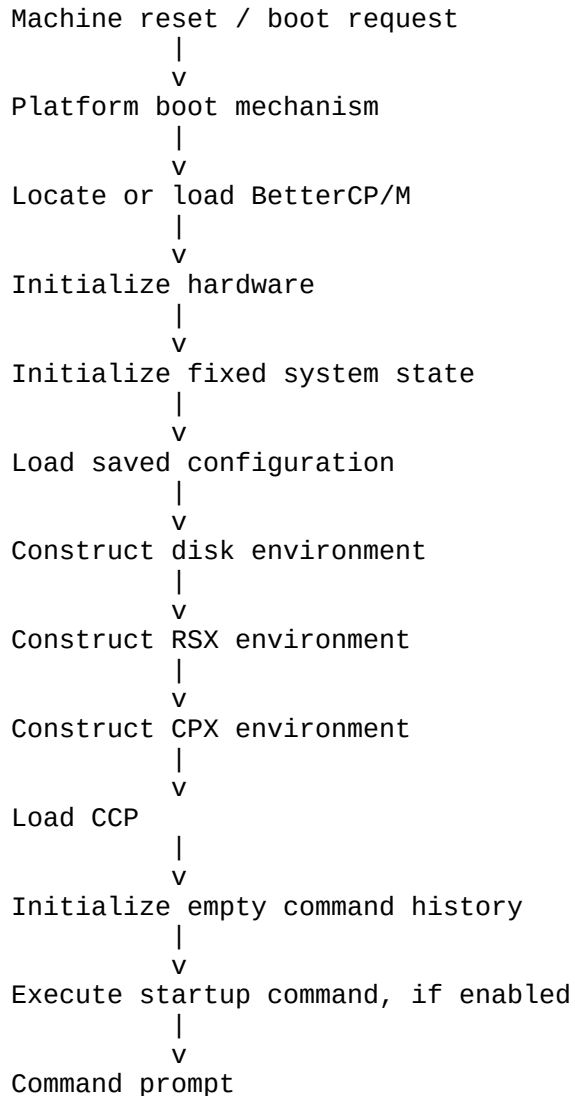
Appendix B. System Component Summary

Component	Principal Responsibility
-----	-----
Boot System	Establish initial BetterCP/M environment
CCP	Core command processing, navigation, transient invocation
CPX	Reclaimable command-environment extension
BDOS / System Services	CP/M-compatible and BetterCP/M system services
RSX	Protected resident system/service extension
Resident-Service Gateway	Discovery of callable resident services
PDS	Persistent command-history state
File System	CP/M files, records, extents, directories and allocation
Disk-Format Layer	Logical disk interpretation and translation
Hardware Abstraction	Machine-dependent console, storage and peripheral operations
BIOS / Platform Code	Platform implementation of hardware- dependent facilities
Program Execution Environment	Establishment and termination of the transient application environment

The principal distinction among these components is responsibility and lifetime, not necessarily separate source files or physical memory regions.

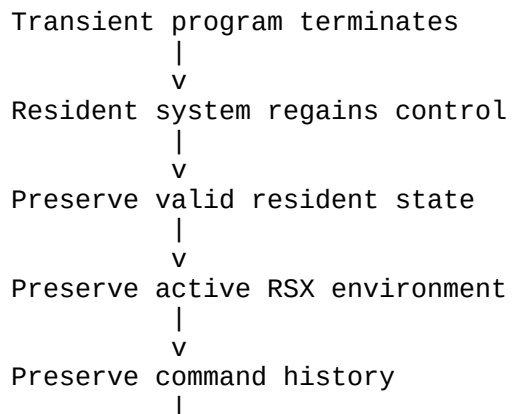
Appendix C. Boot and Reconstruction Sequence

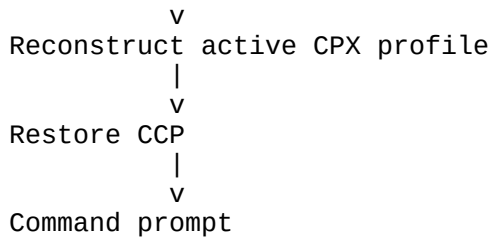
C.1 Cold Boot



A cold boot creates a new runtime instance and initializes state whose lifetime begins with that instance.

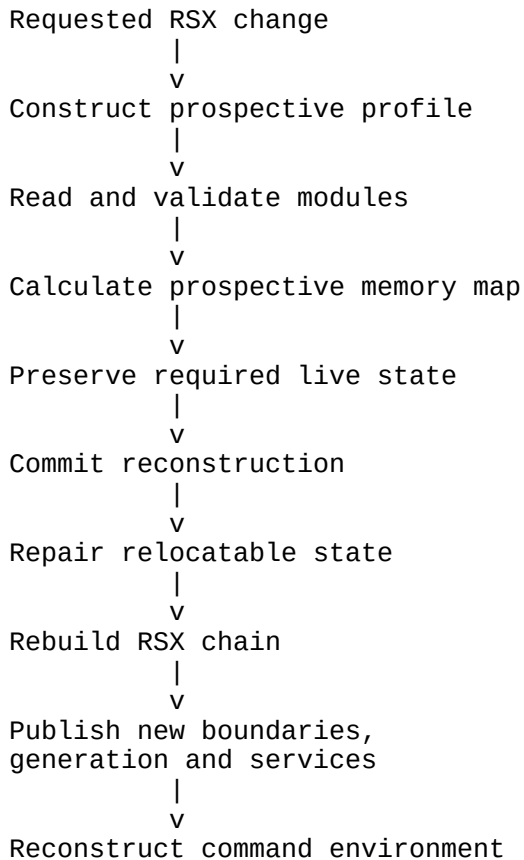
C.2 Warm Boot





Warm boot preserves the active session configuration. It does not reload saved cold-boot defaults.

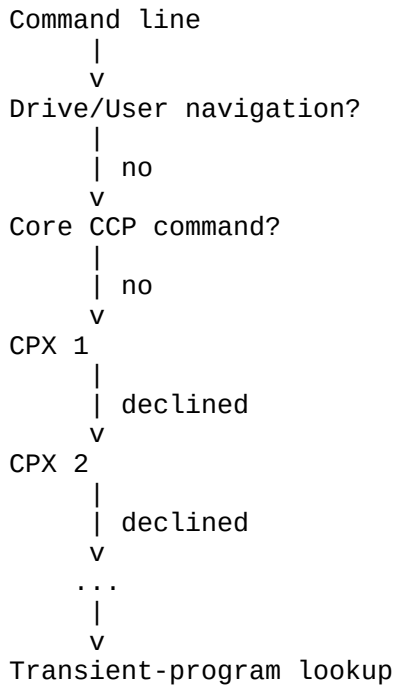
C.3 Resident Reconstruction



Externally visible state is published only after the prospective resident environment has been successfully prepared.

Appendix D. Command Dispatch Sequence

BetterCP/M resolves commands in the following order:



Core CCP commands are:

GET	JUMP	PEEK	P	POKE	GO	SAVE
-----	------	------	---	------	----	------

The standard RCP.CPX provides:

DIR	ERA	TYPE	REN	USER
CLS	VER	COPY	MOVE	

Drive/user navigation is resolved before ordinary command dispatch.

Examples include:

B:
5:
C3:

The first CPX that accepts a command terminates CPX dispatch. If every CPX declines it, BetterCP/M attempts transient-program lookup.

Appendix E. BDOS and Extended-Service Summary

BetterCP/M preserves the CP/M 2.2 BDOS interface and adds explicitly allocated private services without redefining standard CP/M functions.

E.1 BetterCP/M Private Namespace

Function	Service
-----	-----
176	CPX profile control
177	RSX profile control and reconstruction
178	Console/video cursor support
179	Command-environment printer-echo control
180	Protected subsystem version descriptor
181	Disk configuration and utility I/O
182	Callable resident-service registry
183	FDF service
184-196	Unassigned
197	Reserved future escape selector
198-199	Internal test/conformance functions

Functions 198 and 199 are not public application interfaces.

E.2 Historical Compatibility Functions

Function	Interface
-----	-----
200	P2DOS GET TIME
201	P2DOS SET TIME

Functions 200 and 201 are supplied by the optional P2DOS.RSX. They are not BetterCP/M-private assignments.

E.3 Resident-Service Gateway

Function 182 provides:

Operation	Function
-----	-----
0	LOOKUP
1	ENUMERATE
2	DESCRIBE_REGISTRY

The gateway discovers callable resident services; individual service operations are defined by their respective ABIs.

Appendix F. CPX and RSX Summary

Property	CPX	RSX
-----	-----	-----
Primary purpose	Extend command environment	Extend resident system services
Environment	Command processor	Protected resident environment
Present during transient execution	Not necessarily	Yes
Memory class	Reclaimable	Protected
Effect on transient TPA	No permanent TPA charge	Can reduce TPA
Organization	Ordered command dispatch chain	Ordered resident service chain
Can be loaded/unloaded	Yes	Yes
Can move	Yes	Yes, according to reconstruction class
Persistent runtime address	No permanent address	No permanent address
Module carrier	BCPX	BRSX v2
State surviving reconstruction	Must belong to an appropriate longer-lived owner	Governed by RSX reconstruction class

RSX reconstruction classes are:

- STATELESS
- STATEFUL
- STATE_PRESERVING

COLD_ONLY

STATELESS modules are reloaded and initialized.

STATEFUL modules preserve their complete live allocation and repair declared internal pointers.

STATE_PRESERVING reserves a versioned export/import model for modules requiring explicit state transfer.

COLD_ONLY modules cannot move during the active session.

Appendix G. System-State Ownership Summary

State	Owner
-----	-----
Current drive/user	Core system state
Current DMA address	System Services
Active disk bindings	Disk subsystem
Normalized disk state	Disk subsystem
Directory/allocation state	File-system / disk subsystem
Physical-device state	Platform / hardware subsystem
Active CPX profile	Extension / command subsystem
Active RSX profile	Extension subsystem
Resident-service registry	Fixed extension subsystem
RSX private state	Owning RSX
CPX private temporary state	Owning CPX
Command history	Persistent Data Segment
Startup configuration	Configuration subsystem
Clock-provider state	Owning clock-provider RSX
Application state	Current transient application
Stacks	Owning execution subsystem
Temporary workspaces	Owning subsystem or defined

shared-workspace users

The component through which state is inspected or modified is not necessarily its owner.

Shared state has one authoritative representation. Derived or cached state remains valid only while the assumptions from which it was constructed remain valid.

Appendix H. Architectural Invariants

The principal BetterCP/M 1.0 invariants are summarized below.

Compatibility

CP/M-compatible applications execute without requiring BetterCP/M-specific modification.

Compatibility-visible interfaces preserve their adopted CP/M 2.2 semantics.

BetterCP/M extensions do not silently redefine existing CP/M behavior.

Execution

At most one transient application executes at a time.

Transient termination can reestablish a usable Command Environment.

Memory

The ordinary CP/M-compatible 16-bit address space is preserved.

Every memory region has a defined purpose, owner, and lifetime.

Protected resident memory is charged against application capacity.

The default complete-record transient-program floor is 53 KiB.

Components

Major system responsibilities remain explicit.

Hardware-independent components do not unnecessarily depend upon machine-specific implementation details.

Architectural separation does not require unnecessary runtime indirection.

Storage

Logical CP/M storage remains distinct from physical storage.

Each configured logical drive has one authoritative disk interpretation.

A change of medium or format invalidates working state dependent upon the previous interpretation.

State

Every persistent state object has a defined owner and lifetime.

Shared state has one authoritative value.

Continued physical presence in memory does not imply validity.

Extensions

CPXs belong to the reclaimable command environment.

RSXs belong to the protected resident environment.

Runtime address is not persistent module or service identity.

Resident-layout changes are validated and reconstructed transactionally.

Resources

Resident memory, code size, execution time, storage, stacks, workspaces, and complexity remain architectural costs.

Evolution

Published compatibility contracts are not changed accidentally.

Architectural revisions are deliberate and documented.

Appendix I. Glossary

ABI **Application** **Binary** **Interface**
A defined binary contract governing such matters as calling conventions, structures, versions, registers, errors, and object lifetimes.

BDOS **Basic** **Disk** **Operating** **System**
The conventional CP/M application-facing system-service environment. BetterCP/M preserves its compatibility-visible interface while using its own internal implementation.

BCPX
The BetterCP/M module carrier used for Command Processor Extensions.

BRSX
The BetterCP/M module carrier used for Resident System Extensions. BetterCP/M 1.0 production RSXs use BRSX version 2.

CCP **Console** **Command** **Processor**
The core BetterCP/M command processor.

Cold **Boot**
Creation of a new BetterCP/M runtime instance from saved startup configuration.

CPX **Command** **Processor** **Extension**
A relocatable, reconstructible extension to the command environment. CPX memory is reclaimable during transient execution.

DMA **Direct** **Memory** **Access** **Address**
In the CP/M interface, the memory address used for applicable record transfers and related operations. The term retains its historical CP/M meaning and does not imply hardware DMA.

DU **Drive/User**
A BetterCP/M notation identifying a CP/M logical drive and user area as one location.

FCB **File** **Control** **Block**
The CP/M data structure used to identify files and maintain file-operation state.

FDF **Disk** **Format** **Definition**
A BetterCP/M description of a logical CP/M disk format.

Hardware **Abstraction**
The BetterCP/M boundary separating hardware-independent operating-system behavior from machine-dependent implementation.

Logical **Drive**
A CP/M drive in the A: through P: namespace. A logical drive need not correspond one-to-one with a physical storage device.

PDS **Persistent** **Data** **Segment**
A protected BetterCP/M memory area for explicitly defined persistent state. BetterCP/M 1.0 provides a fixed 192-byte PDS owned solely by command history.

Resident **Service**
A callable service supplied by an RSX or suitable fixed resident component and discovered through the BetterCP/M resident-service gateway.

RSX	—	Resident	System	Extension
------------	----------	-----------------	---------------	------------------

A protected resident extension capable of intercepting or extending system services or publishing callable resident services.

Service	ID
----------------	-----------

The four-character logical identifier used to discover a BetterCP/M resident service, such as **TIME**.

System	Services
---------------	-----------------

The hardware-independent operating-system functions supplied to applications and other BetterCP/M components.

TIME

The native BetterCP/M callable date-and-time service.

TPA	—	Transient	Program	Area
------------	----------	------------------	----------------	-------------

The portion of the CP/M-compatible address space available to a transient application.

Transient	Program
------------------	----------------

An application loaded into the TPA, conventionally from a **.COM** file, and executed one at a time under BetterCP/M.

Warm	Boot
-------------	-------------

Return to the running BetterCP/M system without creating a new runtime instance. Valid session state is preserved and the reclaimable command environment can be reconstructed.

Working	State
----------------	--------------

Derived or temporary state valid only while the assumptions from which it was constructed remain true.